

1. Introduzione

Dai sistemi in rete ai sistemi distribuiti

Centralizzato

In un sistema centralizzato c'è un'entità centrale con cui i client comunicano.

Le soluzioni centralizzate non scalano

La root del DNS:

- Centralizzato logicamente.
- Distribuito fisicamente.
- Decentralizzato in diverse organizzazioni

Le soluzioni centralizzate hanno un singolo point of failure

Un singolo point of failure, in realtà, è spesso:

- più facile da gestire -> monitorare tutte le componenti software risulta complicato.
- più facile da rendere robusto.

Decentralizzato

Necessità di tracciare pagamenti (Stripe, Paypal...); i server centrali sono necessariamente costretti a comunicare con altre entità per processare i dati.

Varie entità comunicano con altre per processare dati e comunicare tra loro.

Sistema decentralizzato

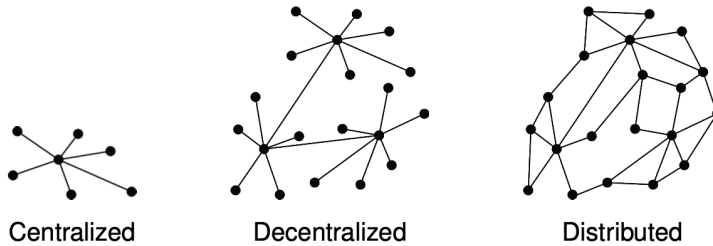
Un **sistema decentralizzato** è un sistema in cui processi e risorse sono *necessariamente* distribuiti su più computer.

Distribuito

Sistemi dove i nodi hanno lo stesso valore.

❶ Sistema distribuito

Un **sistema distribuito** è un sistema informatico in cui i processi e risorse sono *sufficientemente* distribuito su più computer.



Caratteristiche dei sistemi distribuiti

1. **Architettura:** organizzazioni dei sistemi distribuiti di riferimento.
2. **Processi** e le loro relazioni.
3. **Comunicazione** per lo scambio di dati.
4. **Coordinazione** tramite algoritmi indipendenti dall'applicazione.
5. **Naming** per identificazione di risorse (es: DNS).
6. **Consistenza e replicazione** dei dati per renderli sempre gli stessi.
7. **Fault tolerance** per garantire continuità in caso di guasti.
8. **Sicurezza** per l'accesso autorizzato alle risorse.

Tipi di approcci

- **Visione integrativa:** per necessità siamo costretti a comunicare con altre entità (prenotazioni, billing, pagamenti...).
- **Visione espansiva:** sistema informatico in rete esistente viene esteso con computer aggiuntivi (es.: CDN). I contenuti vengono replicati su vari nodi e l'utente viene renderizzato in maniera trasparente al contenuto geograficamente più vicino.

Obiettivi di progettazione

1. **Condivisione delle risorse.**
2. **Trasparenza della distribuzione:** mascherare la distribuzione nello spazio facendo pensare all'utente di comunicare con una sola entità.
3. **Openness:** interfacce ben definite con il quale dialogare (usare l'API a disposizione da un servizio per dialogare con terze parti).
4. **Scalabilità:** adattarsi al bisogno in base al numero di utenti ed altre metriche.

Condivisione delle risorse

Esempi canonici: Storage di file condiviso in Cloud, Streaming multimediali Peer-to-peer, Servizi email condivisi, Servizi di web hosting...

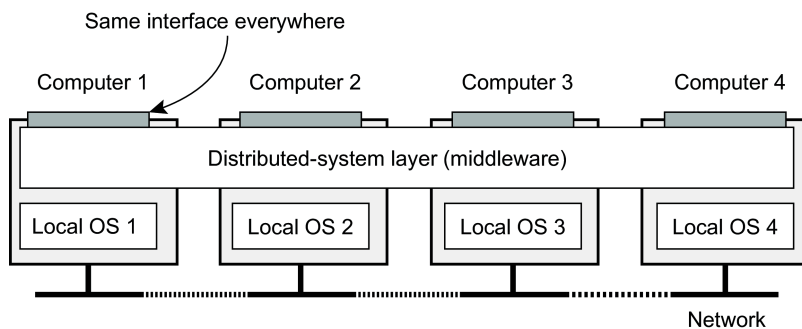
🔗 *The network is the computer.*

Trasparenza della distribuzione

Il fenomeno con cui un sistema distribuito tenta di *nascondere* il fatto che i suoi processi e risorse sono fisicamente distribuiti su più computer, possibilmente separati da grandi distanze.

📌 Middleware

La trasparenza rispetto alla distribuzione viene gestita con tecniche differenti in un layer tra le applicazioni ed i S.O: un layer **middleware**.



Tipi di trasparenza

Tipologia	Descrizione
Accesso	Nascondere le differenze nella rappresentazione dei dati e nel modo in cui si accede a un oggetto
Posizione	Nascondere dove si trova un oggetto
Rilocazione	Nascondere che un oggetto può essere spostato in un'altra posizione durante l'uso
Migrazione	Nascondere che un oggetto potrebbe spostarsi in un'altra posizione
Replicazione	Nascondere che un oggetto è replicato
Concorrenza	Nascondere che un oggetto può essere condiviso da più utenti indipendenti
Failure	Nascondere il fallimento e il ripristino di un oggetto

Gradi di trasparenza

Ci sono latenze di comunicazione che non possono essere nascoste.

Nascondere completamente i guasti è impossibile: non si può distinguere tra un computer lento da

uno guasto e non si può mai essere sicuri che un server abbia effettivamente eseguito un'operazione prima del crash.

La trasparenza totale costerà in termini di performance.

- Mantenere le repliche esattamente aggiornate con il master richiede tempo.
- Svuotare immediatamente le operazioni di scrittura su disco per garantire la tolleranza agli errori.

Esporre la distribuzione potrebbe essere una buona idea a volte: servizi basati sulla localizzazione, differenti fusi orari o quando c'è bisogno di sapere cosa succede su un server (guasti...).

Openness dei sistemi distribuiti

① Open Distributed System

Un sistema che **offre componenti** facilmente utilizzabili o **integrabili in altri sistemi**.

Un sistema distribuito aperto spesso consiste in componenti che provengono da terze parti.

Essere in grado di interagire con servizi di altri sistemi aperti, indipendentemente dall'ambiente sottostante, i sistemi dovrebbero quindi:

- Essere conformi a **interfacce** ben definite.
- Essere facilmente **interoperabili**.
- Supportare la **portabilità** delle applicazioni.
- Dovrebbero essere facilmente **estensibili**.

Policy

- Quale livello di coerenza richiediamo per i dati memorizzati nella cache del client?
- Quali operazioni consentiamo al codice scaricato di eseguire?
- Quali requisiti di QoS (*Quality of Service*) adattiamo in base alle variazioni di larghezza di banda?
- Quale livello di segretezza richiediamo per la comunicazione?

Meccanismi

- Consentire l'impostazione (dinamica) delle policy di caching.
- Supportare diversi livelli di attendibilità per il codice mobile.
- Fornire parametri QoS regolabili per flusso di dati.
- Offrire diversi algoritmi di crittografia.

Osservazione

Quanto più rigorosa è la separazione tra policy e meccanismo, tanto più è necessario garantire meccanismi adeguati, il che potrebbe portare a molti parametri di configurazione e a una gestione complessa.

Le policy hard-coding spesso semplificano la gestione e riducono la complessità, a scapito di una minore flessibilità.

Dipendenza

Un componente fornisce **servizi** ai client. Per far ciò, il componente potrebbe richiedere servizi da altri componenti => un componente potrebbe **dipendere** da altri componenti.

Un componente C dipende da C^* se la **correttezza** del comportamento di C dipende dalla correttezza del comportamento di C^* .

I componenti sono *processi* o *canali*.

Requisiti di dipendenza

1. **Disponibilità**: prontezza all'uso.
2. **Affidabilità**: continuità del servizio.
3. **Sicurezza**: bassa probabilità di catastrofi.
4. **Manutenibilità**: quanto facile è la riparazione di un sistema guasto.

Affidabilità $R(t)$ di un componente

Probabilità condizionata che C abbia funzionato correttamente durante $[0, t)$ dato che C funzionava correttamente al momento $T = 0$.

Metriche tradizionali

- **Mean Time to Failure (MTTF)**: il tempo medio tra i guasti di un componente.
- **Mean Time to Repair (MTTR)**: il tempo medio necessario a riparare il componente.
- **Mean Time Between Failure (MTBF)**: $MTTF + MTTR$.

Fallimento, errore, guasto

- **Fallimento**: un componente non è all'altezza delle sue specifiche (*programma bloccato*).
- **Errore**: parte di un componente che può portare a un guasto (*bug*).
- **Guasto**: causa di un errore (*programmatore sciatto*).

Gestire i guasti

- **Prevenzione**: prevenire il verificarsi di un errore (*non assumere sviluppatori non adatti*).

- **Tolleranza:** costruire un componente e fargli mascherare il verificarsi di un guasto (*far costruire lo stesso componente da due programmatori*).e
- **Rimozione:** ridurre la presenza, numero o serietà di un guasto.
- **Previsione:** stimare la presenza attuale, l'incidenza futura e le conseguenze dei guasti.

Sicurezza

Un sistema distribuito che non è sicuro, non è affidabile.

Abbiamo bisogno di:

- **Confidenzialità** -> informazioni divulgate solo a parti autorizzate.
- **Integrità** -> modifiche da chi è autorizzato.

Autorizzazione, autenticazione, verifica

- **Autenticazione:** verificare la correttezza di un'identità nota.
- **Autorizzazione:** controllo dei diritti di accesso di un'entità.
- **Fiducia:** un'entità può essere certa che un'altra eseguirà determinate azioni secondo una specifica aspettativa.

Meccanismi di sicurezza

Possiamo garantire la sicurezza con **crittografia** e **decrittografia** dei dati utilizzando **chiavi di sicurezza**.

$K(\text{data})$: abbiamo usato la chiave K per crittografare/decrittografare **data**.

Algoritmi simmetrici

Abbiamo una chiave di crittografia $E_k(\text{data})$ e una chiave di decrittazione $D_k(\text{data})$.

Se $\text{data} = D_k(E_k(\text{data})) \implies D_k = E_k$

Le chiavi sono le stesse e vanno mantenute **segrete**.

Algoritmi asimmetrici

Coppia di chiavi: una **chiave pubblica** $PK(\text{data})$ ed una **chiave privata (segreta)** $SK(\text{data})$.

Da Alice a Bob:

$$\underbrace{\text{data} = SK_{\text{bob}}}_{\text{Action by Bob}} \left(\overbrace{PK_{\text{bob}}(\text{data})}^{\text{Sent by Alice}} \right)$$

1. Alice cripta il messaggio con la chiave pubblica di Bob.
2. Bob decritta il messaggio con la sua chiave segreta.

Alice firma il messaggio per Bob:

$$[[data, \underbrace{data \stackrel{?}{=} PK_{alice}(SK_{alice}(data))}_{\text{Check by Bob}}]] = \underbrace{[data, SK_{alice}(data)]}_{\text{Sent by Alice}}$$

Hash

Usiamo una **funzione di hash sicura**: $H(data)$ che ritorna una *stringa di lunghezza fissa*.

Ogni cambiamento nei dati in chiaro produrrà una **stringa diversa**.

Dato un hash, è impossibile tornare indietro al messaggio in chiaro.

Alice firma il messaggio per Bob:

$$[data, \underbrace{H(data) \stackrel{?}{=} PK_{alice}(sgn)}_{\text{Check by Bob}}] = \underbrace{[data, H, sgn = SK_{alice}(H(data))]}_{\text{Sent by Alice}}$$

Scalabilità

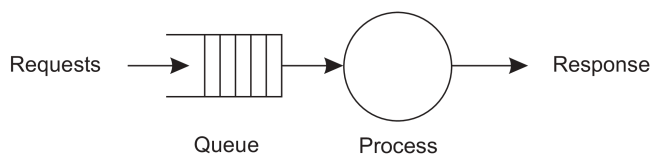
- **Scalabilità delle dimensioni**: numero degli utenti o processi.
- **Scalabilità geografica**: massima distanza tra i nodi.
- **Scalabilità di domini amministrativi**: numero di domini amministrativi.

Scalabilità delle dimensioni

Cause principali dei problemi di scalabilità con soluzioni centralizzate:

- La capacità di calcolo, limitata dalle CPU.
- La capacità di archiviazione, inclusa la velocità di trasferimento tra CPU e dischi.
- La rete tra l'utente e il servizio centralizzato.

Un servizio centralizzato può essere modellato come un semplice sistema di code:



Scalabilità geografica

Non è possibile passare semplicemente dalla LAN alla WAN: molti sistemi distribuiti presuppongono **interazioni client-server sincrone**.

Il client invia una richiesta e attende una risposta. La **latenza** può facilmente impedire questo schema.

La soluzione è sviluppare **servizi di naming e directory** separati (che presentano problemi di scalabilità).

Scalabilità amministrativa

Politiche contrastanti riguardanti l'utilizzo (e quindi il pagamento), la gestione e la sicurezza.

Ad esempio:

- **Griglie computazionali**: condivisione di risorse costose tra diversi domini.
- **Attrezzature condivise**: come controllare, gestire e utilizzare un radiotelescopio condiviso costruito come una rete di sensori condivisa su larga scala?

Eccezione: diverse reti peer-to-peer

- Sistemi File-sharing (*BitTorrent*).
- Telefonia peer-to-peer (*Skype*).
- Audio streaming assistito da peer (*Spotify*).

Nota: collaborano gli utenti finali e non le entità amministrative.

Tecniche per scalare

1. Nascondere le latenze di comunicazione

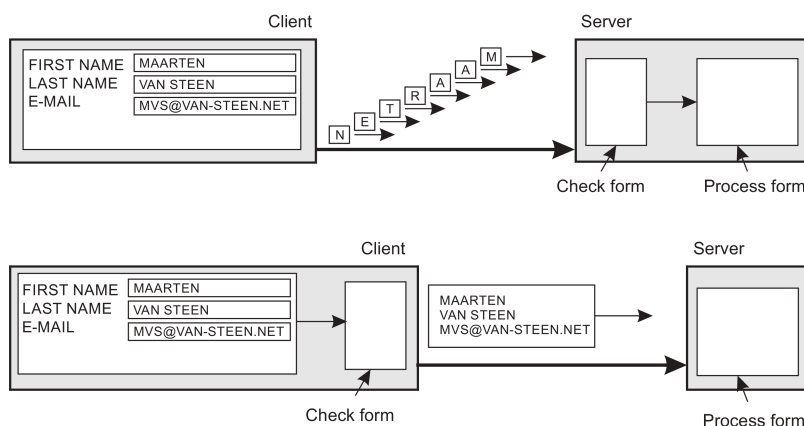
- **Comunicazioni asincrone**: invece di attendere una risposta da un server remoto (bloccando il sistema), l'applicazione continua a svolgere altri compiti.
- Gestori **separati** per le richieste in arrivo.

Tuttavia non tutte le applicazioni si adattano a questo modello.

2. Spostare i calcoli sul client

Molti framework e librerie (come React) processano i dati sul client prima di inviarli al server, magari in formato JSON.

Un esempio è l'invio di un modulo web con codice di validazione (come JavaScript); il controllo dei dati avviene sul dispositivo dell'utente invece di richiedere un viaggio di andata e ritorno verso il server.



3. Partizionare i dati e i calcoli su più macchine

Consiste nel suddividere un componente in parti più piccole e distribuirle su più macchine.

- **Distribuzione Orizzontale:** Si divide un client o un server in parti logicamente equivalenti, dove ciascuna opera sul proprio set di dati, bilanciando così il carico.
- **Esempio (DNS):** Il Domain Name System è strutturato gerarchicamente in zone; la risoluzione di un nome è distribuita tra diversi server sparsi nel mondo, evitando che un unico server debba gestire tutte le richieste.

4. Replicazione e caching

Queste tecniche migliorano le prestazioni posizionando copie dei dati vicino ai processi che li utilizzano, riducendo i tempi di accesso.

- **Caching:** Una forma speciale di replicazione dove la decisione di tenere una copia è presa dal "consumatore" (il client) piuttosto che dal proprietario dei dati.
- **Replicazione:** Il server mantiene attivamente più copie dei dati su diversi nodi per gestire un numero maggiore di richieste e aumentare l'affidabilità.

Il principale svantaggio è che mantenere aggiornate tutte le copie richiede larghezza di banda aggiuntiva e complessi protocolli di coerenza.

Inoltre avendo molte copie si incorre nell'**inconsistenza**: modificarne una la rende diversa dal resto, e mantenerle consistenti richiede una **sincronizzazione globale** (impedendo soluzioni su larga scala).

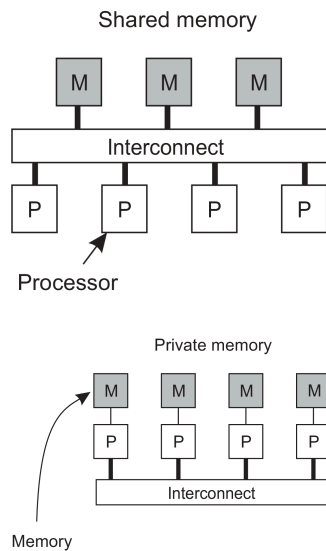
Classificazione dei sistemi distribuiti

Parallel computing

Il calcolo distribuito ad alte prestazioni è iniziato con il calcolo parallelo.

Multiprocessore e multicore vs multicomputer

Caratteristica	Multiprocessore / Multicore	Multicomputer
Memoria	Condivisa (un unico spazio di indirizzamento)	Distribuita (ogni nodo ha la sua memoria locale)
Interazione	Tramite variabili condivise in memoria	Tramite scambio di messaggi in rete
Costo Comunicazione	Molto basso (accesso alla RAM)	Più elevato (latenza di rete)
Esempio	Una moderna workstation con CPU Intel/AMD	Un cluster di server o un supercomputer



Sistemi di memoria condivisa distribuita

I multiprocessori sono relativamente più semplici da programmare in confronto ai multicomputer, ma presentano problemi quando si aumenta il numero di processori (o core).

La soluzione è provare a implementare un **modello a memoria condivisa** su un multicomputer.

≡ Esempio con pagine di memoria virtuale

Mappare tutte le pagine della memoria principale (da processori diversi) in un *unico spazio di indirizzamento virtuale*. Se un processo sul processore A indirizza una pagina P situata sul processore B, il sistema operativo su A intercetta e recupera P da B, proprio come farebbe se P fosse situata sul disco locale.

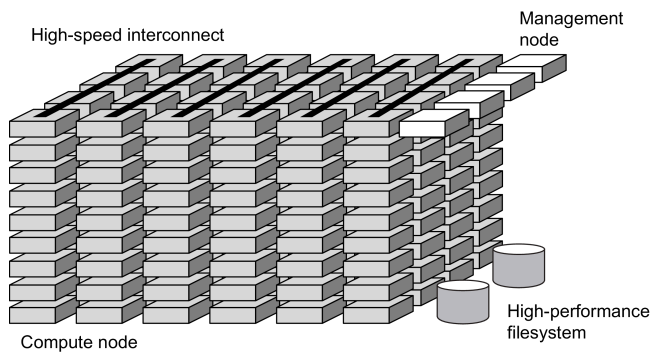
Tuttavia le prestazioni della memoria condivisa distribuita non hanno mai potuto competere con quelle dei multiprocessori e non sono riuscite a soddisfare le aspettative dei programmatori. Ormai è stata ampiamente abbandonata.

Cluster computing

Il **cluster computing** è una forma di calcolo distribuito ad alte prestazioni in cui l'hardware sottostante è costituito da una collezione di **nodi di calcolo simili** (omogenei), collegati tra loro tramite una **rete ad alta velocità**.

- Hanno lo stesso SO e un hardware quasi identico.
- Nodi di gestione responsabili della raccolta dei lavori (job) dagli utenti e della **distribuzione dei compiti** tra i vari nodi di calcolo.

È lo step che ci avvicina al cloud computing.



Grid computing

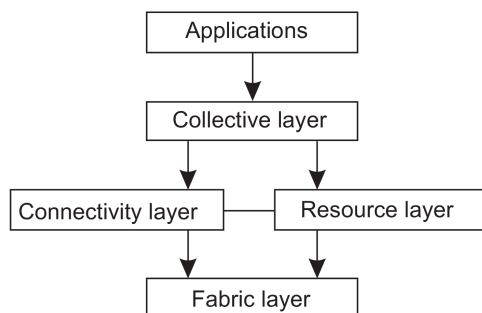
Le università avevano necessità di avere la stessa potenza: l'idea era di federare più cluster unendo più organizzazioni in una unica digitale.

Molti nodi da ogni parte:

- Eterogenei.
- Risorse di diverse organizzazioni vengono messe a fattor comune per permettere la collaborazione tra persone di istituzioni differenti.
- Può facilmente estendersi su una rete WAN.

Per consentire la collaborazione, le griglie generalmente utilizzano **organizzazioni virtuali**. Si tratta di un raggruppamento di utenti (o meglio: dei loro ID) che consente l'autorizzazione all'allocazione delle risorse.

Architettura di una griglia computazionale



- **Fabric:** interfacce per risorse locali (interrogare stato e capacità, blocchi...).
- **Connectivity:** protocolli di comunicazione/transazione (es.: spostamento di dati tra risorse). Anche vari protocolli di autenticazione.
- **Resource:** gestisce una singola risorsa, come la creazione di processi o lettura di dati.
- **Collective:** gestisce l'accesso a più risorse: discovery, scheduling e replicazione.
- **Application:** contiene applicazioni di griglia reali in una singola organizzazione.

Integrazione delle applicazioni

Un'applicazione in rete è un'applicazione che viene eseguita su un **server**, rendendo i suoi servizi disponibili ai **client** remoti.

I client combinano le richieste per (diverse) applicazioni; le inviano; raccolgono le risposte e presentano un risultato coerente all'utente.

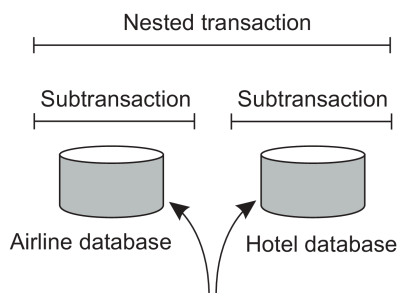
Bisogna permettere la comunicazione diretta tra applicazioni: **Enterprise Application Integration (EAI)**.

Esempio EAI: transazioni (annidate)

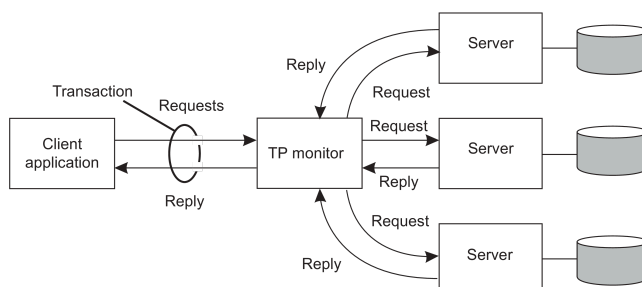
Primitiva	Descrizione
<i>BEGIN_TRANSACTION</i>	Inizio della transazione
<i>END_TRANSACTION</i>	Termina la transazione e prova il commit
<i>ABORT_TRANSACTION</i>	Interrompe la transazione e ripristina i vecchi valori
<i>READ</i>	Legge i dati da un file, tabella...
<i>WRITE</i>	Scrive i dati su un file, tabella...

Le transazioni sono:

- **Atomiche**: accadono indivisibilmente.
- **Consistenti**: non viola gli invarianti del sistema.
- **Isolate**: non c'è interferenza reciproca.
- **Durevoli**: *commit* significa che le modifiche sono permanenti.



TPM: Transaction Processing Monitor

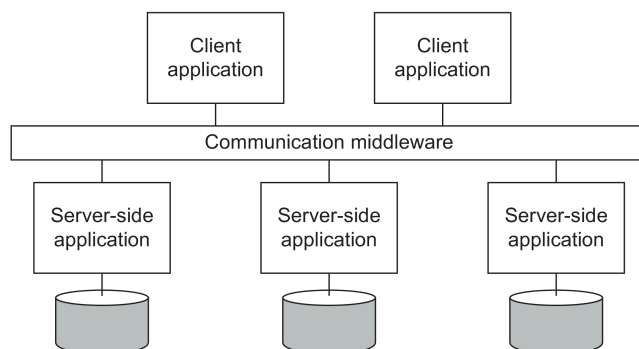


Spesso, i dati coinvolti in una transazione sono distribuiti su più server. Un **monitor TP** è responsabile del coordinamento dell'esecuzione di una transazione.

Lo schema mostra come il TPM separi il client dalle risorse fisiche:

1. Il **Client** invia una richiesta di transazione al TPM.
2. Il **TPM** analizza la richiesta e decide quali **Resource Managers** devono essere coinvolti (possono essere molti e distribuiti su server diversi), usa un *protocollo di transazione distribuita*.
3. Il TPM apre una transazione distribuita e coordina l'esecuzione delle operazioni su ciascun database.

Middleware ed EAI



Il **middleware** offre funzionalità di comunicazione per l'integrazione.

Esso è lo strato di software che si posiziona tra le applicazioni e i sistemi operativi sottostanti. Il suo scopo è mascherare l'eterogeneità dei nodi e fornire un'interfaccia unificata.

Il middleware permette a diverse applicazioni di comunicare attraverso protocolli specifici come:

- [Remote Procedure Call \(RPC\)](#) -> Le richieste vengono inviate tramite una chiamata di procedura locale, confezionate come messaggio, elaborate, ricevono risposta tramite messaggio e il risultato viene restituito come risultato della chiamata.
- [Message Oriented Middleware \(MOM\)](#) -> I messaggi vengono inviati al punto di contatto logico (pubblicati) e inoltrati alle applicazioni sottoscritte.

Come integrare le applicazioni

- **File transfer**: semplice ma non flessibile, bisogna definire il formato e il layout dei file, definire la gestione dei file e propagare degli aggiornamenti con notifiche.
- **Database condiviso**: richiede comunque uno schema dati comune, oltre al rischio di colli di bottiglia.
- **RPC**: efficace quando si ha bisogno di eseguire una serie di azioni.
- **Messaging**: le RPC richiedono che il chiamante e il destinatario siano attivi e operativi contemporaneamente. La messaggistica consente il disaccoppiamento nel tempo e nello spazio.

Sistemi distribuiti pervasivi

Pervasive Systems

Sistemi distribuiti di nuova generazione in cui i nodi sono piccoli, mobili e spesso integrati in un sistema più ampio, caratterizzati dal fatto che **il sistema si integra naturalmente nell'ambiente dell'utente**.

Ci sono vari tipi di sistemi distribuiti pervasivi:

- **Sistemi di elaborazione ubiquitaria:** pervasivi e costantemente presenti, ovvero c'è un'interazione continua tra sistema e utente.
- **Sistemi di elaborazione mobile:** pervasivi, i dispositivi sono intrinsecamente mobili.
- **Reti di sensori (e attuatori):** pervasivi, con enfasi sull'effettiva rilevazione e attuazione (collaborativa) dell'ambiente.

Sistemi di elaborazione ubiquitaria

1. I dispositivi sono collegati in rete, **distribuiti** e accessibili in modo trasparente.
2. L'**interazione** tra utenti e dispositivi è estremamente discreta.
3. Il sistema è **consapevole del contesto** dell'utente per ottimizzare l'interazione.
4. I dispositivi operano in modo **autonomo** senza intervento umano e sono quindi altamente autogestiti.
5. Il sistema è **intelligente**: nel suo complesso può gestire un'ampia gamma di azioni e interazioni dinamiche.

Sistemi di elaborazione mobile

Caratteristiche distinte:

- Una miriade di dispositivi mobili diversi (smartphone, tablet, dispositivi GPS, telecomandi, badge attivi).
- Mobile implica che la posizione di un dispositivo possa cambiare nel tempo $\Rightarrow$ cambiamento dei servizi locali, della raggiungibilità, ecc. (**discovery**).
- Mantenere una comunicazione stabile può comportare seri problemi.
- Per molto tempo, la ricerca si è concentrata sulla condivisione diretta delle risorse tra dispositivi mobili. Questa strategia non ha mai avuto successo ed è ormai considerata una strada infruttuosa per la ricerca.

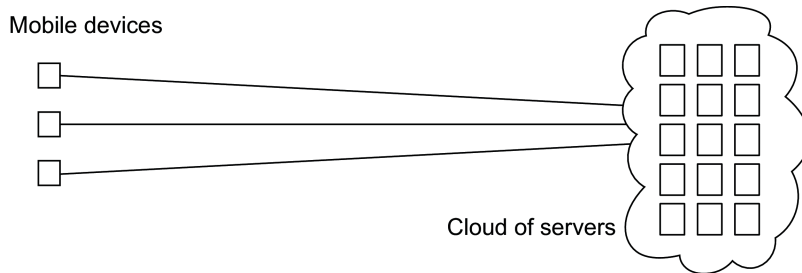
I dispositivi mobili stabiliscono connessioni con server fissi, portando essenzialmente il mobile computing nella posizione di client di servizi basati su cloud.

Mobile cloud computing

L'elaborazione e la memorizzazione avvengono in **data center remoti** (il "cloud").

Praticamente ci sono risorse computazionali illimitate, ma con latenza dovuta alla distanza fisica e agli

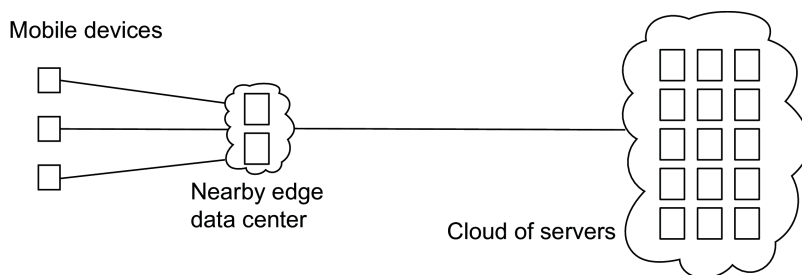
hop di rete necessari per raggiungere il server centrale.



Mobile edge computing

L'elaborazione avviene al **"bordo" della rete**, ovvero su server posizionati molto vicino all'utente (ad esempio, direttamente nelle stazioni radio base della rete 5G o in piccoli micro-data center locali).

La latenza è minimizzata e c'è un risparmio di banda sulla rete principale, ma le risorse computazionali sono limitate rispetto ad un intero data center.



Reti di sensori

Una rete di sensori è una collezione di nodi che collaborano per **monitorare e influenzare l'ambiente** circostante.

I nodi a cui sono collegati i sensori sono:

- *Molti* (decine-migliaia).
- *Semplici* (capacità di memoria/elaborazione/comunicazione ridotte).
- Spesso *alimentati a batteria* (o addirittura senza batteria)

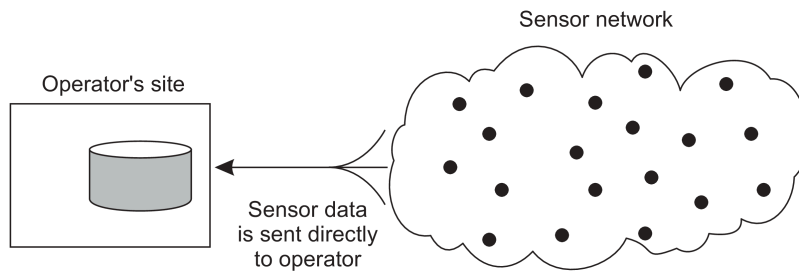
Direct Diffusion

Si basa su un meccanismo di pubblicazione e sottoscrizione ([publish-subscribe](#)) per connettere i dati ai nodi interessati.

Un nodo "sink" (il destinatario) diffonde un **"interesse"** nella rete, specificando il tipo di dati richiesti (es. "inviarmi letture di temperatura $> 30^{\circ}\text{C}$ ").

Man mano che l'interesse si propaga, i nodi creano dei "gradienti" che puntano verso il destinatario. Quando un sensore rileva l'evento richiesto, invia i dati lungo questi percorsi. Il sistema "rinforza" i percorsi più efficienti (quelli con minore latenza o maggiore affidabilità).

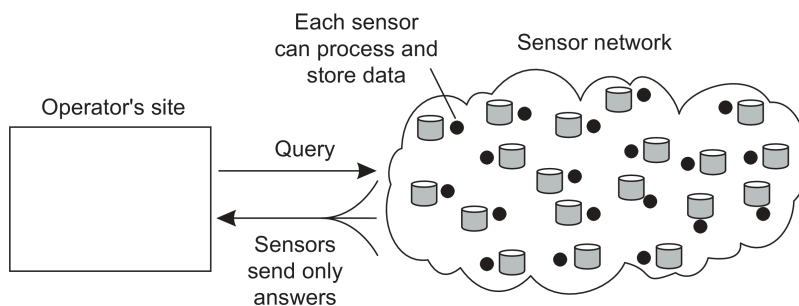
Ottimizza dinamicamente i percorsi di comunicazione basandosi sulla disponibilità effettiva dei dati richiesti, rendendo il sistema molto resiliente ai guasti dei singoli nodi.



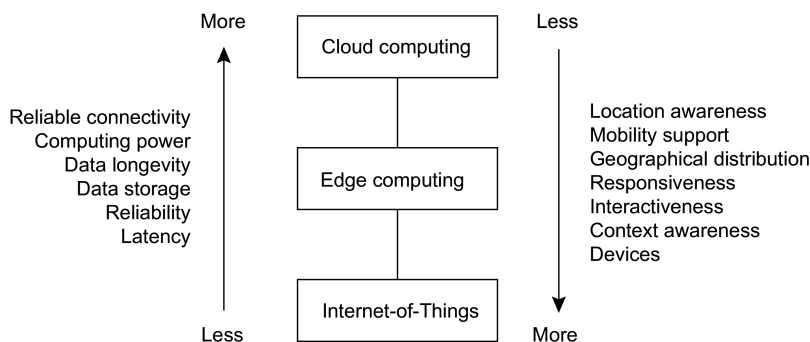
Modello TinyDB

La rete di sensori è vista come un'unica tabella di database virtuale dove ogni sensore contribuisce con una riga di dati (es. temperatura, luce, umidità).

Invece di inviare tutti i dati a un server centrale, le query (es. in linguaggio simile a SQL) vengono iniettate nella rete. I nodi sensore filtrano e aggregano i dati localmente prima di inoltrare solo il risultato necessario. Riduce drasticamente il traffico di rete, poiché viaggiano solo le risposte aggregate e non le letture individuali di ogni sensore.



Il continuum tra nuvole e margini



- **Edge (Il Bordo):** Qui si trovano i sensori e i dispositivi IoT. L'elaborazione è immediata ma limitata. Serve per risposte in tempo reale (es. frenata automatica di un'auto).
- **Fog (Nebbia):** È lo strato intermedio. Include router potenziati, micro-data center locali o stazioni base 5G. Qui si aggregano i dati provenienti da molti sensori prima di mandarli al cloud.
- **Cloud (Il Centro):** Sono i grandi server (AWS, Google Cloud). Hanno potenza infinita ma sono lontani. Qui si fa l'analisi storica e il "big data".

Non c'è una divisione netta. Un compito può essere spostato (*offloaded*) lungo questa scala a seconda della necessità: se serve velocità si sta sull' **Edge**, se serve potenza si va sul **Cloud**.

Insidie nello sviluppo di sistemi distribuiti

Molti sistemi distribuiti sono inutilmente complessi, causati da errori ed *assunzioni errate*:

- La rete è affidabile.
 - La rete è sicura.
 - La rete è omogenea.
 - La topologia non cambia.
 - La latenza è zero.
 - La larghezza di banda è infinita
 - Il costo di trasporto è zero.
 - C'è un solo amministratore.
-

Progettazione del sistema di Netflix

L'obiettivo principale è fornire contenuti video in streaming a milioni di utenti globali con bassa latenza e alta disponibilità, indipendentemente dal dispositivo o dalla qualità della connessione.

Requisiti funzionali

- Video streaming istantanei.
- Un'esperienza di visualizzazione uniforme e fluida, indipendentemente dal numero di utenti online o dall'ora del giorno.
- Accessibilità tra una varietà di dispositivi.

Requisiti non funzionali

- 100M Daily Active Users (DAU).
- Ogni utente guarda 1h di video al giorno.
- Mantenere i dati per 10 anni.
- Un peso medio di 500MB per ogni video.
- Bassa latenza

Pianificazione delle capacità

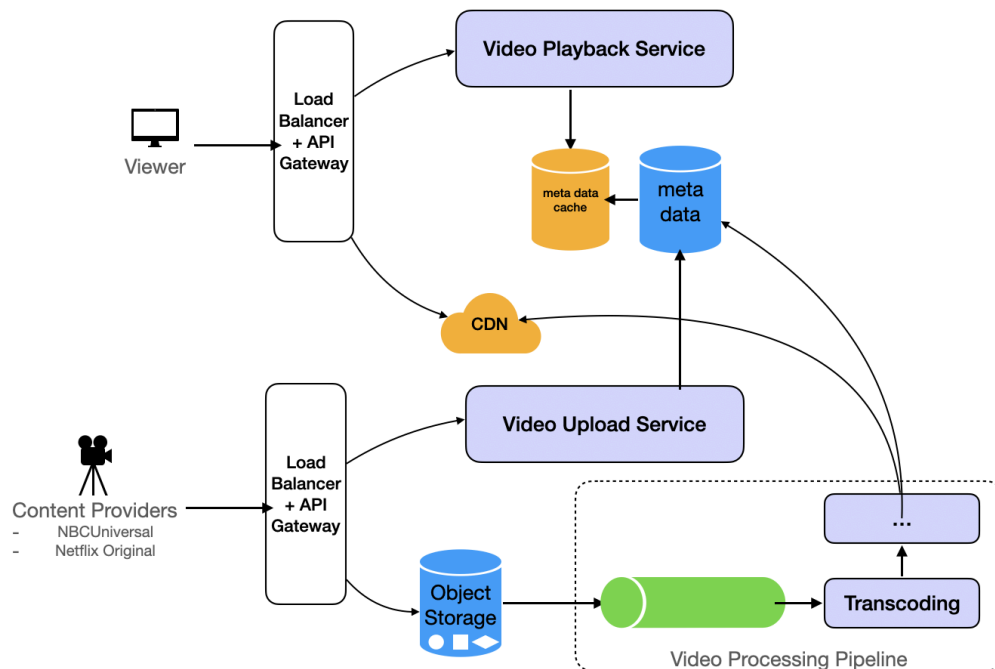
Stimiamo intorno alle 70.000 QPS (**Query Per Second**) -> 60 richieste per utente * 100.000.000 utenti. Dobbiamo tenere conto almeno del doppio, quindi circa 140.000 QPS.

Per quanto riguarda le **esigenze di archiviazione**: con una dimensione media dei video di 500 MB, se Netflix pubblica 1.000 nuovi video al giorno, l'aumento giornaliero dello spazio di archiviazione è:

$500\text{MB/video} * 1,000 \text{ videos} = 500,000\text{MB}$ or 500GB per day

Progettazione di alto livello

Ci basiamo su un diagramma di alto livello che delinea l'interazione tra i servizi.



- **API Gateway:** l'entry point per tutte le richieste dei client.
- **Video Playback Service:** gestisce lo streaming dei video e indirizza le richieste allo storage o CDN.
- **Object Storage:** salva i file video.
- **CDN (Content Delivery Network):** distribuisce il contenuto video agli utenti per minimizzare la latenza.

Progettazione dettagliata

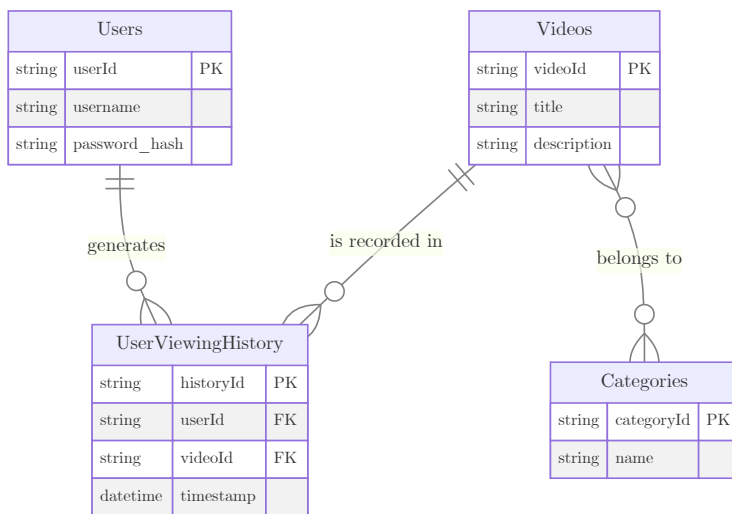
API Design

- **GET /videos** - Restituisce la lista di video.
- **POST /videos** - Uploads di un nuovo video.
- **GET /videos/{videoId}/play** - Streams di un video.
- **POST /users/{userId}/history** - aggiorna l'history di un utente.

Per ogni endpoint abbiamo una risposta JSON:

```
// Sample API Request to GET /videos
{
  "method": "GET",
  "url": "/videos",
  "query": {
    "genre": "action",
    "language": "en"
  }
}

// Sample API Response
{
  "status": 200,
  "data": [
    {
      "id": "1234567890",
      "title": "Exciting Action Movie",
      "description": "An action-packed journey."
    },
    ...
  ]
}
```



2. Architetture

Stili architetturali

Uno stile architetturale è composto da:

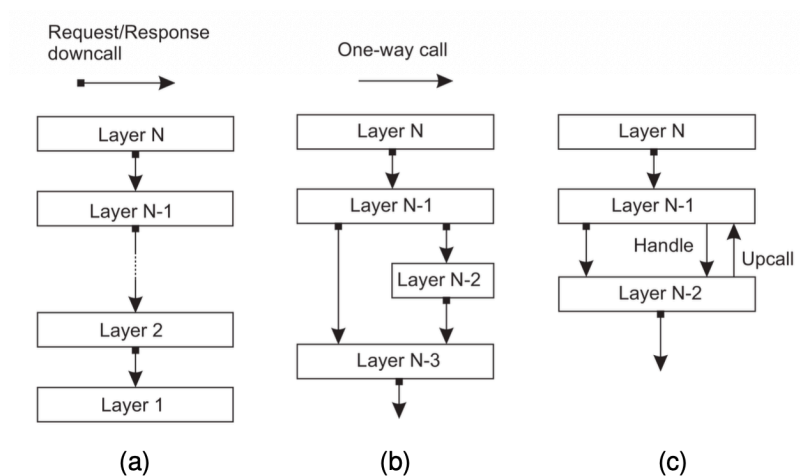
1. **Componenti** (riutilizzabili) con interfacce ben definite.
2. Il modo in cui i componenti sono **connessi**.
3. I **dati** scambiati.
4. La **configurazione** dei componenti e connettori in un sistema.

📘 Connector

Un meccanismo che media la **comunicazione**, il **coordinamento** o la **cooperazione** tra i componenti.

Esempio: funzionalità per chiamate di procedura (remote), messaggistica o streaming.

Architettura a strati



L'idea di base è la **separazione delle responsabilità**:

- Ogni **strato** (Layer L_i) contiene **componenti** che offrono servizi allo strato superiore (L_{i+1}).
- Lo strato superiore vede solo l'**interfaccia** (il "cosa" fa lo strato sotto) ma non vede l'implementazione (il "come" lo fa).

Direzione delle chiamate

- **Richiesta (Request):** Parte dall'alto verso il basso. Un'applicazione o un utente interagisce con lo strato più esterno, che a sua volta delega il lavoro agli strati inferiori.
- **Risposta (Response):** Una volta elaborata l'operazione (spesso toccando i dati o l'hardware alla base), il risultato risale la piramide fino all'utente.

Strati standard nei sistemi distribuiti

Strato	Nome	Funzione
Top	User Interface	Gestisce l'interazione con l'utente (GUI, Browser).
Middle	Processing/Business Logic	Contiene il "cuore" dell'applicazione e le regole decisionali.
Bottom	Data Layer	Gestisce la persistenza dei dati (Database, File System).

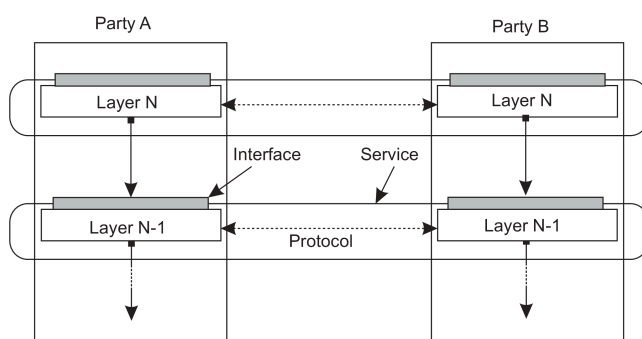
Vantaggi

- **Modularità:** Puoi cambiare il database (strato inferiore) senza dover riscrivere l'interfaccia grafica (strato superiore), a patto che l'interfaccia tra i due resti invariata.
- **Facilità di test:** Ogni strato può essere testato indipendentemente.

Svantaggi

- **Performance:** Ogni richiesta deve attraversare tutti gli strati ("pass-through"), il che può aggiungere latenza.
- **Rigidità:** A volte un cambiamento in alto richiede modifiche a cascata su tutti i livelli.

Esempio: protocolli di comunicazione



Il **servizio** definisce cosa lo strato fa per quello superiore.

È un insieme di operazioni che lo strato L_i offre allo strato L_{i+1} .

Il servizio dice: "Cara applicazione, io posso inviare i tuoi dati in modo affidabile", ma non ti dice come ci riesce.

È un concetto **verticale**: lo strato inferiore fornisce il servizio a quello superiore.

Il **protocollo** è un concetto **orizzontale**.

È l'insieme di regole che governano la comunicazione tra "entità paritarie" (*peer entities*) che si trovano su **macchine diverse** ma allo **stesso livello**.

Il protocollo definisce il formato e il significato dei messaggi scambiati per realizzare il servizio.

Esempio: Due strati di trasporto su due PC diversi usano il protocollo TCP per mettersi d'accordo su quali pacchetti sono arrivati e quali no.

Comunicazione a due parti

```
1 from socket import *
2
3 s = socket(AF_INET, SOCK_STREAM)
4 (conn, addr) = s.accept() # returns new socket and addr. client
5 while True:               # forever
6     data = conn.recv(1024) # receive data from client
7     if not data: break     # stop if client stopped
8     msg = data.decode()+"*" # process the incoming data into a response
9     conn.send(msg.encode()) # return the response
10    conn.close()           # close the connection
```

(a) A simple server

```
from socket import *
.
.
s = socket(AF_INET, SOCK_STREAM)
s.connect((HOST, PORT)) # connect to server (block until accepted)
msg = "Hello World"     # compose a message
s.send(msg.encode())    # send the message
data = s.recv(1024)     # receive the response
print(data.decode())    # print the result
s.close()               # close the connection
```

(b) A client

Layering applicativo

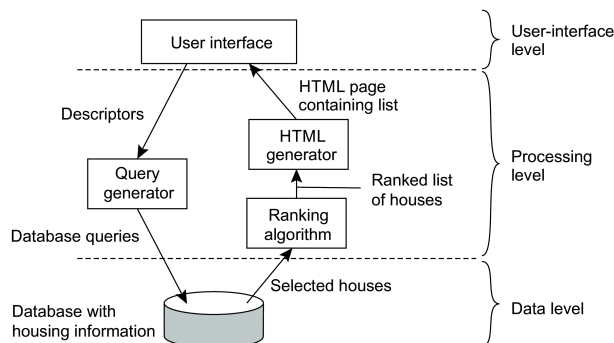
Visione tradizionale a tre livelli

1. **Application-interface layer:** contiene unità per l'interfacciamento con gli utenti applicazioni esterne.
2. **Processing layer:** contiene le funzioni di un'applicazione, ovvero senza dati specifici.
3. **Data layer:** contiene i dati che un client desidera manipolare tramite i componenti dell'applicazione.

Questa stratificazione è presente in molti sistemi informativi distribuiti, che utilizzano la tecnologia tradizionale dei database e le relative applicazioni.

Esempio: search engine

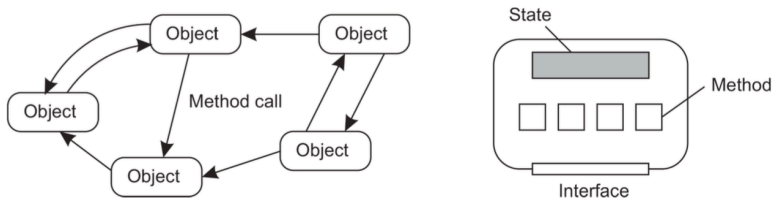
1. **UI level:** gestisce la presentazione dei dati e l'interazione con l'utente. Trasforma i comandi dell'utente in richieste per il livello successivo (*browser*).
2. **Processing level:** contiene la logica, regole di business e calcoli. Decide cosa deve succedere quando un utente preme un tasto. Riceve input dal layer superiore, richiede i dati necessari al layer inferiore, li elabora e restituisce il risultato.
3. **Data level:** gestisce la memorizzazione ed il recupero delle informazioni su DB o file system. Garantisce la coerenza, accessibilità e sicurezza dei dati (*MySQL*, *PostgreSQL*).



Object-based style

I componenti sono **oggetti**, collegati tra loro tramite chiamate di procedura.

Gli oggetti possono essere posizionati su macchine diverse; le chiamate possono quindi essere eseguite attraverso una rete.



❶ Incapsulamento

Si dice che gli oggetti **incapsulano i dati** e offrono **metodi** su tali dati senza rivelare l'implementazione interna.

Architetture RESTful

Considera un sistema distribuito come un insieme di risorse, gestite individualmente dai componenti. Le risorse possono essere aggiunte, rimosse, recuperate e modificate da applicazioni (remote).

1. Le **risorse** sono identificate tramite un unico schema di denominazione.
2. Tutti i servizi offrono la **stessa interfaccia**.
3. I messaggi inviati da o verso un servizio sono completamente **autodescritti**.
4. Dopo aver eseguito un'operazione su un servizio, il componente **dimentica** tutto ciò che riguarda il chiamante.

Operation	Descrizione
POST	Crea una nuova risorsa
GET	Restituisce lo stato di una risorsa in qualche rappresentazione
DELETE	Elimina una risorsa
PUT	Modifica una risorsa trasferendo un nuovo stato

Mappiamo le funzioni che andiamo ad usare sulle risorse, in cui ogni metodo HTTP indica un'azione che possiamo fare su una risorsa.

Interfacce

Molti apprezzano gli approcci RESTful perché l'interfaccia con un servizio è molto semplice. Il problema è che c'è molto da fare nello spazio dei parametri.

Mentre con SOAP bisogna usare Java o Python, con REST si possono fare delle semplici chiamate CURL (*bash*), passando anche header.

Coordinazione

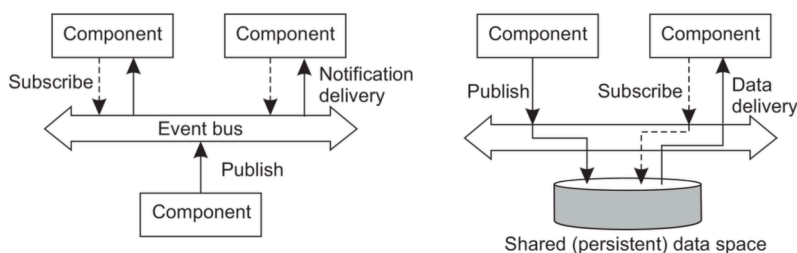
Accoppiamento temporale e referenziale

	Accoppiato temporalmente	Disaccoppiato temporalmente
Accoppiato referenzialmente	Diretto	Mailbox
Disaccoppiato referenzialmente	Event-based	Spazio dati condiviso

1. I processi devono conoscersi per nome (accoppiamento referenziale) e devono essere entrambi attivi contemporaneamente per comunicare (accoppiamento temporale).
Una classica chiamata di procedura remota (**RPC**) o una comunicazione tramite **Socket**. Se il server non è attivo quando il client invia la richiesta, la comunicazione fallisce. Il client deve inoltre conoscere l'indirizzo o l'ID esatto del server.
2. I processi non hanno bisogno di conoscersi esplicitamente (disaccoppiamento referenziale), ma devono essere entrambi in esecuzione nello stesso momento per scambiarsi informazioni (accoppiamento temporale).
Il mittente lancia un **evento**; chi è online in quel momento lo riceve, ma il mittente non sa chi siano.
3. Mittente e destinatario non devono essere attivi contemporaneamente (disaccoppiamento temporale), ma devono comunque concordare su un "luogo" specifico (la mailbox) dove scambiarsi i messaggi (accoppiamento referenziale).
Nelle **mailbox** ho bisogno di un riferimento (indirizzo del destinatario), ma non è necessario che i client siano app per far avvenire la comunicazione.
4. I processi non sanno chi sono gli altri (disaccoppiamento referenziale) e non devono essere attivi nello stesso momento (disaccoppiamento temporale). Si limitano a inserire o prelevare dati da uno spazio comune.
Una bacheca digitale o un sistema come **JavaSpaces**. Un processo "A" scrive un risultato in uno spazio comune; due giorni dopo, un processo "B" (che "A" non conosce) passa di lì, vede il dato e lo elabora.

Event-based e spazio dati

Questi modelli fanno parte degli stili di architettura **publish-subscribe**, che mirano a separare le attività di elaborazione da quelle di coordinamento (comunicazione e cooperazione).



- **Publish** -> un componente genera i dati e li pubblica sul bus.
- **Subscribe** -> chi è interessato a ricevere gli aggiornamenti.

- **Spazio di persistenza** -> salva gli eventi in modo che il componente che non ha ricevuto la notifica può fare una query per controllare gli eventi di interesse.

Linda tuple space

Metodo di programmazione in cui tutti i dati scambiati sono tuple. La comunicazione avviene tramite tuple ed espone solo tre metodi:

- `out(t)` : aggiunge una tupla `t` nel tuple space.
- `rd(t)` : ottiene una copia della tupla che corrisponde al template `t`.
- `in(t)` : rimuove una tupla che corrisponda al template `t`.

Chiamando `out(t)` due volte di seguito, si ottengono due copie della tupla `t` => uno spazio di tuple è modellato come un **multiset**.

Sia `in` che `rd` sono **bloccanti**: il chiamante sarà bloccato finché una tupla corrispondente non verrà trovata, o diventa disponibile.

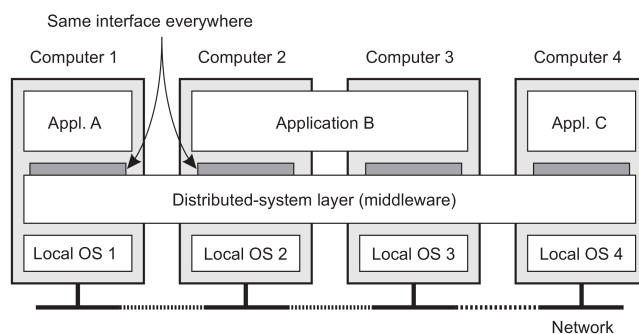
Publish-subscribe

Gli eventi sono descritti da una coppia (`attributo`, `valore`), ci sono due metodi per abbinare gli eventi:

1. **topic-based subscription**: specificare una serie di `attributo = valore`.
2. **content-based subscription**: specificare una serie di `attributo ∈ range`.

Tuttavia il secondo metodo ha problemi di scalabilità: più una descrizione può essere espressiva, più difficile sarà verificare se un evento corrisponde a una descrizione.

Middleware



Abbiamo bisogno di un **middleware** che ci fornisce delle componenti e funzioni già scritte che non devono essere implementati separatamente dalle applicazioni.

Tra i vari vantaggi abbiamo che:

- Sono distribuiti orizzontalmente su più macchine.

- Nasconde i SO sottostanti su cui viene eseguito.
- Possono esserci più applicazioni che girano nello stesso cluster.

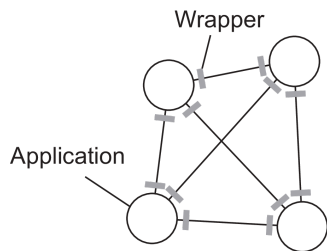
Utilizzo di componenti legacy nei middleware

Le interfacce offerte da un componente legacy molto probabilmente *non sono adatte a tutte le applicazioni*.

I **wrapper** (o **adapter**) sono metodologie software che interfacciano sistemi con lingue diverse.

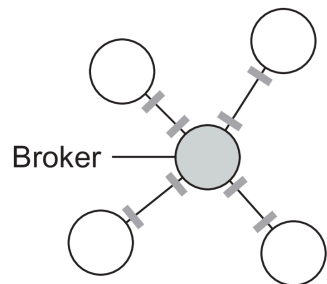
Organizzare i wrappers

Soluzione 1 a 1



Se dobbiamo sviluppare N applicazioni, dobbiamo creare: $N \cdot (N - 1) = O(N^2)$ wrappers.

Utilizzando un broker



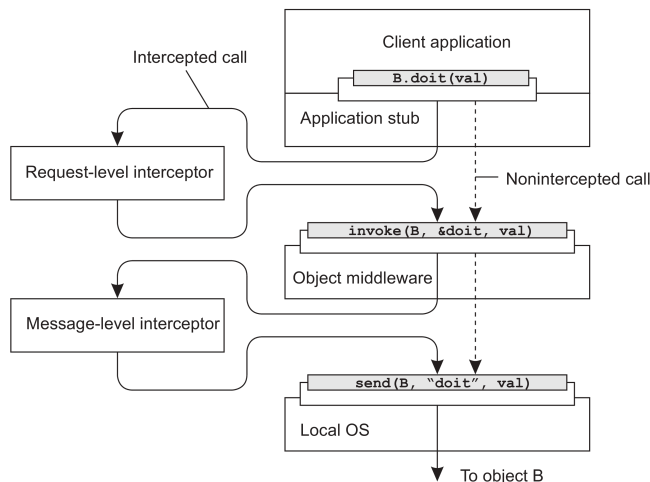
Il **broker** è un adapter centralizzato che parla con le varie applicazioni, riducendo il numero massimo di wrapper da scrivere. Tuttavia esso è un altro componente da mantenere.

Abbiamo bisogno $2N = O(N)$ wrappers.

Approfondimento

Vedi: [Node.js nei sistemi distribuiti](#).

Intercettare il normale flusso di controllo



In questo caso viene simulata una chiamata ad un oggetto remoto su tre livelli.

Dopo il primo passaggio, la chiamata `B.doit(val)` viene trasformata in una chiamata generica, come `invoke(B,&doit,val)` con un riferimento al metodo di B e ai parametri che accompagnano la chiamata. Ora immaginiamo che l'oggetto B venga replicato. In tal caso, ogni replica dovrebbe essere effettivamente invocata. Questo è un chiaro punto in cui l'intercettazione può essere utile.

Ciò che farà l'**intercettore a livello di richiesta** sarà semplicemente chiamare `invoke(B,&doit,val)` per ciascuna delle repliche. Il bello di tutto ciò è che l'oggetto A non deve essere a conoscenza della replica di B, ma anche il middleware dell'oggetto non deve avere componenti speciali che gestiscano questa chiamata replicata. Solo l'intercettore a livello di richiesta, che può essere aggiunto al middleware, deve essere a conoscenza della replica di B.

Alla fine, una chiamata a un oggetto remoto dovrà essere inviata attraverso la rete. In pratica, ciò significa che sarà necessario invocare l'interfaccia di messaggistica offerta dal sistema operativo locale.

A quel livello, un **intercettore a livello di messaggio** può aiutare a trasferire l'invocazione all'oggetto di destinazione.

Ad esempio, immaginiamo che il parametro `val` corrisponda effettivamente a un enorme array di dati. In tal caso, potrebbe essere opportuno frammentare i dati in parti più piccole per poi riassemblarli a destinazione. Tale frammentazione potrebbe migliorare le prestazioni o l'affidabilità. Anche in questo caso, il middleware non deve essere a conoscenza di questa frammentazione; l'intercettore di livello inferiore gestirà in modo trasparente il resto della comunicazione con il sistema operativo locale.

Architetture di sistemi a strati

Modello client-server

I processi che offrono servizi sono detti **servers**, i processi che usano servizi sono detti **clients**.

Client e server possono essere su macchine diverse.

I client seguono il *modello richiesta/risposta* quando usano i servizi: il client richiede un'operazione al server ed aspetta la risposta da esso.

Architetture centralizzate a più livelli

Ci sono varie configurazioni per i tre livelli tradizionali: UI, processing layer e gestione dei dati.

Single-tiered

Tutto il sistema eseguito su **una sola macchina**.

Semplice da sviluppare, con alte prestazioni locali e nessuna latenza.

Tuttavia non è scalabile e non adatto a sistemi distribuiti.

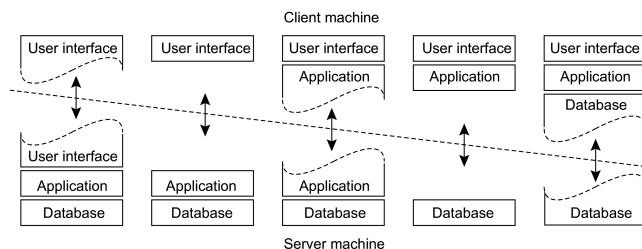
Two-tiered

Il sistema è diviso in due livelli:

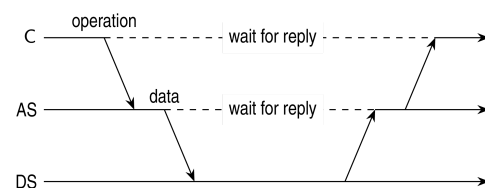
1. **Client** -> interfaccia + parte logica.
2. **Server** -> gestione dei dati (database).

Più scalabile del single-tier, in quanto c'è la separazione tra dati ed interfaccia e più utenti possono accedere allo stesso DB.

La logica viene tuttavia duplicata su più client, facendolo diventare pesante, e c'è scarsa scalabilità lato server.



Three-tiered



Il sistema viene diviso in tre livelli:

1. **Client (C)** -> presentation layer.
2. **Application server (AS)** -> application layer.
3. **DB Server (DS)** -> data layer.

C richiede operazioni all'AS, AS recupera i dati da DS e risponde a C.

Si ha alta scalabilità, logica centralizzata, client leggero, manutenzione migliore e sicurezza del DB migliore (non accessibile direttamente) .

Ovviamente si aggiunge complessità, comunicazioni in rete e maggiore latenza.

Esempio: Network File System

Ogni server NFS fornisce una vista standardizzata del proprio file system locale: ogni server supporta lo stesso modello, indipendentemente dall'implementazione del file system.

1. Il client richiede al server di accedere ad un **file remoto**, localizzato nel server (Remote access model).
2. Il file viene **scaricato** e aperto dal client.
3. Quando il client ha effettuato le modifiche, il file viene **restituito** al server (modello upload/download).

Il servizio *FTP* può essere utilizzato quando un client scarica un file completo, lo modifica e poi lo restituisce.

Un client accede al file system utilizzando le chiamate di sistema fornite dal suo sistema operativo locale. Tuttavia, l'interfaccia del file system Unix locale viene sostituita da un'interfaccia al **Virtual File System (VFS)**, che ormai è uno standard de facto per l'interfacciamento con diversi file system (distribuiti).

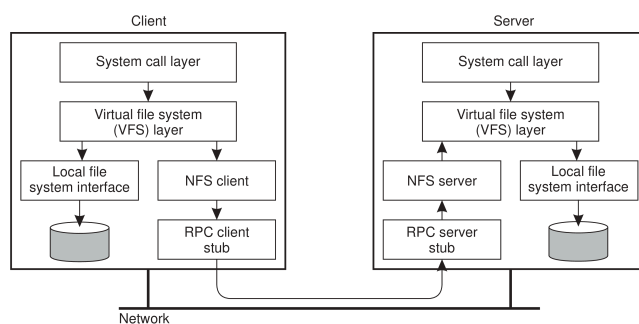
Con NFS, le operazioni sull'interfaccia VFS vengono passate a un file system locale o a un componente separato noto come **NFS client**, che si occupa di gestire l'accesso ai file memorizzati su un server remoto. In NFS, tutte le comunicazioni client-server avvengono tramite le cosiddette **chiamate di procedura remota (RPCs)**.

Le operazioni offerte dall'interfaccia VFS possono essere diverse da quelle offerte dal client NFS. L'idea alla base del VFS è quella di nascondere le differenze tra i vari file system.

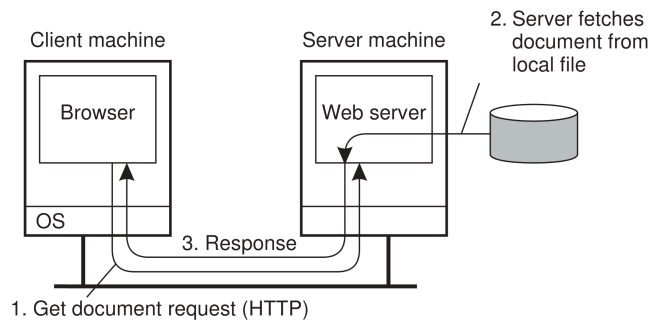
Il **server NFS** è responsabile della gestione delle richieste client in arrivo. Il componente RPC sul server converte le richieste in arrivo in normali operazioni sui file VFS che vengono successivamente passate al livello VFS.

Anche in questo caso, il VFS è responsabile dell'implementazione di un file system locale in cui vengono archiviati i file effettivi.

L'unico aspetto importante è che questi file system sono compatibili con il modello di file system offerto da NFS.



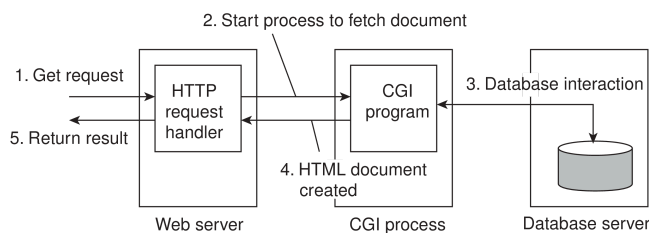
Esempio: Web server



All'inizio del web un sito web era una collezione di file HTML referenziati tra loro da **hyperlink**. Un web server necessitava di un solo hyperlink per recuperare un file.

Ai giorni d'oggi un sito web è costruito grazie ad un database: una pagina può ancora essere referenziata tramite hyperlink, ma un programma separato (**Common Gateway Interface, CGI**) compone la pagina.

Il browser si accerta del corretto rendering del contenuto di un file.



Architetture di sistemi distribuiti simmetricamente

Distribuzione verticale

Deriva dalla suddivisione delle applicazioni distribuite in tre livelli logici e dall'esecuzione dei componenti di ciascun livello su un server (macchina) diverso.

Distribuzione orizzontale

Un client o un server può essere fisicamente suddiviso in parti logicamente equivalenti, ma ogni parte opera sulla propria quota del set di dati completo.

Architetture Peer-To-Peer

I processi sono tutti uguali: le funzioni che devono essere eseguite sono rappresentate da ogni processo $\Rightarrow$ ogni processo agirà come client e server allo stesso tempo (ovvero, fungendo da **servant**).

Structured P2P

Utilizzano un **indice privo di semantica**: ogni data item è univocamente associato a una chiave, a sua volta usata come indice. Pratica comune: utilizzare una **funzione hash**:

$$\text{key}(\text{data item}) = \text{hash}(\text{data item's value})$$

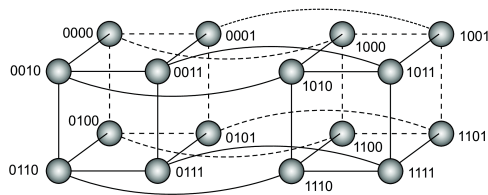
Di norma P2P era utilizzato per muovere dati e condividere file.

Ipercubo

Ogni nodo è identificato da un ID, che corrisponde all'hash del data item, quando faccio l'hash controllo quale nodo è responsabile del contenuto.

Se si cerca il file con hash 0001 allora il nodo è direttamente connesso e la ricerca finisce.

Se si cerca il file con hash 1001, devo chiedere ai nodi vicini, e chiederò a quello con la minore variazione di bit rispetto all'hash da cercare (**distanza di Hamming**), in questo caso 1000, successivamente posso raggiungere il file cercato.



Esempio: Chord

I nodi sono organizzati logicamente in un anello. Ogni nodo ha un **identificatore** di m bit.

Ogni elemento dati viene hashato su una **chiave** di m bit.

L'elemento dati con chiave k è memorizzato al nodo con ID identificatore più piccolo $\geq k$, chiamato **successore** della chiave k .

Se ho un data item con hash 3, il nodo che lo gestisce sarà il 4.

Il nodo 9 è responsabile delle chiavi 5, 6, 7, 8, 9.

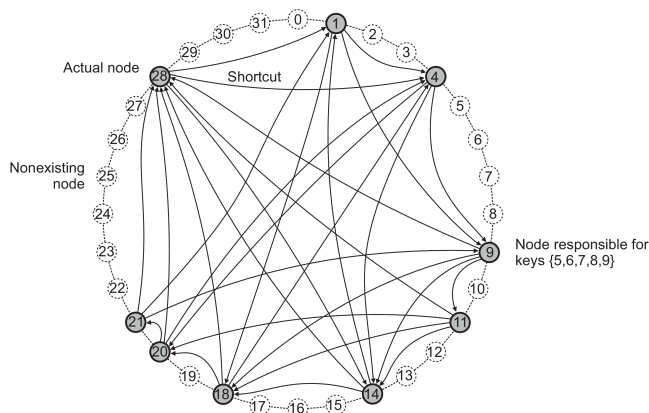
Complessità: O^n .

L'anello è esteso con vari collegamenti (scorciatoie) ad altri nodi.

Ogni nodo tiene una **finger table** che collega il nodo a dei vicini, per evitare di fare tutto il giro.

Se 4 cerca 25 e può chiedere a 1, 14 e 20, allora chiederà ovviamente al più alto, ovvero 20.

Complessità è $O(\log n)$.



Vedi anche: [2.6. Protocollo Chord](#).

Unstructured P2P

In questo caso non sappiamo i nodi partecipanti come sono collegati tra loro: c'è solo una certa probabilità che due nodi siano collegati.

Ogni nodo mantiene una lista dei vicini. La sovrapposizione risultante assomiglia a un **grafo random**: un arco $\langle u, v \rangle$ esiste solo con una certa probabilità $\mathbb{P}[\langle u, v \rangle]$.

Senza conoscere la topologia della rete, abbiamo due tecniche per la ricerca:

- **Flooding**: un nodo emittente u passa la richiesta di d a tutti i vicini.
La richiesta viene ignorata quando il nodo ricevente lo aveva già visto.
Altrimenti, v cerca localmente d (*ricorsivamente*).
Può essere limitato da un Time-To-Live: un numero massimo di salti (hops).
- **Random walk**: un nodo emittente u passa la richiesta di d ad un vicino casuale, diciamo v . Se v non ha d , inoltra la richiesta a uno dei suoi vicini scelti casualmente, e così via.
Con questo metodo non sempre posso trovare un dato, spesso bisogna ripetere varie random walk.

Flooding vs random walk

Assumiamo N nodi e ogni elemento è replicato randomicamente in r nodi scelti.

Nel random walk la probabilità $\mathbb{P}[k]$ che l'elemento venga trovato dopo k tentativi:

$$\mathbb{P}[k] = \frac{r}{N} \left(1 - \frac{r}{N}\right)^{k-1}$$

La *dimensione della ricerca* (S) è il numero atteso di nodi da sondare:

$$S = \sum_{k=1}^N k \cdot \mathbb{P}[k] \approx N/r$$

Nel flooding, con d vicini randomici, dopo k steps, qualche $R(k) = d \cdot (d-1)^{k-1}$ sarà raggiunto.

Con r/N nodi che hanno dati, se $\frac{r}{N} \cdot R(k) \geq 1$ avremo trovato il dato.

Quindi se $r/N = 0.001 \implies S \approx 1000$.

Con il flooding e $d = 10, k = 4$, contattiamo 7290 nodi.

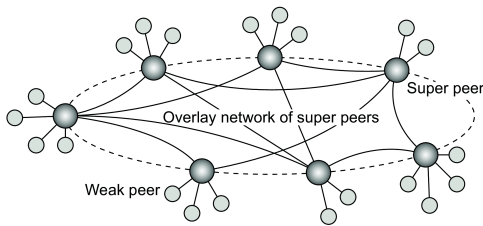
Le random walks sono più efficienti nella comunicazione, ma potrebbe volerci più tempo prima di trovare il risultato.

Reti Super-peer

Quando si cerca in sistemi P2P non strutturati, avere **index servers** migliora le prestazioni.

Decidere dove archiviare i dati può spesso essere fatto in modo più efficiente attraverso i **broker**.

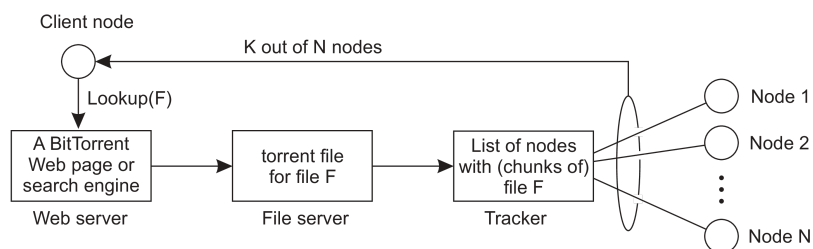
Gli weak peer si collegherà solo ai super peers: esso farà da index server per verificare la disponibilità dei file, e anche da broker, cioè stabilire dove è più conveniente spedire i dati.



Esempio: BitTorrent

Cerchiamo un file F .

- Ricerca file in una directory globale => ritorna un **torrent file**.
- Il file torrent contiene un riferimento al **tracker**: un server che tiene un account di nodi attivi che hanno (*chunks* di) F .
- P può unirsi allo **swarm**, ottenere un pezzo gratuitamente e poi scambiare una copia di quel pezzo con un altro con un peer Q anch'esso nello sciame.

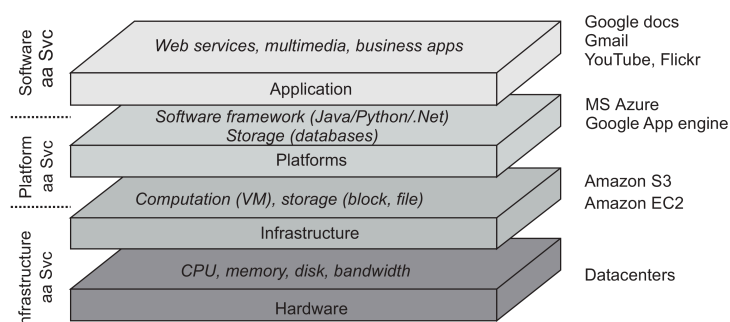


Architetture di sistemi ibridi

Cloud computing

Il **cloud computing** è un modello in cui **risorse informatiche** (server, storage, database, applicazioni) sono fornite tramite **Internet** da grandi data center.

Le applicazioni non girano su un singolo computer ma su **multi server distribuiti nei data center**.

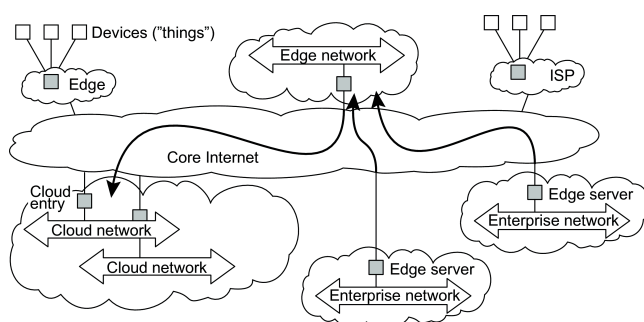


1. **Hardware**: processori, router, sistemi di alimentazione e raffreddamento. I clienti normalmente non li vedono mai.

2. **Infrastructure:** distribuisce tecniche di virtualizzazione. Si evolve intorno all'allocazione e alla gestione di dispositivi di archiviazione virtuale e server virtuali.
3. **Platform:** fornisce astrazioni di livello superiore per l'archiviazione e così via.
Esempio: il sistema di archiviazione Amazon S3 offre un'API per i file (creati localmente) da organizzare e archiviare nei cosiddetti bucket.
4. **Application:** le applicazioni effettive, come suites office.

Architetture edge-server

Le architetture edge sono sistemi distribuiti su Internet in cui i server sono posizionati **ai margini della rete**: il confine tra le reti aziendali e l'Internet reale.



Vantaggi delle infrastrutture edge

- **Latenza e bandwidth:** importante per le applicazioni real-time, come realtà aumentata. Molte persone sottovalutano la latenza e la larghezza di banda del cloud.
- **Affidabilità:** si presume spesso che la connessione al cloud sia inaffidabile, il che è spesso un falso presupposto. Potrebbero esserci situazioni critiche in cui sono necessarie garanzie di connettività estremamente elevate.
- **Security e privacy:** Il presupposto implicito è spesso che quando le risorse sono vicine, possono essere rese meglio protette. La pratica dimostra che questa ipotesi è generalmente falsa. Tuttavia, gestire in modo sicuro le operazioni di dati nel cloud può essere più complicato che all'interno della propria organizzazione.

Orchestrazione edge

Gestire le risorse al confine può essere più complicato che nel cloud.

- **Allocazione delle risorse:** garantire la disponibilità delle risorse necessarie per eseguire un servizio.
- **Posizionamento del servizio:** dobbiamo decidere quando e dove collocare un servizio, rilevante per le applicazioni mobili.
- **Selezione dell'edge:** dobbiamo decidere quale infrastruttura edge dovrebbe essere utilizzata quando è necessario offrire un servizio. Il più vicino potrebbe non essere il migliore.

Blockchains

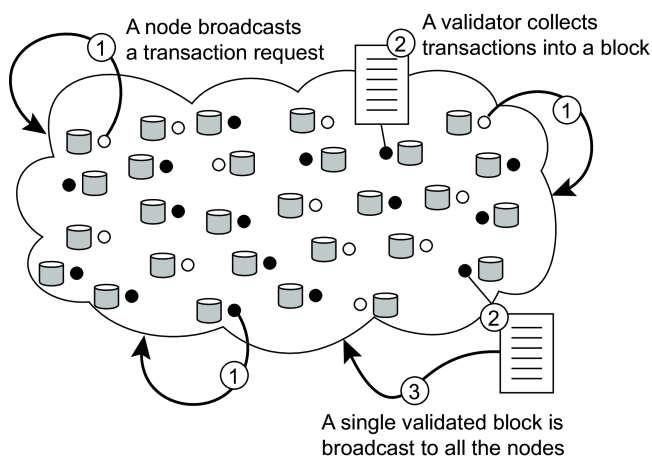
La **blockchain** è una struttura dati distribuita e immutabile utilizzata per registrare transazioni tra più partecipanti in una rete senza la necessità di un'autorità centrale.

È una forma di **registro distribuito (Distributed Ledger)** in cui le informazioni sono condivise e replicate tra molti nodi della rete.

Caratteristiche

- **Decentralizzazione.**
- **Immutabilità** dei dati => replicazione massiva.
- **Trasparenza.**
- **Sicurezza** crittografica: i blocchi sono organizzati in una catena non falsificabile di sola aggiunta.

Ogni partecipante mantiene una copia del registro e verifica le transazioni.



Aggiungere un blocco: consenso distribuito

La blockchain è composta da una sequenza di **blocchi collegati tra loro**.

Ogni blocco contiene:

- un insieme di *transazioni*;
- un *timestamp*;
- un *hash del blocco precedente*;
- un *proprio hash*.

Il problema sta in chi è autorizzato ad aggiungere blocchi alla catena.

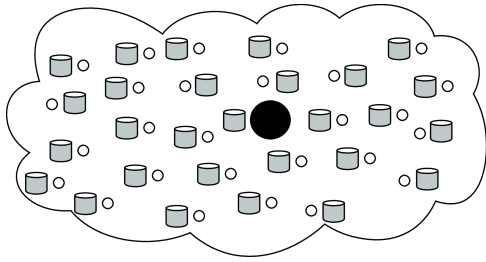
Poiché non esiste un'autorità centrale, i nodi devono raggiungere un **consenso** sullo stato della blockchain.

Il consenso stabilisce:

- Quali transazioni sono valide.
- Quale blocco deve essere aggiunto alla catena.

Soluzione centralizzata

Una singola entità decide su quale validatore può andare avanti e aggiungere un blocco. Non si adatta agli obiettivi di progettazione delle blockchain.

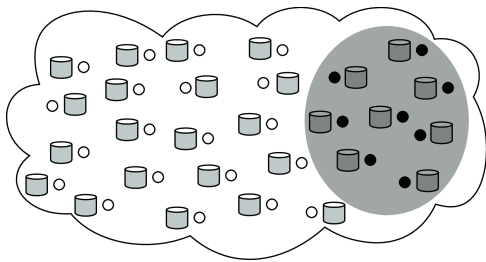


Soluzione distribuita (permesso)

Un gruppo selezionato e relativamente piccolo di server raggiunge congiuntamente un consenso su quale validatore possa andare avanti.

Nessuno di questi server deve essere considerato affidabile, purché circa due terzi si comportino in base alle loro specifiche.

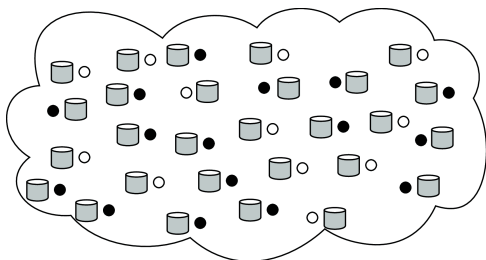
In pratica, solo poche decine di server possono essere ospitati.



Soluzione decentralizzata (senza permesso)

I partecipanti si impegnano collettivamente in un'elezione di leader. Solo il leader eletto è autorizzato ad aggiungere un blocco di transazioni convalidate.

L'elezione di leader su larga scala e decentralizzata che è equa, robusta, sicura e così via, è tutt'altro che banale.



2.5. Node.js nei sistemi distribuiti

Node.js è un **runtime JS lato server** basato su motore V8, open source e multiplatforma. Adotta l'architettura event-driven con modello I/O non bloccante.

Modello di esecuzione

Single thread + Event loop.

Un solo thread principale con operazioni I/O delegate a libuv.

Si basa su:

- **Callback** -> funzione alla quale Node può richiamare quando la risposta è terminata.
- **Promises** -> ti prometto che tra un po' ti rispondo, sia con successo che con fallimento).

Il loop di esecuzione è unico, le operazioni di I/O sono delegate al SO, registro un callback che, quando viene completata l'operazione, viene eseguito ed aggiunto nella coda eventi.

Microservizi

- API Gateway: facile da implementare come API messa a disposizione.
- Backend REST.
- Server WebSocket: basati su HTTP che si comportano come un socket TCP, permettono di scambiare dati in real-time instaurando una connessione sopra HTTP (es.: giochi web based).
- Architetture cloud-native.

Comunicazione distribuita

Node Packet Manager (NPM)

Limiti

Non ideale per carichi CPU-intensive: dato il single thread, blocking code e gestione complessa di errori asincroni.

Meglio evitarlo per calcoli complessi, LLM e in generale applicazioni CPU-bounded.

Wrapper XML to JSON

L'applicazione A parla XML, l'applicazione B parla JSON.

Serve un wrapper per convertire la tipologia dei dati da scambiare.

Service A - XML Legacy

```
// serviceA.js

const express = require('express');
```

```

const app = express();

app.get('/user', (req, res) => {
  const xmlResponse = `
    <user>
      <id>1</id>
      <name>Mario</name>
      <email>mario.rossi@example.com</email>
    </user>
  `;

  res.set('Content-Type', 'application/xml');
  res.send(xmlResponse);
})

app.listen(3001);

```

Wrapper Service - JSON API

```

// wrapperService.js

const express = require('express');
const axios = require('axios');
const xml2js = require('xml2js');

const app = express();

const parser = new xml2js.Parser({ explicitArray: false });

app.get('/user', async (req, res) => {
  // Chiamata bloccante con await
  const response = await axios.get('https://localhost:3001/user');

  const xmlData = response.data;

  parser.parseString(xmlData, (err, result) => {
    /* Ritorno nella funzione di callback,
       il risultato verrà restituito al completamento del parse
    */
    return res.json(result);
  })
})

app.listen(3000);

```

2.6. Protocollo Chord

Cos'è Chord

Chord è un **protocollo di lookup distribuito per sistemi P2P strutturati** basato su **Distributed Hash Tables (DHT)**.

Serve per risolvere il problema:

dato un identificatore di una chiave $k \rightarrow$ trovare il nodo che la gestisce.

Formalmente:

- input: **chiave** k .
- output: **nodo responsabile di** k .

Chord garantisce:

- **lookup in $O(\log N)$ messaggi.**
- nessun nodo centrale
- scalabilità fino a **milioni di nodi.**

Problemi dei P2P non strutturati

Prima di Chord esistevano sistemi P2P **non strutturati**, come **Gnutella**.

Problema del flooding

Quando un nodo cerca un file:

1. invia la query ai vicini
2. i vicini la inoltrano
3. l'intera rete viene inondata.

Complessità: $O(n)$ messaggi per ogni ricerca.

Confronto con Chord

Proprietà	Gnutella	Chord
Lookup	$O(N)$	$O(\log N)$
Messaggi	flooding	routing strutturato
Scalabilità	bassa	molto alta
Garanzia di trovare dati	no	sì

Proprietà	Gnutella	Chord
Centralizzazione	nessuna	nessuna

Consistent hasing

Chord usa **consistent hashing** per distribuire nodi e chiavi nello stesso spazio di identificatori.

Si usa **SHA-1**.

Produce un identificatore di **m bit**.

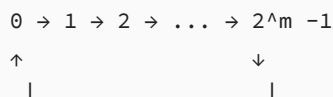
Tipicamente $m = 160$.

Lo spazio degli identificatori è:

$$0 \rightarrow 2^m - 1$$

Spazio degli identificatori

Lo spazio è **circolare (anello)**.



$$2^m - 1 + 1 = 0$$

Mapping dei nodi

Ogni nodo ottiene un ID:

$$\text{nodeId} = \text{hash}(\text{IP}, \text{port})$$

Quindi i nodi sono posizionati sull'anello.

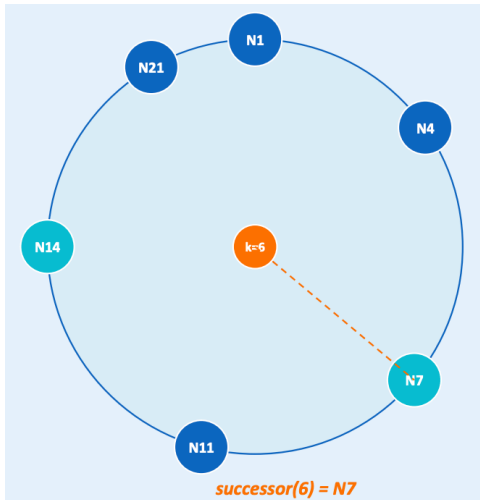
Mapping delle chiavi

Ogni chiave k è assegnata al **successore**.

$$\text{successor}(k) = \min \{n \in \text{Nodes} | n \geq k\}$$

con aritmetica modulo 2^m .

Esempio



Chiave $k = 6$.

Nodi: N1, N4, N7, N11, N14.

Il primo nodo con $\text{id} \geq 6$ è N7.

Quindi: $\text{successor}(6) = N7$

N7 gestisce la chiave.

Struttura dell'anello

Ogni nodo mantiene almeno due informazioni: **successor** e **predecessor**.

Successor

Il primo nodo successivo nell'anello.

Predecessor

Il nodo precedente nell'anello.

$$\text{predecessor} \leftarrow n \rightarrow \text{successor}$$

Questi puntatori mantengono la struttura dell'anello.

Finger Table

Per evitare ricerche lineari, ogni nodo mantiene una **finger table**, di dimensione **m** entry.

Per un nodo n :

$$\text{finger}[i] = \text{successor}(n + 2^{i-1}), 1 \leq i \leq m$$

Ogni entry punta ad un nodo **sempre più lontano**.

i	distanza
1	(2^0)
2	(2^1)
3	(2^2)
4	(2^3)

Quindi la distanza **raddoppia ogni volta**.

Proprietà

1. Ogni nodo conosce solo $O(\log N)$ nodi.
2. Ogni entry copre distanza doppia.
3. Ogni lookup dimezza la distanza, con complessità del lookup: $O(\log N)$.
4. La tabella va aggiornata periodicamente.

Lookup senza finger table

Il nodo:

1. controlla se la chiave è tra lui e il successore
2. se sì $\rightarrow$ ritorna il successore
3. altrimenti $\rightarrow$ inoltra al successore

```
def find_successor_simple(self, key):  
  
    if self.id == key:  
        return self  
  
    succ = self.successor  
  
    if key in (self, succ]:  
        return succ
```

```
return succ.find_successor_simple(key)
```

Nel caso peggiore attraversa tutto l'anello: $O(N)$.

Lookup con finger table

Si sceglie il nodo **più vicino alla chiave senza superarla**.

Quindi ogni passo **salta grandi porzioni dell'anello**.

```
def find_successor(self, key):  
  
    if key ∈ (self, successor]:  
        return successor  
  
    n_prime = closest_preceding_node(key)  
  
    if n_prime == self:  
        return successor  
  
    return n_prime.find_successor(key)
```

$O(\log N)$ messaggi per lookup.

closest_preceding_node

```
for i = M-1 → 0:  
  
    if finger[i] ∈ (self, key):  
        return finger[i]
```

Gestione rete dinamica: Join

I nodi possono **entrare nella rete** tramite vari passaggi.

1. Bootstrap

Il nodo nuovo n contatta un nodo esistente n' .

2. Trova il successore

Per determinare la posizione nell'anello:

```
successor = existing_node.find_successor(n.id)
```

3. Init finger table

Costruzione delle entry:

```
finger[i] = successor(n + 2i).
```

Ogni entry richiede un lookup: $O(\log^2 N)$.

4. Update others

Altri nodi devono aggiornare la loro finger table.

Alcuni finger potrebbero ora puntare al nuovo nodo.

5. Migrazione delle chiavi

Il successore trasferisce alcune chiavi.

```
chiavi <= n.id
```

vengono spostate da:

```
successor -> nuovo nodo
```

Stabilizzazione della rete

La rete cambia continuamente:

- join
- leave
- crash

Chord usa **protocolli periodici di stabilizzazione**.

stabilize()

Eseguito periodicamente, serve a correggere il successore.

```
x = successor.predecessor  
  
if x ∈ (n, successor):  
    successor = x
```



```
successor.notify(n)
```

notify()

Un nodo dice al successore: "potrei essere il tuo predecessore".

Il successore aggiorna il predecessore se necessario.

fix_fingers()

Aggiorna una entry casuale della finger table, mantenendo le rotte aggiornate.

```
i = random
finger[i] = find_successor(n + 2^i)
```

check_predecessor()

Controlla se il predecessore è ancora vivo, serve a rilevare fallimenti.

```
if predecessor crashed:
    predecessor = None
```

Fault tolerance

Per gestire crash multipli, ogni nodo mantiene `successor_list = r` nodi successivi.

Tipicamente $r = O(\log N)$.

Se il successore fallisce si usa il prossimo nella lista.

Questo garantisce che il lookup continui a funzionare.

Prestazioni

Sistema	Lookup	Stato nodo	Fault tolerance
Gnutella	$O(N)$	$O(1)$	alta
Napster	$O(1)$	server centrale	bassa
Chord	$O(\log N)$	$O(\log N)$	alta

Limitazioni

- non considera la **prossimità di rete** (latenza).
- lookup incoerente con **churn elevato**.
- replica e caching **non inclusi nel protocollo base**.

3. Processi

Thread

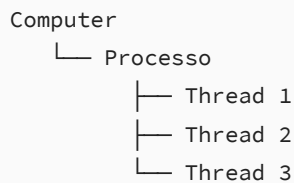
L'idea è di costruire un **processore virtuale** software, oltre al processore fisico.

Il **processore** fornisce una serie di istruzioni insieme alla capacità di eseguire automaticamente una serie di tali istruzioni.

Un **thread** è un processore software minimo nel cui contesto è possibile eseguire una serie di istruzioni. Salvare un contesto di thread implica interrompere l'esecuzione corrente e salvare tutti i dati necessari per continuare l'esecuzione in una fase successiva.

Un **processo** è un processore software nel cui contesto possono essere eseguiti uno o più thread. Eseguire un thread significa eseguire una serie di istruzioni nel contesto di quel thread.

Concetto	Descrizione
CPU	processore hardware
Thread	unità minima di esecuzione
Processo	contenitore di thread



Context switching

Per eseguire un thread o processo il sistema deve conoscere il **contesto**.

Il **contesto** è l'insieme di informazioni che descrivono lo stato dell'esecuzione.

- **Processor context:** la raccolta minima di valori salvati nei registri di un processore usato per l'esecuzione di una serie di istruzioni (es.: stack pointer, *registri di indirizzamento*, *program counter*).
- **Thread context:** la raccolta minima di valori memorizzati nei registri e nella memoria, utilizzata per l'esecuzione di una serie di istruzioni (es.: *contesto del processore*, *stato*).
- **Process context:** la raccolta minima di valori memorizzati nei registri e nella memoria, utilizzata per l'esecuzione di un thread (cioè il *contesto del thread*, ma ora anche almeno i valori del registro MMU).

① Context switching

Il **context switch** è l'operazione con cui il sistema operativo:

1. salva il contesto di un thread/processo
2. carica il contesto di un altro
3. continua l'esecuzione.

Favorisce per multitasking, parallelismo e gestione CPU.

I thread condividono lo stesso spazio di indirizzamento. Il context switching del thread può essere fatto indipendentemente dal SO.

Cambiare thread è veloce e spesso non richiede il kernel.

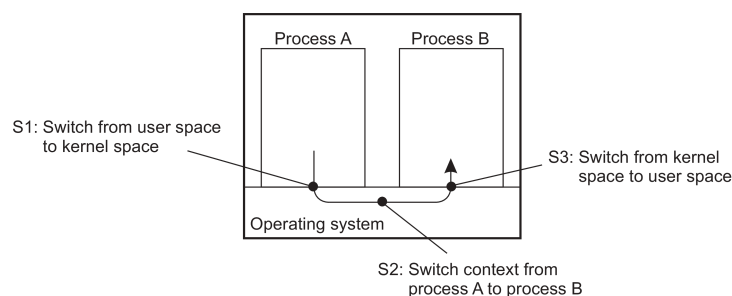
Il **process switching** è **generalmente più costoso** in quanto comporta l'attenere il sistema operativo nel ciclo, cioè l'intrappolamento del kernel.

Creare e distruggere i thread è molto più economico che farlo per i processi.

Perché usare i thread

1. **Evita il blocco inutile:** un processo a thread singolo bloccherà quando si esegue I/O; in un processo multithread, il sistema operativo può passare la CPU a un altro thread in quel processo.
2. **Parallelismo:** i thread in un processo multithread possono essere programmati per funzionare in parallelo su un processore multiprocessore o multicore.
3. **Evitare il process switching:** strutturare applicazioni di grandi dimensioni non come una raccolta di processi, ma attraverso più thread.

Evitare il cambio di processo



Condivisione di memoria

I thread utilizzano lo **stesso spazio di memoria** => più incline agli errori (un thread può *corrompere dati di un altro*) e race conditions.

Protezione limitata

Il sistema operativo **non protegge i thread tra loro** come fa tra processi.

Nei processi: `Process A memory` $\neq$ `Process B memory`

Nei thread: `Thread A memory` = `Thread B memory`

Il costo di un context switch

Il cambio di contesto ha due tipi di costo.

- **Costi diretti:** sono i costi dell'operazione stessa (salvare registri, caricare nuovi registri eseguire handler dell'interrupt).
- **Costi indiretti:** altri costi, come la cache CPU. Quando cambia il processo la cache contiene i dati del processo precedente, il nuovo processo ha però altri dati.

```
cache miss
↓
accesso alla RAM
↓
prestazioni peggiori
```

Esempio

```
from multiprocessing import Process
from time import sleep
from random import randint

def sleeper(name):
    s = randint(1,20)
    print(name, "sleep", s)
    sleep(s)
    print(name, "woke up")

p = Process(target=sleeper, args=('eve',))
q = Process(target=sleeper, args=('bob',))

p.start()
q.start()

p.join()
q.join()
```

1. Vengono creati due processi: `Process eve`, `Process bob`.
2. Entrambi dormono per tempo casuale.
3. L'esecuzione avviene **in parallelo**.

Output esempio:

```
eve is going to sleep for 14 seconds
bob is going to sleep for 4 seconds
bob has woken up
eve has woken up
```

Questo dimostra **esecuzione concorrente**.

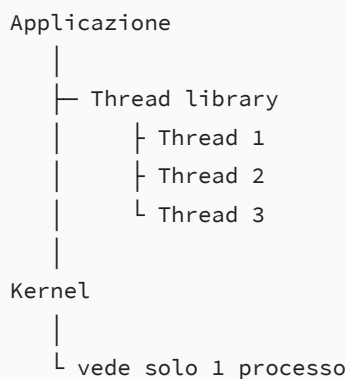
Thread nei sistemi operativi

② Problema principale

Un kernel del sistema operativo dovrebbe fornire thread o dovrebbero essere implementati come pacchetti a livello di utente?

Thread a livello utente (User-level threads)

Tutte le operazioni potrebbero essere gestite **con un singolo processo** => implementazioni efficienti.



Il **kernel non sa nemmeno che esistono più thread**.

Dal punto di vista del sistema operativo 1 processo = 1 flusso di esecuzione, ma in realtà l'applicazione gestisce **più thread internamente**.

Una **thread library** (libreria di gestione thread):

- crea thread
- salva i contesti
- fa lo scheduling
- effettua il **thread switching**.

Il cambio di thread avviene **interamente nello spazio utente**.

Tutti i servizi forniti dal kernel vengono eseguiti **per conto del processo in cui risiede un thread** => se il kernel decide di bloccare un thread, l'intero processo verrà bloccato.

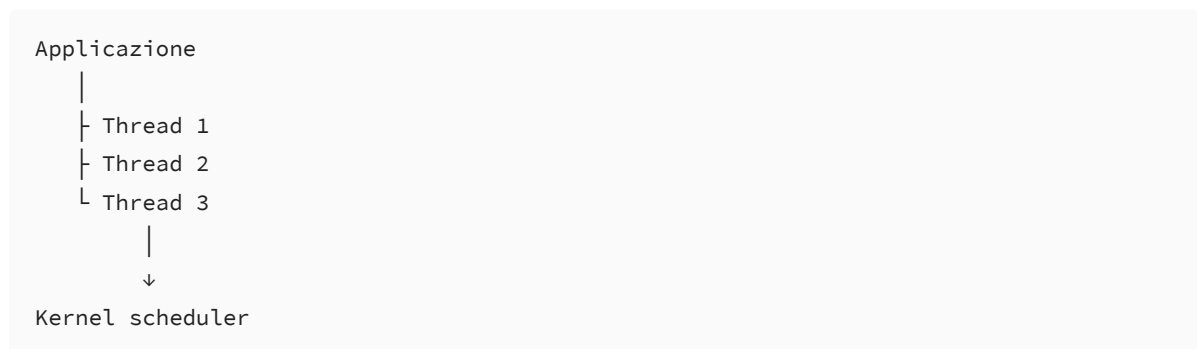
I thread vengono utilizzati quando ci sono molti eventi esterni: **si bloccano per evento** => se il kernel non può distinguere i thread, come può supportare gli eventi di segnalazione ad essi?

C'è l'assenza di parallelismo: il kernel assegna **un solo core al processo** ed i thread utente non possono essere eseguiti in parallelo.

Thread a livello kernel (Kernel-level threads)

L'idea è che il kernel contenga l'implementazione di un pacchetto thread => tutte le operazioni ritornano come chiamate di sistema:

- Le operazioni che bloccano un thread non sono più un problema: il kernel **pianifica un altro thread disponibile** all'interno dello stesso processo.
- Gestire eventi esterni è semplice: il kernel **schedula** il thread associato all'evento.



Il problema è la **perdita di efficienza** perché ogni operazione di thread richiede una trappola per il kernel.

Creare o cambiare thread richiede:

- **system call**
- passaggio kernel mode
- strutture del kernel.

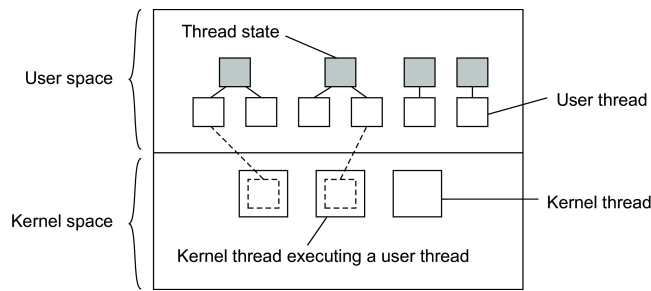
Quindi è più lento rispetto ai thread utente.

Conclusione

Provare a mescolare i thread a livello di utente e a livello di kernel in un unico concetto, tuttavia, il guadagno di prestazioni non si è rivelato superare l'aumento della complessità.

Thread user-level e kernel-level

Introduciamo un approccio di **threading a due livelli**: thread kernel che possono eseguire thread a livello di utente.



- Il thread utente fa system call $\Rightarrow$ **il thread del kernel che sta eseguendo quel thread utente, viene bloccato**. Il thread utente rimane legato al thread del kernel.
- Il kernel può **schedulare un altro thread kernel con un thread utente eseguibile ad esso collegato**. Nota: questo thread utente può passare a qualsiasi altro thread utente eseguibile attualmente nello spazio utente.
- Un thread utente chiama un'operazione di blocco a livello di utente $\Rightarrow$ context switch a un thread utente eseguibile, (quindi legato allo stesso thread del kernel).
- Quando non ci sono thread utente da schedulare, il thread kernel potrebbe rimanere in stato di idle, e potrebbe essere rimosso (*distrutto*) dal kernel.

Thread client side

I **client-side threads** sono thread che eseguono nel programma client, cioè nella macchina che **fa la richiesta al server**.

Servono principalmente per:

- gestire **più richieste contemporaneamente**;
- evitare che il client **rimanga bloccato mentre aspetta una risposta**.

Se il client ha **un solo thread**, esso deve aspettare la risposta dal server, facendo rimanere la CPU inutilizzata (blocking communication).

Multithreaded web client

Mentre un thread aspetta la risposta altri thread **continuano a lavorare**, riducendo la latenza di rete.

1. Il browser web scansiona una pagina HTML in arrivo e scopre che è necessario recuperare più file.
2. Ogni file viene recuperato da un thread separato, ognuno dei quali effettua una richiesta HTTP (bloccante).
3. Man mano che i file arrivano, il browser li visualizza.

Chiamate di richiesta-risposta multiple ad altre macchine (RPC)

Un client fa più chiamate contemporaneamente, ognuna tramite un thread diverso. Quindi aspetta che tutti i risultati siano stati restituiti.

Nota: se le chiamate sono verso server diversi, potremmo avere un'**accelerazione lineare**.

Thread-Level Parallelism (TLP)

c_i denota la frazione di tempo in cui i thread vengono eseguiti contemporaneamente.

$$\text{TLP} = \frac{\sum_{i=1}^N i \cdot c_i}{1 - c_0}$$

con N massimo numero di thread che possono essere eseguiti allo stesso tempo.

Un tipico browser Web ha un valore TLP compreso tra 1,5 e 2,5 $\Rightarrow$ i thread vengono utilizzati principalmente per organizzare logicamente i browser.

Thread server-side

I **server-side threads** sono thread che girano nel server, cioè nel computer che **riceve e gestisce le richieste dei client**.

```
Client 1 → richiesta
Client 2 → richiesta
Client 3 → richiesta

Server
├ Thread 1 → gestisce client 1
├ Thread 2 → gestisce client 2
└ Thread 3 → gestisce client 3
```

Migliorare le performance

Avviare un thread è più economico che iniziare un nuovo processo.

Avere un server single thread vieta il semplice scale-up a un sistema multiprocessore.

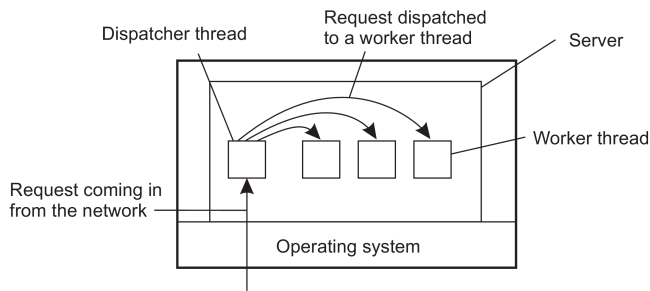
Come per i client: nasconde la latenza di rete rispondendo alla richiesta successiva mentre si risponde a quella precedente.

Migliore struttura

La maggior parte dei server ha elevate richieste di I/O. L'uso di **chiamate di blocco semplici** semplifica la struttura.

I programmi multithread tendono ad essere più piccoli e più facili da capire grazie al **flusso di controllo semplificato**.

Dispatcher/worker



Il **Dispatcher/Worker model** è un modo comune di organizzare un **server multithread**.

Nel server ci sono due tipi di thread:

1. **Dispatcher thread** -> riceve le richieste dalla rete e decide quale worker thread deve gestirle.
Quindi il dispatcher non elabora la richiesta, ma la assegna.
2. **Worker thread** -> prendono una richiesta, la elaborano e mandano la risposta al client.
Ogni worker può gestire una richiesta diversa.

```
Client → richiesta
      ↓
Dispatcher thread
      ↓
assegna richiesta
      ↓
Worker thread
      ↓
elaborazione
      ↓
risposta al client
```

Questo permette di gestire **molte richieste contemporaneamente**.

Il vantaggio principale è che il **dispatcher resta sempre libero**, gli **worker lavorano in parallelo**.

Modello	Caratteristiche
Multithreading	Parallelismo, system call bloccanti
Single-threaded process	Nessun parallelismo, system call bloccanti
Finite-state machine	Parallelismo, system call non bloccanti

Multithreading

Il server usa **più thread**, gestendo richieste gestite **in parallelo** e può usare **system call bloccanti** (es. I/O).

Single-threaded process

Il server usa **un solo thread**: non c'è parallelismo e se il thread si blocca, tutto il server si blocca.

Finite-state machine (event-driven)

Questo modello usa **un solo thread** e **I/O non bloccante**.

Invece di bloccare il thread il server gestisce eventi e passa da una richiesta all'altra (*Node.js*, *Nginx*).

Approfondimento: CPU, processi e thread

[3.5 CPU, Processi e Thread](#)

Virtualization

La **virtualizzazione** è una tecnica che permette di **simulare (emulare)** una macchina o un ambiente software sopra un altro sistema.

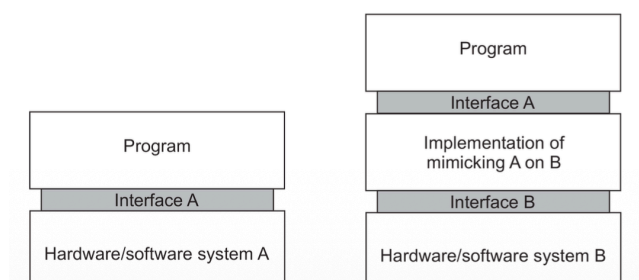
- si crea una **macchina virtuale (VM)**
- che si comporta **come un computer reale**
- ma gira **sopra un altro sistema**.

Ogni VM può avere il proprio **SO**, i propri **processi** e la propria **memoria**.

📘 Virtualizzazione

- L'hardware **cambia più velocemente** del software => eseguire SW vecchio su HW nuovo.
- Facilità di **portabilità** e migrazione del codice => macchine diverse e senza modifiche.
- **Isolamento** di componenti guasti o attaccati => se una VM crasha o viene attaccata, le altre non vengono influenzate.

Interfacce virtualizzabili



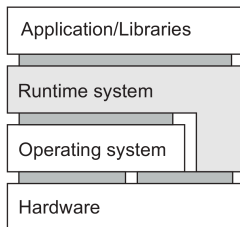
Possiamo avere quattro tipi di interfacce a tre diversi livelli:

1. **Instruction set architecture**: l'insieme di istruzioni macchina, con due sottoinsiemi:

1. Istruzioni privilegiate: possono essere eseguite solo dal sistema operativo (*gestione memoria, interrupt, configurazioni hw*).
 2. Istruzioni generali: possono essere eseguite da qualsiasi programma (*add, load, store*).
2. **System calls** offerte dal SO (*creare file, leggere dal disco, creare processi*).
 3. **Library calls**, anche conosciute come **Application Programming Interface (API)**.

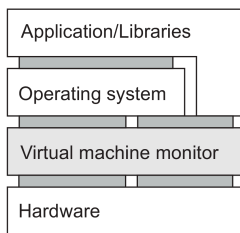
Modi per implementare la virtualizzazione

Process VM



La VM interpreta le istruzioni del programma, in esecuzione su un sistema operativo (es: JVM).

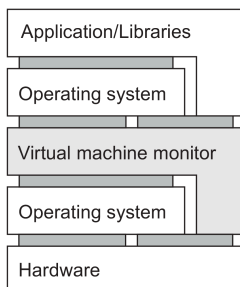
Native VMM



Il **VMM (Virtual Machine Monitor)** gira **direttamente sull'hardware**, è molto efficiente e usato nei data center (es: Hyper-V, VMware ESXi).

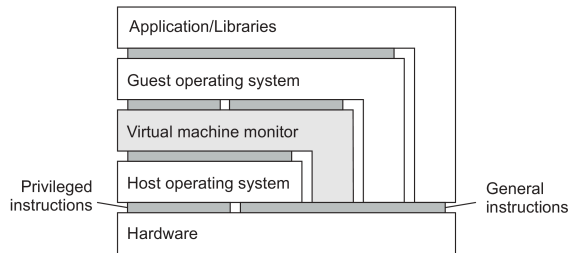
Usa istruzioni di basso livello, insieme a un sistema operativo minimale.

Hosted VMM



Istruzioni di basso livello, ma delega la maggior parte del lavoro a un sistema operativo a tutti gli effetti.

Problema della virtualizzazione



Se un'istruzione **privilegiata** viene eseguita **user mode**, viene generata una trap ed il controllo passa al SO.

Questa trap permette al SO di intercettare l'operazione.

Le istruzioni **non privilegiate** sono tutte le **altre istruzioni normali**, possono essere eseguite direttamente dai programmi e non causano trap.

Istruzioni speciali

Ci sono due tipi di istruzioni **sensibili** che possono **interferire con la virtualizzazione**.

- **Control-sensitive instructions:** possono modificare le configurazioni della macchina (registri di memoria, tabella degli interrupt, ...). Se vengono eseguite senza controllo la VM potrebbe modificare l'HW reale. Devono essere quindi intercettate dal VMM.
- **Behavior-sensitive instructions:** istruzioni il cui comportamento dipende dal contesto di esecuzione. Ad es. `POPF` modifica un flag di interrupt, solo se eseguita in system mode. Il risultato cambia a seconda dell'esecuzione in user mode o kernel mode.

Condizioni per la virtualizzazione

❗ Condizione necessaria

Per virtualizzare correttamente un sistema il **VMM deve intercettare tutte le operazioni che possono modificare la macchina**.

Questo avviene grazie alle **trap** generate dalle istruzioni privilegiate.

Tuttavia ci potrebbero essere istruzioni sensibili che vengono eseguite in modalità utente senza causare una trap al sistema operativo.

Le principali soluzioni sono:

- Emulare tutte le istruzioni.
- Wrappare le istruzioni sensibili non privilegiate per deviare il controllo su VMM.
- **Paravirtualization:** modificare il sistema operativo guest, impedendo istruzioni sensibili non privilegiate o rendendole non sensibili (cioè cambiando il contesto).

Approfondimento

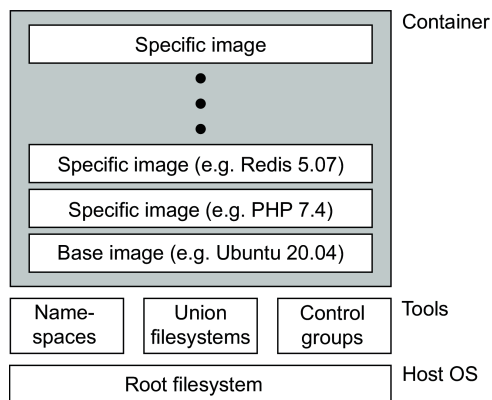
[3.7 Virtualizzazione, Paravirtualizzazione e Container](#)

Container

Un **container** è una forma di **virtualizzazione leggera** in cui un'applicazione viene eseguita **in un ambiente isolato**, ma **condivide lo stesso sistema operativo** della macchina host.

Ogni applicazione ha proprie librerie, propri file e propria configurazione, ma non ha un SO completo come nelle VM.

Un container fornisce un ambiente di esecuzione indipendente per un'applicazione.



Il SO host crea questo isolamento tramite:

1. **Namespaces** -> ogni container vede solo le proprie risorse.
2. **Union file system** -> combina diversi file system in modo stratificato con solo il livello più alto che consente operazioni di scrittura (e quello che fa parte di un contenitore).
3. **Control groups** -> limitano CPU, memoria, ecc..

Vantaggi dell'approccio container

1. **Consumano meno memoria.**
2. **Più facili da gestire.**
3. **Risorse allocabili dinamicamente.**

Infatti una VM richiede memoria fissa, mentre i container possono **condividere risorse dinamicamente**.

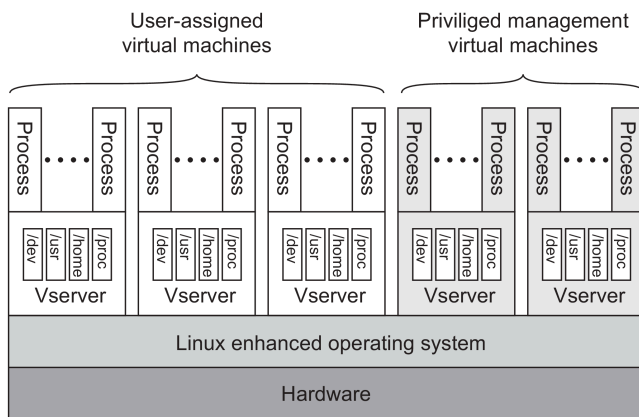
PlanetLab

PlanetLab era una piattaforma di ricerca per **sperimentare sistemi distribuiti su larga scala**.

Era costituito da centinaia di nodi distribuiti in tutto il mondo e forniti da diverse organizzazioni. Ogni organizzazione donava uno o più computer, creando una grande infrastruttura condivisa.

L'obiettivo era testare nuovi sistemi distribuiti su Internet reale.

Il problema era assicurarsi che le diverse applicazioni distribuite non si intralcino l'una con l'altra => virtualizzazione.



Ogni nodo PlanetLab conteneva:

1. **VMM (Virtual Machine Monitor)**: una versione modificata di Linux.
2. **Vservers**: container Linux che eseguivano applicazioni.

Un **Vserver** era essenzialmente:

un container che esegue un gruppo di processi isolati dagli altri container.

Ambiente indipendente e protetto con le proprie librerie, versioni server e così via. Alle applicazioni distribuite viene assegnata una raccolta di Vservers distribuiti su più macchine.

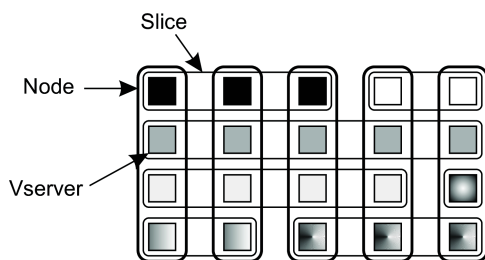
Vservers e slices

Ogni Vserver ha il suo ambiente e Linux regola gli ID dei processi.

Due processi in diversi Vservers potrebbero avere lo stesso userID, ma non lo stesso utente.

Una **slice** è un insieme di Vservers distribuiti su *nodì diversi della rete*.

In pratica una slice è **un cluster distribuito virtuale costruito con container**.



Ogni nodo aveva un **Node manager** che creava nuovi Vservers, gestiva le risorse e controllava CPU, memoria e rete.

VMs e cloud computing

I tre tipi di servizi in cloud sono:

1. **Infrastructure-as-a-Service (IaaS)**: copre l'infrastruttura di base. Il provider fornisce VM, rete e storage; l'utente gestisce SO ed applicazioni (*AWS EC2*).
2. **Platform-as-a-Service (PaaS)**: copre i servizi a livello di sistema. Il provider gestisce infrastruttura,

SO e runtime; l'utente carica solo l'applicazione (*Heroku, Google App Engine*).

3. **Software-as-a-Service (SaaS)**: contiene le applicazioni. Il provider fornisce l'applicazione completa (*Gmail, Google Docs*)

❶ IaaS

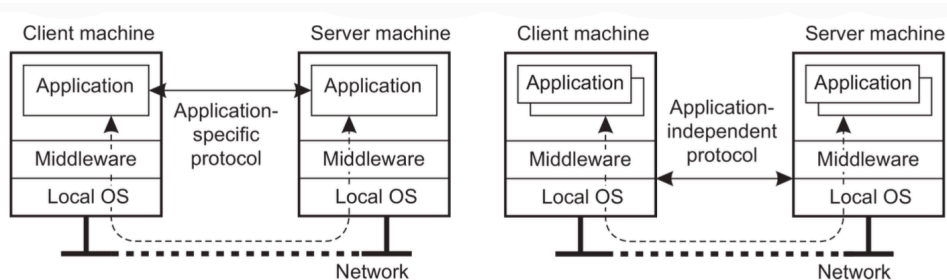
Invece di affittare una macchina fisica, un fornitore di cloud affitterà una VM (o VMM) che potrebbe condividere una macchina fisica con altri clienti $\Rightarrow$ isolamento quasi completo tra i clienti (anche se l'isolamento delle prestazioni potrebbe non essere raggiunto).

Client

Soluzioni a livello di applicazione e a livello di middleware

Ci sono due modi diversi di implementare la comunicazione in un sistema distribuito:

1. **Soluzione a livello di applicazione** -> il programma dell'utente.
2. **Soluzione a livello di middleware** -> strato SW che gestisce comunicazione, naming, sicurezza, RPC ecc. e nasconde la complessità della rete.



Soluzione a livello di applicazione

Nella prima figura la comunicazione avviene direttamente tra le applicazioni, tramite un protocollo **application-specific protocol**.

Ogni applicazione implementa il suo protocollo per gestire il formato dei messaggi, invio, ricezione ed interpretazione.

I vari problemi di questo approccio sono duplicazione del codice, ogni app deve reinventare la comunicazione, difficile manutenzione ed interoperabilità.

Soluzione a livello di middleware

Nella seconda figura il protocollo è **application-independent protocol**.

Le applicazioni non gestiscono direttamente la comunicazione, ma usano servizi forniti dal middleware.

Il MW si occupa di inviare messaggi, serializzare i dati, gestire le connessioni e nascondere la rete.

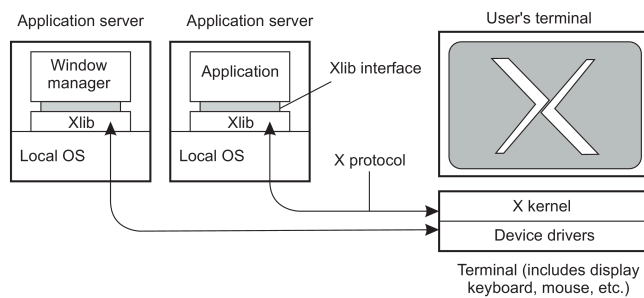
Il middleware del client comunica con il middleware del server.

Esempio: il sistema X Window

X Window System (X) è un sistema utilizzato nei sistemi Unix/Linux per gestire l'**interfaccia grafica** (finestre, mouse, tastiera, schermo).

È stato progettato per supportare interfacce grafiche distribuite su rete.

Questo significa che l'applicazione può girare su una macchina e lo schermo può stare su un'altra macchina.



L'applicazione funge da client per l'X-kernel, quest'ultimo in esecuzione come server sulla macchina del client.

Problemi delle prestazioni

- Spesso non c'è una chiara separazione tra la logica dell'applicazione e i comandi dell'interfaccia utente.
- Le applicazioni tendono a funzionare in modo strettamente **sincrono** con un kernel X.

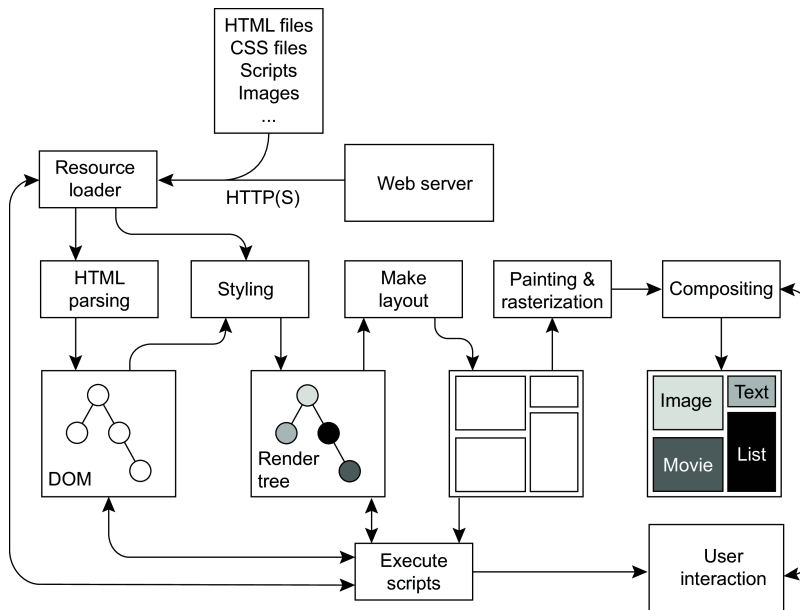
Per migliorare le prestazioni ci sono vari approcci, tra cui:

- Lasciare che le applicazioni controllino **completamente** il display, fino al livello di pixel (ad esempio, *VNC*).
- Fornire solo poche operazioni di visualizzazione di alto livello (a seconda dei driver video locali), consentendo operazioni di visualizzazione più efficienti.

Virtual desktop environment

- **Problema:** sviluppare l'interfaccia utente di rete definitiva.
- **Risposta:** utilizzare un browser Web per stabilire un'esperienza senza soluzione di continuità.

Web browser



1. **Richiesta delle risorse:** per recuperare i file dalla pagina, il Resource Loader invia una richiesta al web server con HTTP(S). Esso restituisce i file necessari (HTML, CSS, JS, img...).
2. **Parsing HTML:** il browser analizza il file HTML tramite il processo di parsing, costruendo il DOM (Document Object Model), ovvero la struttura della pagina.
3. **Styling:** applicazione dei CSS agli elementi del DOM => Render Tree con solo gli elementi visualizzabili.
4. **Layout:** il browser calcola le dimensioni e posizione di ogni elemento (width, height, position...) => fase di reflow.
5. **Painting e Rasterization:** il browser disegna gli elementi, convertendo testo, immagini, color e bordi in pixel.
6. **Composition:** il browser combina i vari livelli grafici, come testo, immagini, video e liste, per creare il frame finale mostrato sullo schermo.
7. **Esecuzione degli script:** i JS vengono eseguiti, modificando il DOM, il Render Tree o il layout (dovendo quindi rifare layout o painting).
8. **Interazione utente:** l'utente può interagire con la pagina (click, scroll, input), modificando elementi del DOM, eseguendo script o causando nuovi rendering.

Client-side software

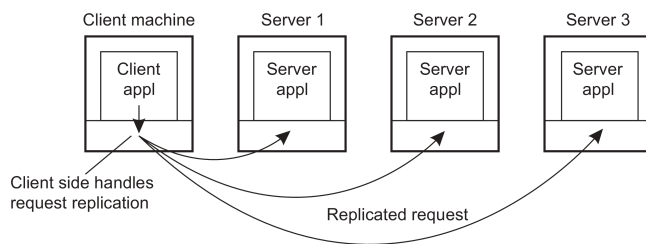
Trasparenza della distribuzione

Il software client comprende componenti per ottenere la trasparenza della distribuzione. Idealmente, un client non dovrebbe essere consapevole che sta comunicando con processi remoti. Al contrario, la distribuzione è spesso meno trasparente per i server per motivi di prestazioni e correttezza.

- **Access transparency:** gestita attraverso la generazione di uno stub client da una definizione di interfaccia di ciò che il server ha da offrire. Lo stub fornisce la stessa interfaccia di quella disponibile sul server, ma nasconde le possibili differenze nelle architetture delle macchine, così come la

comunicazione effettiva. Lo stub client trasforma le chiamate locali in messaggi che vengono inviati al server e viceversa trasforma i messaggi dal server per restituire i valori come ci si aspetterebbe quando si chiama una procedura ordinaria.

- **Location/migration transparency:** quando un client è già legato a un server, il client può essere informato direttamente quando il server cambia posizione. In questo caso, il middleware del client può nascondere l'attuale posizione di rete del server all'utente e anche ricollegarsi in modo trasparente al server, se necessario. Nel peggiore dei casi, l'applicazione del cliente potrebbe notare una perdita temporanea di prestazioni.
- **Replication transparency:** con un sistema distribuito con server replicati, la replica può essere ottenuta inoltrando una richiesta a ciascuna replica. Il software lato client può raccogliere in modo trasparente tutte le risposte e passare una singola risposta all'applicazione client.



- **Failure transparency:** il mascheramento degli errori di comunicazione con un server viene in genere eseguito tramite middleware client: tenta ripetutamente di connettersi a un server, o provare un altro server dopo diversi tentativi. Ci sono anche situazioni in cui il middleware client restituisce i dati che aveva memorizzato nella cache durante una sessione precedente, come a volte viene fatto dai browser Web che non riescono a connettersi a un server.

Server

① Modello semplice

Un processo che implementa un servizio specifico per conto di una raccolta di client.

Attende una richiesta in arrivo da un client e successivamente si assicura che la richiesta sia presa in arrivo, dopodiché attende la richiesta successiva.

- **Iterative server:** il server stesso gestisce la richiesta prima di attendere la successiva.
- **Concurrent server:** usano un dispatcher che prende una richiesta in arrivo e la passano a un thread/processo separato.

I server concorrenti sono la norma: possono facilmente gestire più richieste, in particolare in presenza di operazioni di blocco (a dischi o altri server).

Contattare un server

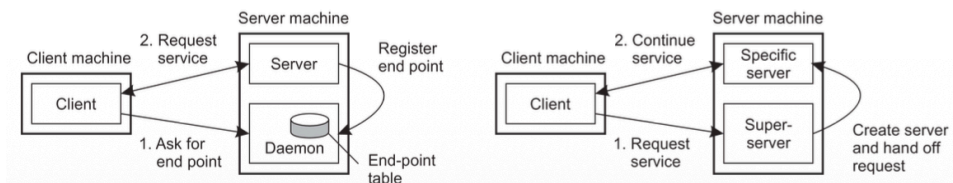
I client inviano richieste a un endpoint, chiamato anche porta, alla macchina in cui è in esecuzione il server. Ogni server ascolta un endpoint specifico:

1. **Endpoint globali** -> servizi sempre in ascolto su una determinata porta, assegnata dalla IANA (FTP, 21 o HTTP, 80).

Con questo approccio il client deve conoscere solo l'indirizzo di rete della macchina del server.

2. **Endpoint dinamici** -> un server time-of-day può utilizzare un endpoint che gli viene assegnato dinamicamente dal suo SO locale.

Si può avere un demone in esecuzione su ogni server, che tiene traccia degli endpoint correnti di ogni servizio implementato da un server; il client contatterà prima il demone, richiedendo l'endpoint e contatterà il server.



Comunicazione out-of-band

È possibile interrompere un server una volta che ha accettato (o sta accettando) una richiesta?

Soluzione 1: porta separata per dati urgenti

Il server ha un thread/processo separato per i messaggi urgenti.

Quando arriva un messaggio urgente **la richiesta associata viene sospesa**.

In questo caso il SO deve supportare lo scheduling basato su priorità.

Soluzione 2: funzionalità del layer di trasporto

Il TCP permette messaggi urgenti nella stessa connessione, che possono essere **intercettati** utilizzando tecniche di signaling del SO.

Server e stati

Server stateless

Non conservano informazioni **accurate** sullo stato di un client dopo aver gestito una richiesta.

Non tengono traccia dell'apertura o no di un file, non permettono di invalidare la cache di un client e non tengono traccia dei client.

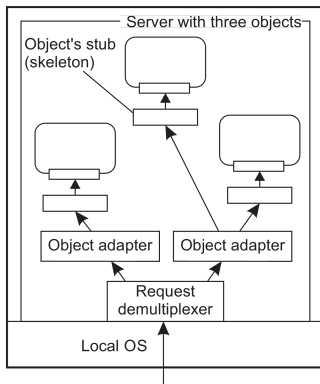
Quindi client e server sono **completamente indipendenti**, riducendo le **incongruenze** di stato dovute a crash di client o server, ma **perdendo in prestazioni** dato che un server non potrebbe anticipare il comportamento dei client (*pre-caricamento di un blocco di file*).

Server stateful

Tiene traccia dello stato dei client: registra l'apertura di un file per effettuare pre-caricamento, conosce i dati copiati nella cache del client e permette ad essi di mantenere copie locali dei dati condivisi.

Prestazioni elevate a patto che ai client sia permesso mantenere copie locali, in questo caso l'affidabilità viene meno.

Object server

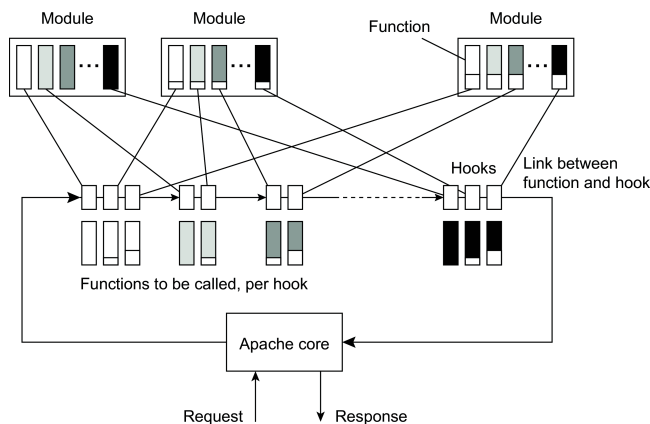


Un **Object Server** è un tipo di server che gestisce **oggetti distribuiti** e permette ai client di **invocare metodi su oggetti remoti**.

Una policy di attivazione decide quali azioni intraprendere quando arriva una richiesta: dove risiedono il codice e dati (stato) dell'oggetto, quale modello di threading utilizzare e se conservare o meno lo stato modificato dell'oggetto.

L'object adapter implementa una specifica policy di attivazione.

Esempio: Apache web server

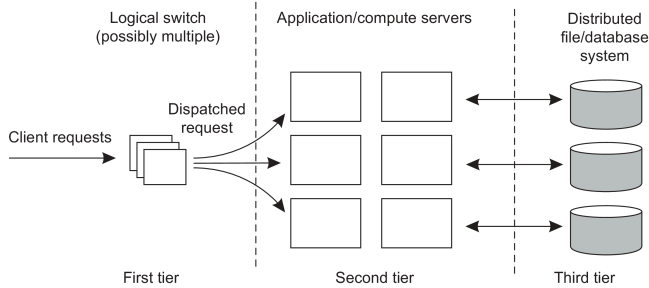


Apache è un **server modulare** che ha un **core centrale**, usa **moduli** per aggiungere funzionalità e usa **hooks**, cioè punti in cui i moduli possono intervenire.

Request → Core → Hooks → Moduli → Response

Il core gestisce la richiesta e nei vari hook chiama funzioni dei moduli.

Cluster computing



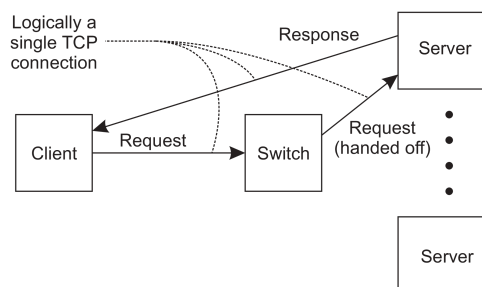
Lo schema rappresenta un sistema a **tre livelli (three-tier)** dentro un **cluster**:

1. *First tier* -> riceve le richieste dei client ad un server appropriato (**request dispatching**).
2. *Second tier* -> insieme di server (cluster) che eseguono la logica applicativa, elaborano le richieste e la business logic.
3. *Third tier* -> database/file system distribuito con più nodi di storage per salvare dati, garantire consistenza e disponibilità.

Gestione delle richieste

Il primo livello gestisce tutta la comunicazione da e verso il cluster => **collo di bottiglia**.

Una soluzione è il TCP handoff.



Il **TCP handoff** è una tecnica in cui:

- il first tier **trasferisce la connessione TCP a un server del cluster**.

Wide-area cluster

Osservazione

Distribuire i server su Internet può introdurre problemi amministrativi, aggirabili utilizzando data center di un singolo provider cloud.

Request dispatching

Utilizzando un DNS il client cerca un servizio specifico tramite DNS, quindi l'indirizzo IP del client fa parte della richiesta.

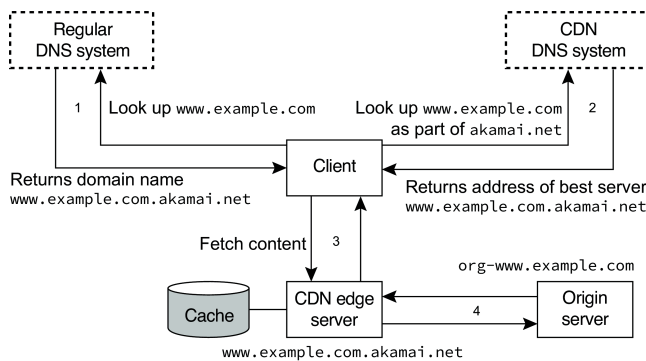
Il server DNS tiene traccia dei server replica per il servizio richiesto e restituisce l'indirizzo del server più vicino fisicamente.

Trasparenza del client

Per mantenere il client all'oscuro della distribuzione si lascia che il resolver DNS deve agire per conto del client.

Tuttavia il resolver potrebbe essere molto lontano dal locale del client reale.

Versione semplificata CDN Akamai



I name resolver Akamai cercheranno il miglior server edge per servire il client e restituiranno il suo indirizzo di rete (2).

Ciò consente al client di contattare il server edge (3), che è a conoscenza del nuovo nome del server di origine.

Se il contenuto richiesto non si trova nella cache del server edge, recupera i documenti dal server di origine (4), memorizza nella cache quei documenti e restituisce quelli richiesti al client.

La cache, oltre che semplici dati passivi, potrebbe contenere parte del codice applicativo del server di origine.

Migrazione del codice

Per distribuire il carico dobbiamo assicurarci che i server in un data center siano **sufficientemente carichi** (evitare sprechi di energia) e minimizzare la comunicazione, assicurandoci che i calcoli siano **vicini** al punto in cui si trovano i dati.

Ragioni per cui migrare il codice

1. Performance

Portare il codice **vicino ai dati** invece di spostare grandi quantità di dati sulla rete.

Se abbiamo un server con un DB enorme ed il client deve fare molte operazioni => si sposta parte del codice client sul server si inviano **solo i risultati**, non tutti i dati.

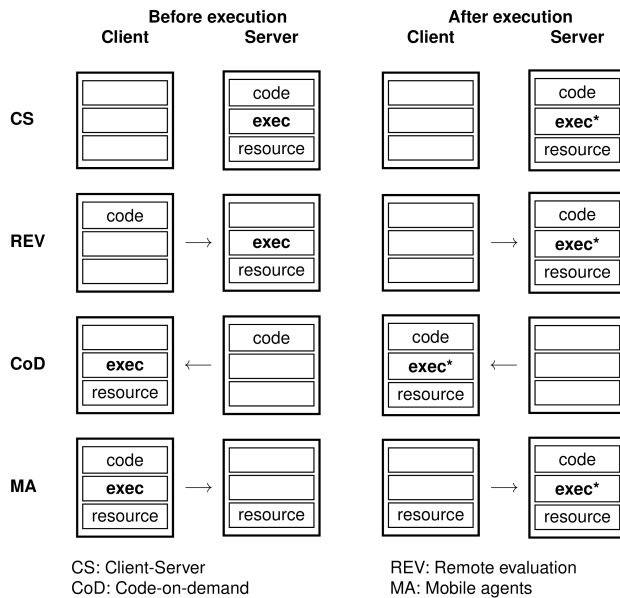
2. Privacy e sicurezza

Se non possiamo spostare i dati in un'altra località (*GDPR*), dobbiamo spostare il codice verso i dati. Così facendo i dati restano locali e si invia il modello (codice) ai dati, aggiornando i risultati localmente.

3. Flessibilità

Non decidere tutto a priori (come nei sistemi statici), ma configurare il sistema dinamicamente in modo che scarichi il codice quando serve.

Paradigmi per la mobilità del codice



Esistono 4 paradigmi di code mobility :

1. Client-Server -> tutto resta sul server ed il client invia le richieste.
2. Remote Evaluation -> il client manda il codice al server ed il codice viene eseguito dove stanno i dati.
Il client invia il codice al server, il server lo esegue e restituisce il risultato.
3. Code-on-Demand -> il client scarica il codice dal server e lo esegue.
4. Mobile Agents -> l'agente parte dal client, si sposta su uno o più server e continua l'esecuzione.

Mobilità debole e forte del codice

Definizione

Nel modello di **code mobility**, un processo è composto da tre parti:

1. **Code segment** → istruzioni del programma.
2. **Execution segment** → stato di esecuzione (stack, variabili, PC).
3. **Resource segment** → risorse usate (file, DB, dispositivi).

Weak mobility

Si trasferisce codice ed eventuali dati iniziali, Il programma **riparte da zero** sul nodo destinazione.

Strong mobility

Si trasferisce codice e stato di esecuzione completo. Il programma **continua da dove era stato interrotto**.

Cloning: avviare un clone e impostarlo allo stesso stato di esecuzione dell'originale.

Migrazione nei sistemi eterogenei

Problema: la macchina di destinazione potrebbe non essere adatta a eseguire il codice migrato.

La definizione di contesto processo/thread/processore dipende dall'HW locale, dal SO e dal sistema di runtime.

La soluzione è usare una **macchina astratta** implementata su diverse piattaforme => linguaggi interpretati (con VM propria) o VMM.

Migrazione container

Poiché i container dipendono direttamente dal sistema operativo sottostante, la loro migrazione in ambienti eterogenei non è banale, o semplicemente impraticabile, proprio come la migrazione dei processi.

Migrazione di una VM

Abbiamo tre alternative:

1. Inviare le pagine di memoria alla nuova macchina e reinviare quelle modificate durante la migrazione.
2. Fermare la VM attuale, migrare la memoria e avviare la nuova VM.
3. Permettere alla nuova VM di inserire nuove pagine secondo necessità: i processi partono sulla nuova VM e copiano le pagine di memoria su richiesta.

Performance

Una migrazione completa può effettivamente richiedere decine di secondi.

Dobbiamo anche renderci conto che durante la migrazione un servizio sarà completamente indisponibile per diversi secondi.

3.5 CPU, Processi e Thread

Contesto della CPU

La **CPU** (Central Processing Unit) esegue istruzioni macchina di un determinato flusso di controllo.

Il **contesto della CPU** è l'insieme dello stato interno del processore in un dato momento: quello che deve ricordare per continuare l'esecuzione dal punto in cui si era fermato.

Questo stato risiede nei **registri hardware** del processore.

Definizione

Il contesto della CPU è lo **snapshot** istantaneo di tutti i registri del processore: è l'identità computazionale del flusso di esecuzione in un preciso istante temporale.

Registri del contesto

Registri di controllo

- **PC (Program Counter)** -> indica l'indirizzo della prossima istruzione da eseguire.
- **SP (Stack Pointer)** -> cima dello stack, insieme al PC definiscono la posizione del flusso di esecuzione nello spazio di indirizzamento.
- **FP (Frame Pointer)** -> punta alla base del frame, usato per l'accesso a parametri e variabili locali.
- **PSW (Program Status Word)** -> area di memoria contenente informazioni sullo stato dei programmi in esecuzione sul SO.

Registri generali e speciali

- **R0-R15** -> registri general-purpose.
- **FLAGS** -> Zero, Carry, Overflow...
- **CR3** -> puntatore alla page table.
- **CS/DS/SS** -> segment registers (*x86*).

MMU, TLB e registri

Per comprendere il contesto di processo, è necessario accennare ai meccanismi hardware che gestiscono la memoria: la **MMU** e la **TLB**.

Memory Management Unit (MMU)

La **MMU** è un'unità hardware interposta tra CPU e memoria fisica. Traduce gli *indirizzi virtuali* (usati dai processi) in *indirizzi fisici* nella RAM.

Ogni processo vede uno spazio di indirizzamento virtuale privato ($0 \rightarrow 2^{64}$ su architetture a 64 bit), ma le pagine fisiche possono essere condivise, protette o scambiate su disco.

❗ Implicazione

Ogni processo ha la propria **page table**, il cui puntatore è memorizzato nel registro CR3.
Al context switch tra processi, CR3 viene aggiornato, invalidando la cache della MMU.

Translation Lookaside Buffer (TLB)

Le page table risiedono in memoria principale, richiedendo **due accessi in memoria**: prima alla page table, poi al dato.

Per diminuire il costo la CPU mantiene un piccolo buffer associativo: **TLB**.

La TLB memorizza le traduzioni recenti (`pagina virtuale -> pagina fisica`).

- **TLB hit**: risolve le traduzioni in pochi cicli.
- **TLB miss**: impone una *page table walk*, più costosa.

Quando si effettua un context switch tra processi, la TLB deve esser **invalidata** (o taggata con un ASID, Address Space ID), poiché le traduzioni del vecchio processo non sono valide per il nuovo.

Contesto di processo

Un **processo** è un programma in esecuzione con le proprie risorse protette.

Il contesto di processo è più ricco del contesto della CPU: comprende tutte le risorse che servono al SO per gestire e riprendere l'esecuzione di esso.

❗ Contesto di processo

Il contesto di processo è formato da:

- Contesto CPU.
- Spazio di indirizzamento virtuale privato.
- Risorse SO (file aperti, socket, segnali...).
- Metadati (PID, priorità, credenziali).
- Stack kernel-side.

Contesto hardware

Comprende lo **stato della CPU** e dei vari registri (registri general-purpose, PC, SP, PSW, CR3).

Contesto di memoria

L'**address space** contiene:

- la page table;
- i segmenti (text, data, bss);
- stack utente;
- heap dinamico;
- memory-mapped files.

Contesto SO

Il **kernel state** contiene:

- PID, PPID, GID, UID dei processi;
- descrittori di file;
- segnali pendenti;
- statistiche di utilizzo;
- stack kernel dedicato.

Il contesto di processo è memorizzato nel **PCB (Process Control Block)**: una struttura dati del kernel che il sistema operativo mantiene per ciascun processo attivo.

In pratica contiene tutto il necessario per sospenderlo e riprenderlo.

Isolamento

Il design dei processi è pensato per il **massimo isolamento**: un processo non può accedere alla memoria di un altro senza meccanismi espliciti (IPC). Questa protezione è fondamentale nei sistemi distribuiti, dove i componenti devono essere fault-isolated, ma ha un costo significativo in termini di overhead del context switching.

Contesto di thread

Un **thread** è un'unità di esecuzione leggera che condivide lo spazio di indirizzamento e le risorse del processo padre.

Più thread possono coesistere all'interno dello stesso processo, eseguendo in parallelo su più core o in concorrenza su un singolo core.

📌 Contesto di thread

Il contesto di thread è **minimo**: il contesto CPU + lo stack privato. Tutto il resto (memoria, file, segnali) è condiviso con gli altri thread del processo.

Privato (per thread)

Il **thread context** comprende:

- i registri della CPU (PC, SP, FP...);
- stack di esecuzione privato;
- thread ID (*TID*);
- stato (*running/blocked/ready*);
- Thread-Local Storage (TLS).

Condiviso (dal processo)

Le **shared resource** condivise dal processo sono:

- lo spazio di indirizzamento;
- page table (stessa CR3);
- file descriptor table;
- codice (text segment);
- heap e dati globali.

Thread Control Block (TCB)

La **TLB** è formata da:

- TID e link al PCB padre.
- Saved register set.
- Stack base e size.
- Scheduling info.
- Puntatore TLS.

La differenza rispetto ai processi è che non c'è bisogno di cambiare lo spazio di indirizzamento. Il registro **CR3** rimane lo stesso, la TLB non deve essere invalidata. Il context switch tra thread dello stesso processo è quindi **radicalmente più economico**.

Context switching

Il **context switch** è l'operazione con cui il sistema operativo sospende l'esecuzione di un'entità (CPU context, processo, thread) e ne ripristina un'altra. È l'operazione fondamentale che consente il multitasking.

CPU Context

1. Hardware salva PC, CS e FLAGS sullo stack.
2. CPU carica il vettore di interrupt (`IDT[n]`) -> nuovo PC.
3. Transizione `user mode` -> `kernel mode` (bit nel PSW).
4. Esecuzione dell'Interrupt Service Routine (ISR).
5. Ripristino PC, CS, FLAGS dallo stack con IRET.
6. Ritorno al codice interrotto (nessun cambio spazio di indirizzamento).

Processo -> Processo

1. Kernel salva il contesto CPU di A nel suo PCB (tutti i registri).
2. Scheduler seleziona il processo B da eseguire.
3. Kernel carica il contesto CPU di B dai suoi PCB.
4. Aggiornamento CR3 => nuova page table (spazio di indirizzamento B).
5. TLB flush: tutte le traduzioni precedenti sono invalide.
6. CPU riprende l'esecuzione dal PC salvato di B.

Thread -> Thread

1. Kernel salva i registri CPU di T1 nel suoi TCB
2. Scheduler seleziona T2, stesso processo e stesso spazio di indirizzamento.
3. Kernel carica i registri CPU di T2 dal TCB.
4. CR3 invariato => nessun cambio di page table.
5. TLB NON invalidata, le traduzioni sono ancora valide.
6. Esecuzione ripresa dal PC di T2 (overhead minimo).

Costo comparativo

Non tutti i context switch hanno lo stesso costo. La tabella seguente riassume l'overhead relativo nelle tre tipologie:

Operazione	CPU (interrupt)	Processo	Thread
Salvataggio registri CPU	✓ (~ns)	✓ (~ns)	✓ (~ns)
Cambio page table (CR3)	×	✓ (costoso)	×

Operazione	CPU (interrupt)	Processo	Thread
TLB flush / invalidazione	×	✓ (molto costoso)	×
Cache pollution (L1/L2)	Parziale	Elevata	Bassa
Overhead kernel tipico	1–5 μ s	50–100 μ s	1–5 μ s
Creazione entità	—	~1 ms (fork)	~50 μ s

Thread vs processi

Nel contesto dei **sistemi distribuiti**, la scelta tra processi e thread ha implicazioni dirette su *scalabilità*, *latenza* e *utilizzo delle risorse*. I thread offrono vantaggi strutturali che li rendono spesso la scelta preferita per server ad alta concorrenza.

1. **Context switch leggero:** nessun cambio di spazio di indirizzamento, nessuna TLB flush. Il passaggio tra thread dello stesso processo è 10–100x più rapido rispetto al cambio tra processi. Cruciale per sistemi con migliaia di richieste concorrenti.
2. **Comunicazione a costo zero:** i thread condividono heap e dati globali: una variabile modificata da un thread è immediatamente visibile agli altri. Nessun bisogno di IPC (pipe, socket, shared memory) per la comunicazione intra-processo.
3. **Footprint di memoria ridotto:** un thread aggiuntivo richiede solo un nuovo stack (~1–8 MB) e un TCB. Un nuovo processo richiede la duplicazione dell'intero spazio di indirizzamento (anche con copy-on-write, il page table overhead è significativo).
4. **Creazione rapida:** `pthread_create()` è ordini di grandezza più veloce di `fork()`. Nei server che generano una nuova unità esecutiva per ogni richiesta (thread-per-request), questo vantaggio si moltiplica con il carico.
5. **Scalabilità su sistemi multi-core:** i thread dello stesso processo possono essere schedulati su core diversi con pieno accesso ai dati condivisi. La comunicazione tra core avviene via cache condivisa (L3), non via memoria principale o kernel.
6. **Sincronizzazione:** la memoria condivisa richiede meccanismi di sincronizzazione (mutex, semafori, monitor). La complessità di programmazione aumenta, ma i pattern moderni (lock-free, atomics) la gestiscono efficacemente.

3.6 Docker

Container

Esistono diverse tecnologie di **virtualizzazione leggera** su Linux, basate su:

- **cgroups** → gestione risorse
- **namespaces** → isolamento

Esempi principali:

- **LXC (Linux Containers)**
- **Docker**

Container vs Virtual Machine

Caratteristica	Virtual Machine	Container
Virtualizzazione	Hardware (emulazione)	Sistema operativo
OS	Ogni VM ha un OS completo	Condivide kernel host
Isolamento	Alto	Minore
Efficienza	Minore	Maggiore
Avvio	Lento	Molto veloce

LXC - Linux Containers

LXC è un insieme di strumenti per creare **system containers**

L'obiettivo è **simulare** un ambiente Linux completo senza avere un kernel separato.

Proprietà

- Pensato per **system administrator**
 - Un container LXC è simile a una VM **senza bootloader**.
 - Vantaggi:
 - Migrazione semplice da VM
 - **Nessuna modifica alle applicazioni**
 - Migliori performance
-

Cos'è Docker

Docker è un tool per creare, gestire ed orchestrare **container di applicazioni**.

L'obiettivo è ottimizzare sviluppo, testing e deployment, separando ogni componente dell'app in un container.

È possibile comporre più container in uno stack:

1. proxy Nginx
2. application server
3. DB

LXC è per amministratori di sistema, Docker è pensato per sviluppatori.

Non è sempre immediato passare da VM a Docker:

- Ambiente non è un UNIX completo
- Potrebbero servire modifiche.

Architettura Docker

Modello client-server

L'architettura client-server gestisce immagini, container, volumi e reti, di solito girano su macchine diverse.

Abbiamo una CLI per l'utente che comunica con una REST API.

La registry Docker Hub server per scaricare immagini.

Componenti principali

1. Image

Un'immagine è un **template** read-only contenente codice, librerie, runtime e configurazioni necessari per un componente software.

Un'immagine può essere creata da un archivio (tar) o tramite **dockerfile**: lista di istruzioni specificate in un file di testo.

Tramite il `.dockerfile` diamo istruzioni al comando di build per generare le immagini.

2. Registry

Database di immagini

- pubblico (DockerHub)

- privato

3. Container

Il container è l'**istante di esecuzione** dell'immagine.

Un container può fare diverse operazioni: start, stop, restart, distruzione, connessione a reti ed associazione di volumi.

Essendo che l'immagine è read-only, qualsiasi modifica al file system non modifica l'immagine.

È possibile definire il grado di isolamento di un container:

- limitazione utilizzo CPU
- limitazione allocazione memoria
- accesso filesystem.

4. Network

Le reti sono implementate da switch virtuali ed iptables.

I bridge limitano la connessione al container su un singolo host.

Le reti di overlay consentono la connettività dei container tra diversi host (*cluster*).

5. Volume

Un volume è una **directory esterna** al container che può essere condivisa anche tra più container, ma è indipendente dal suo ciclo di vita.

Utilizzato per condividere lo spazio di archiviazione tra diversi contenitori, o comunque lo storage associato al container dell'utente.

Posso agganciare a runtime o in fase di creazione questa cartelle.

Servono per mantenere il contenuto dei file generati dinamicamente (es.: un DBMS potrebbe scrivere delle informazioni).

Essendo che i container sono effimeri e quando vengono stoppati non mantengono niente, abbiamo bisogno di un volume.

6. Service

Un service Docker è un **gruppo di container che replicano la stessa immagine** e che offrono *bilanciamento del carico*.

Un service può essere aumentato o ridimensionato a seconda del carico di ingresso.

7. Stack

Un gruppo di servizi indipendenti che interagiscono tra loro per implementare un'applicazione completa.

☰ Stack completo

Es.: una web app per condividere foto potrebbe avere un servizio per lo storage e la ricerca di immagini, un servizio per la UI e un servizio per l'encoding delle immagini.

Single-host vs swarm mode

Di default, i container di un Docker stack sono messi in produzione solo sull'host in cui gira il `dockerd` daemon, tuttavia può essere configurato anche in **swarm mode**: si posizionano i container dello stack su tutti i nodi di un cluster.

Uno swarm è composto di uno swarm manager ed un gruppo di nodi swarm worker.

Docker under the hood

Namespaces

Ogni contenitore ha un'istanza di ogni tipo di namespace (`pid`, `net`, `ipc`, `mnt`, `uts`), per limitare l'ambito degli oggetti del kernel host visibili al container.

cgroups

Limitano la quantità di risorse assegnate al container.

Union file system

File system **a strati (layers)**, in cui i layer comuni sono riutilizzati dalle immagini e container, consumando meno spazio e velocizzando la creazione.

Dockerfile

Il Dockerfile è un file di testo che contiene le istruzioni per costruire un'immagine.

```
# Start from an official image with the Python runtime

FROM python:2.7-slim

# Set the container current working directory (PWD) to "/chess"

WORKDIR /chess

# Copy files from current host directory to the /chess directory in the container
```

```
ADD . /chess

# Install some packages

RUN apt-get update && apt-get install -y libcgroup acl

# Flag that the software inside this image listens on port 9000

EXPOSE 9000

# Define an environment variable

ENV PLAYER Ghost

# Specify the command to be run inside the container when it's started

CMD ["python", "./chess.py"]
```

Struttura di un'immagine Docker

Un'immagine è uno stack di layers (*onion-like structure*).

Ogni istruzione nel Dockerfile aggiunge un layer, ogni layer salva solo le differenze dal precedente.

La creazione del container crea un container layer read/write.

Condividere i layer

Quando un container è in running, tutte le modifiche del suo disco sono riflesse al container layer. Tutti gli altri layer sono read-only e possono essere condivisi tra più container.

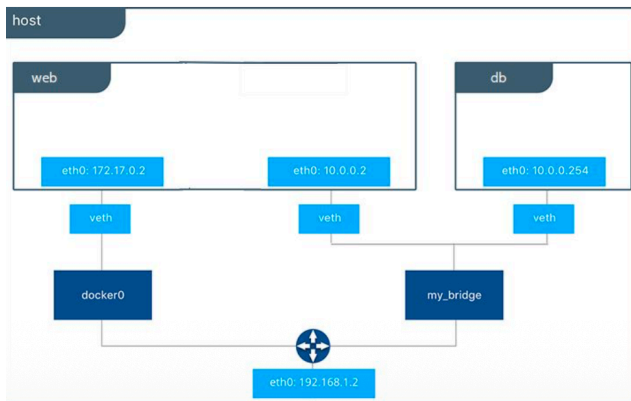
Implicazione dei layer

- Risparmio spazio, salvando solo differenze
 - Creazione container molto rapida
 - Layer comuni riutilizzati
-

Networking Docker

I bridge sono usati per connettere i container tramite interfacce virtuali (*veth*).

L'utente può creare e gestire nuove reti, connettere un container ad una o più reti (anche mentre sta girando).



Ogni rete usa una rete subnet diversa, visibile solo dall'host.

Di default le reti usano NAT + host-only.

Comandi Docker

Immagini

- `docker search {name}`
- `docker image ls` -> elenca immagini esistenti
- `docker pull` -> recupera un'immagine da una registry
- `docker rm` -> rimuove un'immagine localmente
- `docker inspect` -> dettagli dell'immagine
- `docker image prune` -> rimuove immagini non usate
- `docker build -t myimg` -> build dell'immagine dal Dockerfile

Container

- `docker run -it --name ubu1 ubuntu /bin/bash` -> creare un container e lanciarlo.
 - `ubuntu` si riferisce all'immagine
 - `/bin/bash` specifica il comando da eseguire dal container.
 - `-t` specifica l'allocazione di un terminale
 - `-i` comando interattivo (*shell*)
 - `-d` run del container in background
- `docker run -p 8000:80 apache /usr/bin/apacheserv --daemon` -> mappa ogni porta esposta ad una porta host, per abilitare l'accesso all'esterno della rete.
- `docker start`
- `docker stop`
- `docker ps` -> elenca i container attivi

- `-a` tutti gli stati
- `docker rm`

Volumi

- `docker volume create`
- `docker volume ls`
- `docker volume rm`

Possiamo creare dei container senza nome associati al container, ma indipendenti dal suo ciclo di vita:

- `docker run -v /myvol --name ubu1 -it ubuntu /bin/bash`

Possiamo anche collegarli ad altri container:

- `docker run --volumes-from ubu1 --name ubu2 -it ubuntu /bin/bash`

I volumi importati sono montati nel file system `ubu2` negli stessi punti di montaggio utilizzati all'interno di `ubu1`.

=> `ubu1` è un **data volume container**.

Reti virtuali

- `docker network ls` -> reti correnti
- `docker network create --subnet=192.168.13.0/24 mmynet` -> crea un bridge definito dall'utente.
- `docker network (dis)connect mynet ubu2`
- `docker run --network=mynet [options] imagename command` -> lanciare un container connesso ad una rete.
 - `docker run --network=mynet --ip 192.168.13.4 [options] imagename command` -> specificando un IP statico.

Altri comandi

- `docker system df` -> disk usage
- `docker system prune` -> rimuove container in stop, volumi non usati, reti ed immagini.
- `docker rm $(docker ps -a -q)` -> rimuove tutte le immagini del container (inclusa quella in uso)
 - `docker image remove $(docker images -q)`
- `docker stats`

3.7 Virtualizzazione, Paravirtualizzazione e Container

ISA (Instruction Set Architecture)

L'ISA è l'interfaccia astratta tra HW e SW, definisce:

- L'insieme delle **istruzioni macchina**.
- I **registri** visibili al programmatore.
- Modalità di **indirizzamento**.
- Modello di **memoria**.
- Meccanismo di gestione delle **eccezioni**.
- Livelli di **privilegio**.

Istruzioni privilegiate e non privilegiate

In un'ISA come x86 o ARM, le istruzioni si dividono in:

1. **Istruzioni non privilegiate** -> eseguite in modalità utente e non compromettono la sicurezza del sistema (*operazioni aritmetiche, accesso a memoria utente*).
2. **Istruzioni privilegiate** -> eseguite solo in modalità kernel, altrimenti generano una trap (*modifica di page table, I/O diretto, modifica dei registri di controllo, gestione interrupt*).

Ring di protezione

- **Ring 0** -> Kernel mode
- **Ring 3** -> User mode

Il SO gira in Ring 0, le applicazioni in Ring 3.

Trap

Un trap è un **trasferimento sincrono del controllo** dal codice in esecuzione al SO, a causa dell'esecuzione di un'istruzione.

L'ISA definisce quali istruzioni generano trap, il meccanismo di salvataggio del contesto, il vettore delle eccezioni ed il cambio di livello di privilegio.

Trap vs Interrupt vs Exception

Trap -> evento sincrono intenzionale e previsto dall'ISA (*es.: system call*).

Exception -> errore (*es.: divisione per zero*).

Interrupt -> evento asincrono esterno (*es.: arrivo di un pacchetto di rete*).

Trap e system call

La system call è una trap intenzionale.

```
User mode
↓
Istruzione syscall
↓
Trap
↓
Kernel mode
↓
Esecuzione codice della syscall
↓
Ritorno in user mode
```

Flusso della trap

1. Salvataggio automatico di:
 1. PC
 2. SR
 3. Registri minimi
2. Cambio di **stack** (stack kernel)
3. Cambio di livello di **privilegio**
4. Salto all'handler definito dall'Interrupt Descriptor Table.

Costo

Troppe trap riducono le prestazioni, in quanto:

- Flush parziale pipeline
- Cambio di stack
- Salvataggio contesto
- Invalidazione TLB

La paravirtualizzazione riduce le trap, le estensioni HW le rendono più efficienti.

Virtualizzazione

La virtualizzazione è la tecnica che permette di **eseguire più sistemi operativi isolati sulla stessa macchina fisica**, ciascuno convinto di avere accesso esclusivo all'hardware.

L'elemento centrale è l'hypervisor (VMM).

Condizione di virtualizzabilità

🔗 Teorema Popek Goldberg

Un'architettura è virtualizzabile se l'insieme delle istruzioni sensibili è sottoinsieme delle istruzioni privilegiate.

Se un'istruzione è **sensibile** ($\Rightarrow$ modifica lo stato critico della macchina) deve essere anche privilegiata, così se viene eseguita in modalità non privilegiata genera una trap e l'hypervisor può gestirla.

Full virtualization

- Il SO guest non viene modificato e viene spostato in un livello meno privilegiato.
- L'hypervisor gira in Ring 0.
- Le istruzioni privilegiate generano trap, che l'hypervisor intercetta ed emula.

Paravirtualizzazione

La paravirtualizzazione è una tecnica in cui il sistema operativo guest è modificato per essere **consapevole** della virtualizzazione.

Invece di eseguire istruzioni privilegiate direttamente, il guest effettua hypercalls e l'hypervisor esegue l'operazione per suo conto.

Nella virtualizzazione classica si cerca di intercettare automaticamente le istruzioni privilegiate ed il guest non sa di essere virtualizzato.

Nella paravirtualizzazione il guest NON esegue direttamente istruzioni privilegiate, ma le sostituisce con chiamate esplicite all'hypervisor e non si dipende dalla generazione di trap.

Container

📄 Container

Insieme di processi isolati tramite meccanismi del kernel.

- Non virtualizzano l'hardware.

- Non hanno un kernel proprio.
- Condividono il kernel dell'host.
- Usano meccanismi del kernel (*namespaces + cgroups*).

Paravirtualizzazione vs Container

La paravirtualizzazione virtualizza l'hardware (ISA, istruzioni privilegiate, livelli di privilegio).

-> Virtualizzazione di **macchine**.

I container NON virtualizzano l'hardware: isolano processi sopra un kernel condiviso.

-> Virtualizzazione di **ambienti di esecuzione**.

3.8 Live Migration delle VM

Definizione di Live Migration

Live Migration

La **live migration** è il processo di trasferimento di una macchina virtuale (VM) **attiva** da un host fisico a un altro senza interrompere il servizio.

Obiettivi principali

- Minimizzare o eliminare il **downtime percepito**
- Preservare completamente lo **stato della VM**
- Garantire **trasparenza** per utenti e applicazioni

Questo è un esempio concreto di **migration transparency**, cioè la capacità di spostare risorse senza che l'utente se ne accorga.

Stato di una VM

Durante la migrazione bisogna trasferire **tutto lo stato della macchina**:

- **CPU state**
 - registri
 - program counter
 - flag
- **Memoria RAM**
- **Dispositivi virtuali**
- **Connessioni di rete attive**
 - es. sessioni TCP
- **Storage**
 - soprattutto in ambienti *shared-nothing*

Lo stato è **grande e dinamico**, quindi difficile da trasferire senza impatti.

Metriche di valutazione

Per valutare una live migration si usano diverse metriche:

Metrica	Descrizione
Downtime	Tempo in cui la VM è sospesa
Total Migration Time	Durata totale della migrazione
Service Degradation	Impatto sulle performance
Network Overhead	Traffico generato
Convergence Time	Tempo per stabilizzare il sistema

Nei sistemi distribuiti, queste metriche riflettono trade-off tra:

- prestazioni
- trasparenza
- consistenza

Pre-Copy Migration

La memoria viene trasferita **mentre la VM continua a funzionare**.

1. Copia iniziale della memoria
2. Tracking delle **dirty pages** con bitmap delle pagine modificate
3. Copie iterative delle pagine modificate
4. Stop finale (breve) → trasferimento residuo

Condizione di convergenza: Dirty rate < Transfer rate

Post-Copy Migration

Si trasferisce solo lo **stato minimo**, poi la VM riparte subito.

1. Trasferimento iniziale:
 - CPU + metadati
2. Ripresa della VM sul nodo destinazione
3. Recupero pagine su richiesta (page fault remoto)

Vantaggi

- Nessuna copia ripetuta
- Minore traffico totale

Svantaggi

- Più sensibile a **failure di rete**
- Possibile degradazione iniziale

Qui emerge il problema delle **partial failures**, tipico dei sistemi distribuiti.

Hybrid Migration

- Pre-copy iniziale → riduce il working set
- Post-copy finale → trasferimento on-demand

Utilizzata soprattutto in ambienti KVM e Xen moderni.

Vantaggi

- Riduce traffico
 - Migliora efficienza
-

Implementazioni nei principali hypervisor

VMware vSphere (vMotion)

- Pre-copy ottimizzata
- Funzionalità:
 - Storage vMotion
 - compressione memoria
 - multi-NIC
 - encrypted migration
 - RDMA (Remote Direct Memory Access)

Xen / XCP-ng

- Pre-copy (storica)
- Supporto opzionale post-copy

- Usato in ambienti cloud

KVM / QEMU

- Pre-copy di default
- Supporta:
 - post-copy
 - hybrid
- Ottimizzazioni:
 - XBZRLE
 - auto-converge
 - trasferimento multithread

Microsoft Hyper-V

- Pre-copy con compressione
- SMB Direct (RDMA)
- Supporto:
 - shared-nothing migration
 - failover clustering

Confronto architetturale

Hypervisor	Caratteristica
VMware	soluzione enterprise integrata
Xen	microkernel
KVM	integrato nel kernel Linux
Hyper-V	type-1 con stack Microsoft

Problemi e sfide

1. Workload memory-intensive

- Alta frequenza di dirty pages
- Problemi di convergenza (pre-copy)

2. Compatibilità CPU

- Architetture diverse tra host

3. GPU passthrough

- Stato difficile da migrare

4. Latenza WAN

- Impatta performance
- aumenta downtime

5. Fault tolerance

- Critico soprattutto nel post-copy
-

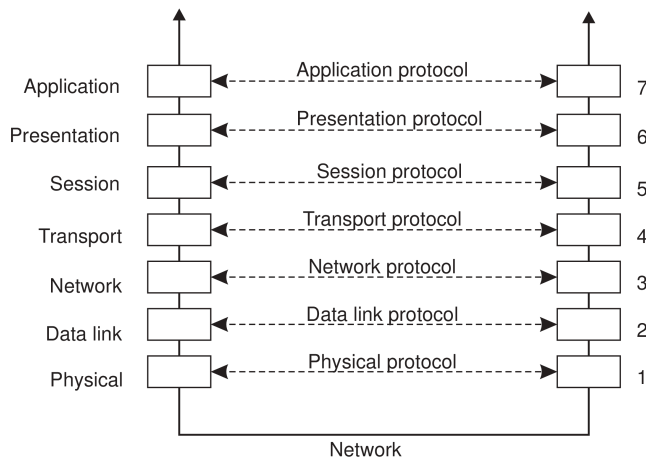
Conclusioni

La live migration è una tecnologia fondamentale nel **cloud computing**.

- **Pre-copy** → tecnica dominante
- **Post-copy** → utile in scenari specifici perché riduce overhead
- **Hybrid** → compromesso moderno
- Evoluzione verso:
 - RDMA
 - data center software-defined

4. Comunicazione

Protocolli a strati e modello OSI



Nel modello OSI c'è la composizione di **messaggi** a cui viene appeso un'header dal livello inferiore, fino al livello fisico.

All'interno di ogni livello c'è un API che interagisce con i dati provenienti dal livello superiore.

Svantaggi

- Focus solo sull'invio di messaggi.
- Spesso ci sono funzionalità non desiderate.
- Violazione della [trasparenza di accesso](#).

Physical, Data link e network layer

Il **livello fisico** contiene le specifiche per la trasmissione dei bit tra mittente e ricevente.

Il **data link** trasmette una serie di bit in un frame per consentire il controllo degli errori e del flusso.

Il **network layer** si occupa del routing dei pacchetti in rete.

Livello di trasporto

Il transport layer fornisce le strutture di comunicazione per i sistemi distribuiti.

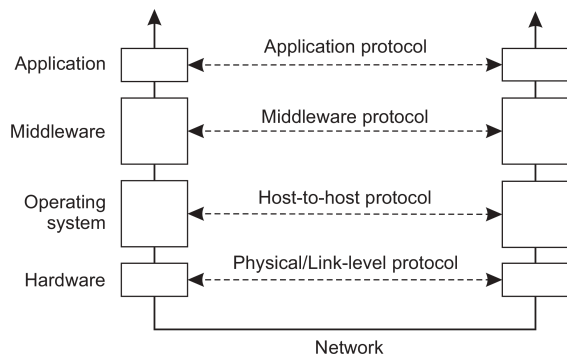
1. **TCP** -> connection-oriented, affidabile, orientato agli stream.
2. **UDP** -> best-effort di tipo datagram.

Livello middleware

Il middleware è stato introdotto per fornire servizi e protocolli **comuni** che possono essere riutilizzati da molte **applicazioni diverse**.

- Protocolli di **comunicazione**.
- **Marshaling** ed unmarshaling dei dati.

- Protocolli di **naming** per la condivisione di risorse.
- Protocolli di **sicurezza**.
- Meccanismi di **scaling** (replicazione e caching).



Il middleware ci serve per astrarre dall'utilizzo di paradigmi come Socket: utilizzarli direttamente può essere complicato.

Tipi di comunicazione

La comunicazione si distingue in:

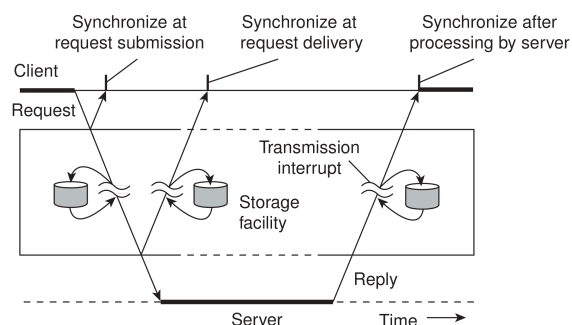
1. **Transitoria** -> il messaggio viene scartato se non può essere consegnato immediatamente o se il destinatario non è attivo. I router di rete tipicamente offrono solo questo tipo di comunicazione.
2. **Persistente** -> un messaggio che è stato inviato per la trasmissione viene memorizzato dal middleware fino alla consegna al destinatario.

Un'altra distinzione:

1. **Asincrona** -> il mittente continua dopo aver inviato il suo messaggio per la trasmissione, esso viene (temporaneamente) memorizzato dal middleware al momento dell'invio.
2. **Sincrona** -> il mittente è bloccato fino all'accettazione della richiesta.

Nella comunicazione sincrona abbiamo tre punti di sincronizzazione:

1. Il mittente può essere bloccato fino a quando il middleware non notifica che prenderà in carico *la trasmissione della richiesta*.
2. Il mittente può sincronizzare fino a quando la sua richiesta non è stata *consegnata* al destinatario.
3. La sincronizzazione può avvenire lasciando che il mittente aspetti fino alla *risposta*.



Client/Server

Client/Server si basa su comunicazione **transitoria sincrona**.

Gli svantaggi sono:

- Client bloccato al punto di sincronizzazione.
- I guasti vanno gestiti immediatamente.
- Il modello non si adatta al tipo di applicazione (chat, posta...).

Message-oriented middleware

In aiuto abbiamo i **middleware orientati ai messaggi**: implementato comunicazione asincrona persistente.

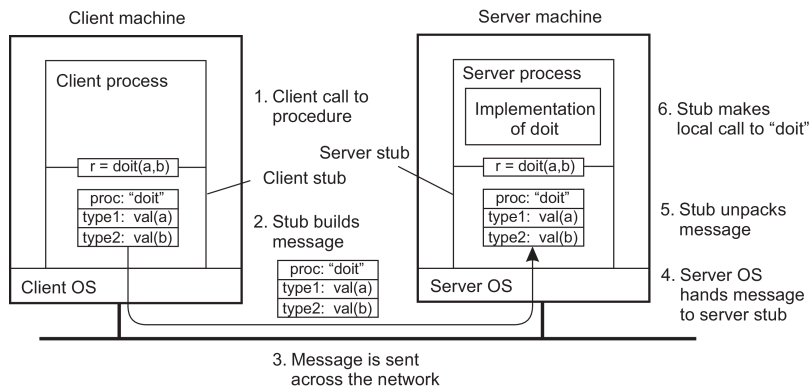
Vengono utilizzate delle [queue](#): i messaggi vengono messi in coda ed il middleware ci garantisce tolleranza ai guasti.

Remote Procedure Call

RPC

L'obiettivo della **RPC (Remote Procedure Call)** è rendere una chiamata a una procedura situata su una macchina remota del tutto simile a una chiamata locale, nascondendo i dettagli del passaggio dei messaggi (*trasparenza d'accesso*).

1. Il client chiama una procedura (locale)
2. Uno **stub** (un pezzetto di codice lato client) trasforma funzione + parametri in un formato trasmissibile (**marshaling**)
3. Invia i dati al server
4. L'OS del server riceve il messaggio e lo passa allo stub
5. Lo stub lato server (**skeleton**) fa **unmarshaling** e ricostruisce i parametri
6. Chiama la funzione reale sul server



Passaggio di parametri

Le macchine possono avere:

- architetture diverse (big-endian, little-endian)
- sistemi operativi diversi
- rappresentazioni diverse dei dati (byte ordering)

Client e server devono interpretare correttamente i messaggi, trasformandoli in formati indipendenti dalla macchina e dalla rete.

Meccanismo di riferimento remoto

Non si possono passare i parametri per riferimento: non conosco l'indirizzo di memoria sull'altra macchina, dobbiamo serializzare/deserializzare i dati.

Si possono usare gli skeleton remoti che restituiscono riferimenti virtuali, tenendo traccia dei puntatori ad oggetti remoti.

RPC asincrono

Questa variante viene utilizzata quando non è necessario restituire un risultato immediato al client o per evitare che il client rimanga inattivo durante l'elaborazione del server.

Non appena riceve la richiesta, il server invia un segnale di **accettazione (acknowledgment)** immediato al client.

Dopo aver ricevuto l'ACK, il client riprende la sua esecuzione senza attendere il completamento.

Il server esegue la procedura localmente dopo aver confermato la ricezione al client.

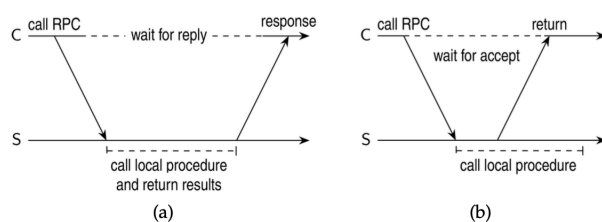


Figure 4.14: (a) The interaction between client and server in a traditional RPC. (b) The interaction using asynchronous RPC.

RPC sincrona differita

Viene utilizzata quando il client ha bisogno del risultato della procedura, ma vuole svolgere altro lavoro nel frattempo.

Spesso implementata tramite una **callback**, ovvero una funzione definita dall'utente che viene invocata automaticamente quando il server restituisce i risultati.

RPC multicast

Consiste nell'invio di una singola chiamata RPC a un **gruppo di server** contemporaneamente, spesso il client non sa nemmeno che la chiamata è indirizzata a più server (*trasparenza*); lo stub si occupa di gestire la comunicazione di gruppo.

Message-oriented communication

RPC e le invocazioni di oggetti remoti contribuiscono a nascondere la comunicazione nei sistemi distribuiti, migliorando la trasparenza dell'accesso.

Quando non si può presumere che il destinatario stia eseguendo un'operazione al momento dell'invio della richiesta, sono necessari servizi di comunicazione alternativi.

La **comunicazione message-oriented** rappresenta un'alternativa ai modelli basati su chiamate di procedura remota (RPC), nata per gestire situazioni in cui il pattern rigido client-server non è appropriato.

Mentre l'RPC mira a nascondere la comunicazione rendendo le chiamate remote simili a quelle locali, la comunicazione message-oriented si concentra sullo **scambio esplicito di messaggi**, offrendo maggiore flessibilità in termini di sincronizzazione e disponibilità delle parti.

Transient messaging con socket

I **socket** rappresentano l'interfaccia fondamentale per la **comunicazione transitoria** nei sistemi distribuiti, agendo come astrazione a basso livello dei protocolli del livello di trasporto (come TCP e UDP).

Socket

Un socket è un **punto finale (end point)** di comunicazione a cui un'applicazione può scrivere dati da inviare sulla rete e da cui può leggere i dati in arrivo. Tecnicamente, astrae la **porta** utilizzata dal sistema operativo locale per un determinato protocollo di trasporto.

A differenza dei sistemi di accodamento (message-queuing), i socket offrono una **comunicazione transitoria (transient)**. Questo significa che:

- Il messaggio viene memorizzato dal sistema di comunicazione solo finché le applicazioni mittente e ricevente sono in esecuzione.

- Se il middleware non può consegnare il messaggio immediatamente (ad esempio, se il destinatario non è attivo o c'è un'interruzione di rete), il messaggio viene semplicemente **scartato**.

Berkley socket interface

L'interfaccia socket standard (derivata da Berkeley Unix e POSIX) prevede una serie di operazioni specifiche per stabilire una comunicazione orientata alla connessione.

Operation	Description
socket	Crea un nuovo punto finale di comunicazione
bind	Associa un indirizzo locale (IP e porta) al socket creato
listen	(Solo lato server) Prenota le risorse per gestire un numero massimo di richieste di connessione in attesa.
accept	Chiamata bloccante che attende l'arrivo di una richiesta; al successo, restituisce un nuovo socket per la comunicazione specifica con quel client.
connect	Il client specifica l'indirizzo remoto e tenta di stabilire attivamente una connessione.
send	Invia i dati
receive	Riceve i dati
close	Rilascia la connessione simmetricamente

⚠ Limitazioni dei socket

- **Basso livello:** Richiedono al programmatore di gestire manualmente molti dettagli.
- **Sincronizzazione rigida:** Forzano un modello sincrono dove il mittente o il ricevente possono rimanere bloccati in attesa dell'altro.

Alternative

- **ZeroMQ:** Che astrae i socket attraverso **pattern di messaggistica** (*come request-reply o publish-subscribe*) e gestisce le riconessioni e le code, permettendo una comunicazione asincrona più fluida.
- **MPI (Message-Passing Interface):** Progettato per il calcolo parallelo ad alte prestazioni, offrendo un'astrazione più ricca rispetto ai semplici send/receive dei socket.

Transient messaging avanzato: ZeroMQ

ZeroMQ è un approccio avanzato alla messaggistica transitoria che astrae l'uso dei socket tradizionali organizzandoli secondo specifici **pattern di comunicazione** (*request-reply, publish-subscribe e pipeline*) e gestisce le connessioni in modo trasparente per lo sviluppatore.

Una caratteristica è la sua natura **asincrona**: un mittente può inviare un messaggio anche se il destinatario non è ancora attivo, poiché ZeroMQ accoda i messaggi e gestisce la trasmissione non appena la connessione diventa disponibile.

A differenza dei flussi di byte di TCP, ZeroMQ lavora con **messaggi discreti**, evitando al programmatore di dover gestire la dimensione dei dati in ricezione.

1. Request-Reply

Questo pattern è utilizzato per la comunicazione tradizionale **client-server**, simile al modello delle RPC.

- Un'applicazione client utilizza un socket di tipo **REQ** (request) per inviare richieste, mentre il server utilizza un socket **REP** (reply) per rispondere.
- Elimina la necessità di gestire manualmente le operazioni di `listen` e `accept`.
- Quando un server riceve un messaggio tramite un socket **REP**, la successiva operazione di invio **indirizza automaticamente** la risposta al client originale.

2. Publish-Subscribe

Questo pattern implementa una comunicazione di tipo **multicast** da un server a molti client.

- Il server utilizza un socket di tipo **PUB** (*publisher*) per trasmettere i dati, mentre i client utilizzano socket **SUB** (*subscriber*) per riceverli.
- Per impostazione predefinita, un socket SUB non riceve nulla; il client deve specificare esplicitamente i **tag o gli argomenti** (es. "TIME") di suo interesse.
- Se un publisher invia messaggi per i quali non esistono sottoscrittori attivi, tali informazioni vengono perse.

3. Pipeline

Il pattern pipeline è progettato per distribuire compiti tra diversi processi, ottimizzando l'elaborazione parallela.

- Un processo produttore (*farmer*) utilizza un socket **PUSH** per distribuire il lavoro, mentre uno o più processi lavoratori (*worker*) utilizzano socket **PULL** per ricevere i compiti.
- Il sistema è strutturato affinché il **primo lavoratore disponibile** prelevi il risultato prodotto, garantendo che tutti i processi rimangano operativi il più possibile.
- L'obiettivo è spingere i risultati attraverso una pipeline di processi nel modo più rapido possibile, senza che il produttore debba preoccuparsi di quale specifico lavoratore preleverà il compito.

Transient messaging avanzato: MPI

L'**MPI (Message-Passing Interface)** è uno standard per la comunicazione a passaggio di messaggi, progettato specificamente per applicazioni parallele che richiedono alte prestazioni ed è indipendente dall'hardware.

❶ Operazioni rappresentative

- **MPI_BSEND (Buffered Send)**: Operazione asincrona in cui il messaggio viene copiato in un buffer locale del sistema MPI, permettendo al mittente di proseguire immediatamente la sua esecuzione.
- **MPI_SEND**: Un'operazione di invio bloccante la cui semantica dipende dall'implementazione; può bloccare il mittente finché il messaggio non è stato copiato localmente o finché il ricevente non ha avviato la ricezione.
- **MPI_SSEND (Synchronous Send)**: Il mittente si blocca finché la trasmissione non ha inizio o finché la richiesta non viene accettata per l'elaborazione.
- **MPI_SENDRECV**: La forma più forte di comunicazione sincrona, simile a una **RPC**; il mittente invia una richiesta e si blocca finché non riceve una risposta dal destinatario

Modello di comunicazione

MPI assume che la comunicazione avvenga all'interno di un gruppo noto di processi.

Una coppia (**groupID**, **processID**) identifica univocamente la sorgente o la destinazione di un messaggio al posto di un indirizzo di rete tradizionale.

MPI è orientato alla comunicazione transitoria e assume che guasti gravi (come il crash di un processo) siano fatali e non richiedano un recupero automatico da parte del middleware.

Il middleware MPI mantiene i propri buffer e la sincronizzazione è spesso legata al momento in cui i dati vengono copiati dall'applicazione al middleware stesso.

Queue-based messaging

Il **Queue-based messaging** (messaggistica basata su code) supporta una comunicazione di alto livello che è:

1. **Persistente** -> il sistema di comunicazione memorizza il messaggio in una coda per il tempo necessario alla consegna.
2. **Asincrona** -> il mittente non deve attendere la ricezione o elaborazione; può continuare la propria esecuzione dopo aver inserito il messaggio nella coda.

Con questo approccio abbiamo 4 possibili combinazioni: sender e receiver attivi, entrambi passivi o uno attivo e l'altro passivo.

Operazioni

Operazione	Descrizione
PUT	Aggiunge il messaggio in una coda specifica
GET	Blocca finché la coda specificata non è vuota e recupera il primo messaggio
POLL	Controlla una coda specifica per i messaggi e recupera il primo

Operazione	Descrizione
NOTIFY	Installa un handler da chiamare quando un messaggio è inserito in una coda specifica

Message broker

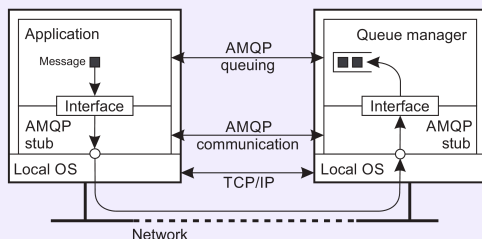
Poiché le applicazioni in un sistema distribuito possono utilizzare formati di dati e protocolli differenti, vengono spesso introdotti i **Message Broker**.

Questi agiscono come gateway a livello applicativo con i seguenti compiti:

- **Convertono** i messaggi in arrivo nel formato comprensibile dal destinatario.
- **Instradano** i messaggi in base a regole specifiche o soggetti ([modello publish subscribe](#)).

AMQP

Un esempio significativo di protocollo per la messaggistica basata su code è l'**Advanced Message Queuing Protocol (AMQP)**. AMQP è stato progettato per garantire l'interoperabilità tra diverse implementazioni di sistemi di messaggistica, gestendo connessioni, canali per il multiplexing e garantendo l'affidabilità della consegna dei messaggi attraverso stati di conferma (settlement).

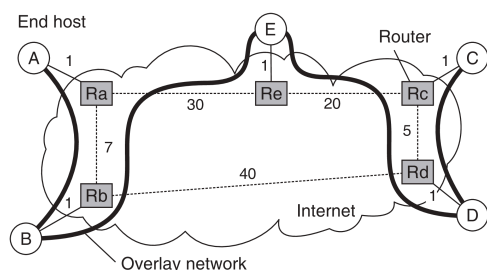


Application-level multicasting

Poiché le connessioni tra i nodi dell'overlay attraversano diversi collegamenti fisici, il routing dei messaggi potrebbe non essere ottimale rispetto a quanto ottenibile a livello di rete. Esistono due approcci principali per costruire queste reti:

- **Basato su alberi (Tree-based):** I nodi si organizzano in un albero dove esiste un percorso unico tra ogni coppia di nodi. Un esempio è il sistema **Scribe**, costruito su Chord, dove un iniziatore genera un identificatore di multicast e il nodo responsabile di tale identificatore diventa la radice dell'albero.
- **Basato su maglie (Mesh-based):** I nodi formano una rete a maglie in cui ognuno ha più vicini e percorsi multipli. Questo approccio è generalmente **più robusto** in caso di guasti ai nodi, poiché permette di continuare la diffusione dei dati senza dover riorganizzare immediatamente l'intera rete.

ALM: Costi



L'efficienza di un albero di multicast a livello applicativo viene misurata attraverso tre criteri:

- **Link Stress (Sforzo del collegamento):** Conta quante volte un pacchetto attraversa lo stesso collegamento fisico (ad esempio, se l'overlay passa due volte sullo stesso router fisico).
- **Stretch o Relative Delay Penalty (RDP):** È il rapporto tra il ritardo sperimentato nella rete overlay e quello che si verificherebbe nella rete sottostante.
- **Costo dell'albero (Tree Cost):** Misura il costo totale del mantenimento e dell'uso dell'albero per la diffusione dei dati.

Flooding

Il **flooding** (*inondazione*) è una tecnica di diffusione dei dati nei sistemi distribuiti, in cui un nodo mittente invia un messaggio a tutti i suoi vicini, i quali a loro volta lo inoltrano a tutti i propri vicini (tranne quello da cui è arrivato), a patto di non averlo già ricevuto in precedenza.

In un grafo con $|V|$ nodi e $|E|$ archi, il flooding "puro" genera un totale di $2|E| - |V| + 1$ messaggi.

-> Risulta ottimale solo se la rete è organizzata come un **albero** (dove i messaggi sono esattamente $|V| - 1$). In reti densamente collegate, il numero di messaggi può crescere in modo quadratico ($O(|V|^2)$), portando a un enorme spreco di risorse.

Variazioni

Per mitigare i costi elevati, vengono adottate diverse strategie:

- **Probabilistic Flooding:** Un nodo inoltra il messaggio con una probabilità *pflood*. Questa tecnica riduce drasticamente il traffico (ad esempio, una riduzione di 50 volte in una rete di 10.000 nodi con *pflood* = 0.01), ma introduce il rischio che non tutti i nodi vengano raggiunti.
- **Time-To-Live (TTL):** Per evitare che i messaggi circolino all'infinito, si assegna un valore massimo di "salti" (hops) oltre i quali il messaggio viene scartato. Una variante è l'**iterative TTL**, dove si inizia con un raggio corto e lo si espande solo se non si trovano risultati.
- **Flooding Strutturato:** Se la topologia della rete è nota (come in un **ipercubo** o un **anello Chord**), è possibile progettare schemi di flooding che raggiungano tutti i nodi con esattamente $N - 1$ messaggi, eliminando i duplicati.

Protocolli epidemici

I **protocolli epidemici**, noti anche come protocolli di **gossiping**, sono algoritmi progettati per diffondere rapidamente informazioni in sistemi distribuiti su larga scala utilizzando solo **informazioni**

locali e senza alcun coordinamento centrale. Il loro nome deriva dall'analogia con la teoria delle epidemie, che studia la diffusione delle malattie infettive: in ambito informatico, l'obiettivo è "infettare" tutti i nodi con nuovi dati nel minor tempo possibile.

Modelli di Propagazione

Esistono due strategie principali per lo scambio di informazioni:

1. **Anti-entropy (Anti-entropia):** un nodo P sceglie casualmente un altro nodo Q e scambia con lui gli aggiornamenti per rendere i loro stati identici. Questo scambio può avvenire in tre modi:
 - **Push:** P invia i propri aggiornamenti a Q.
 - **Pull:** P preleva i nuovi aggiornamenti da Q.
 - **Push-Pull:** entrambi si scambiano i dati a vicenda (l'approccio più efficiente). Si è dimostrato che la strategia push-pull permette di raggiungere tutti gli N nodi in un tempo proporzionale a $O(\log(N))$ round.
2. **Rumor Spreading (Diffusione di pettegolezzi):** se un nodo P riceve un aggiornamento, tenta di inviarlo a un nodo casuale Q. Se Q possiede già quell'informazione, P potrebbe smettere di diffonderla con una probabilità *pstop*. Questo modello è eccellente per la velocità, ma non garantisce matematicamente che tutti i nodi vengano aggiornati.

Cancellazione dei dati

il problema principale della **cancellazione dei dati** risiede nel fatto che la rimozione di un'informazione è difficile da propagare correttamente e può essere involontariamente annullata.

In un sistema basato su epidemie, i nodi si scambiano aggiornamenti per mantenere la coerenza. Se un nodo si limita a rimuovere un dato dal proprio archivio locale, non rimane alcuna traccia di tale operazione. Di conseguenza, quando quel nodo contatterà un altro peer che possiede ancora una vecchia copia di quel dato, interpreterà tale copia come un **nuovo aggiornamento** di un elemento che non ha mai visto, finendo per memorizzarlo nuovamente.

Soluzione: certificati di morte

Per risolvere questo paradosso, la cancellazione non deve essere una rimozione silenziosa, ma deve essere trattata come un **particolare tipo di aggiornamento**.

Viene creato un **certificato di morte**, ovvero un record che attesta esplicitamente la cancellazione di un dato item.

Questo certificato si propaga tra i nodi proprio come un normale aggiornamento, informando tutti che il dato non è più valido.

Tuttavia se i certificati venissero conservati per sempre, ogni nodo accumulerebbe un database enorme di informazioni storiche su dati cancellati. Esistono due strategie per pulirli:

1. **Garbage Collection globale:** un algoritmo che rileva quando la notizia della cancellazione ha raggiunto ogni nodo del sistema, permettendo la rimozione sicura del certificato.
2. **Certificati dormienti (TTL):** si assegna ai certificati un TTL. Dopo questo periodo, si assume che tutti abbiano ricevuto la notizia e il certificato viene rimosso, anche se ciò comporta il rischio che

qualche nodo isolato possa non essere mai stato informato.

5. Coordinazione

Nei sistemi distribuiti **non esiste un tempo globale condiviso**. Ogni macchina ha un proprio orologio e **non sono perfettamente sincronizzati**.

Questo crea problemi per logging, auditing, ordinamento degli eventi e coordinazione tra processi

| L'obiettivo è capire “quando” avvengono gli eventi e in che ordine.

Sincronizzazione degli orologi

UTC (Coordinated Universal Time)

UTC è una misura del tempo **assoluta** e non basata sulla rivoluzione terrestre.

Gli **orologi atomici** misurano le oscillazioni di un isotopo radioattivo, ogni secondo oscilla ~9 milioni di volte.

Questo sistema è coordinato a livello planetario: ci sono 50 orologi atomici e si fa la media tra essi, ottenendo la misura UTC distribuita a livello mondiale.

Per compensare la rotazione della Terra **non perfetta**, si introduce il **Leap second (secondo intercalare)**.

Clock hardware

Un PC abbiamo la CPU che tiene il tempo all'avvio ed una RAM specifica (non volatile) collegata ad una batteria statica che al momento dello spegnimento/accensione segna l'orario e al riavvio inizia a contare, con un suo riferimento temporale.

Ci sono due **registri** appositi della CPU per contare il tempo che passa, la CPU ha vari clock interni e ogni tick di frequenza viene decrementato il contatore, quando raggiunge 0 viene generata un interrupt della CPU.

Internamente alla macchina, la data non è una stringa (=> millenium bug), ma un intero molto lungo a 32 bit: i **secondi trascorsi** dal 1 gennaio 1970.

Con 32 bit ci sarà un overflow nel 2038, infatti adesso è a 64 bit (miliardi di anni).

| Il tempo per noi è un intero (**UNIX Epoch time**).

Clock drift

Gli orologi non sono perfetti, possono andare più veloci (**fast clock**) o più lenti (**slow clock**), causando:

- **clock skew** = differenza tra due orologi;
- deriva accumulata (es. ~32 secondi/anno).

Metriche di qualità del tempo

Precisione

La **precisione** (π) indica quanto due macchine sono sincronizzate tra loro:

$$\forall t, \forall p, q : |C_p(t) - C_q(t)| \leq \pi$$

con $C_p(t)$ l'ora dell'orologio calcolata della macchina p all'ora UTC t .

Accuratezza

L'**accuratezza** (α) è la differenza rispetto al tempo reale (UTC):

$$\forall t, p : |C_p(t) - t| \leq \alpha$$

Network Time Protocol (NTP)

Viene introdotto il Network Time Protocol per **sincronizzare gli orologi**.

È un sistema distribuito gerarchico (a livelli = stratum) che usa UDP per propagare il tempo dall'alto verso il basso.

Livello	Descrizione
Stratum 0	Orologi atomici, GPS
Stratum 1	Server collegati a stratum 0
Stratum 2+	Client/Server sincronizzati

⚠ Limite del tempo fisico

Tuttavia, anche con NTP, non si ottiene una sincronizzazione perfetta, dati i ritardi di rete, jitter ed errori.

Per le applicazioni che necessitano di ordine preciso degli eventi **non basta il tempo fisico** (es.: *sistemi distribuiti con messaggi, multicast o eventi concorrenti*).

Orologi logici

Ciò che di solito conta non è che tutti i processi siano d'accordo sull'ora esatta, ma che siano d'accordo sull'ordine in cui si verificano gli eventi

I clock logici si basano sul timestamp, ma bisogna mettere in relazione i timestamp per mettere in ordine gli eventi.

Quando sono in multicast con diversi sistemi, se mando un messaggio ed un clock, come mi coordinano con altre macchine per avere la certezza che queste operazioni avvengano con una certa sequenza?

Relazione Happened-Before

Questa relazione introduce un **ordinamento parziale** degli eventi in un sistema con processi operativi simultanei.

- Se a e b sono due eventi che avvengono nello stesso processo ed a accade prima di b , allora a **precede** b : $a \rightarrow b$.
- Se a è l'invio di un messaggio e b è il destinatario, allora $a \rightarrow b$.
- **Transitività**: se $a \rightarrow b$ e $b \rightarrow c$, allora $a \rightarrow c$.

Orologi logici (Lamport)

L'obiettivo è **assegnare timestamp** $C(e)$ ad ogni evento e tali che:

1. Se a e b sono due eventi nello stesso processo e $a \rightarrow b$, allora $C(a) < C(b)$.
2. Se a corrisponde all'invio di un messaggio m e b alla ricezione di tale messaggio, allora anche $C(a) < C(b)$.

Per allegare un timestamp a un evento (anche se non c'è un orologio globale) bisogna mantenere un insieme di orologi logici, uno per processo.

Ogni processo P_i mantiene un contatore locale C_i :

1. **Evento locale**: P_i incrementa il contatore prima di ogni evento ($C_i \leftarrow C_i + 1$).
2. **Invio messaggio**: Il messaggio m include il timestamp $ts(m) = C_i$.
3. **Ricezione messaggio**: Quando P_j riceve m , aggiorna il proprio orologio come $C_j \leftarrow \max\{C_j, ts(m)\} + 1$.

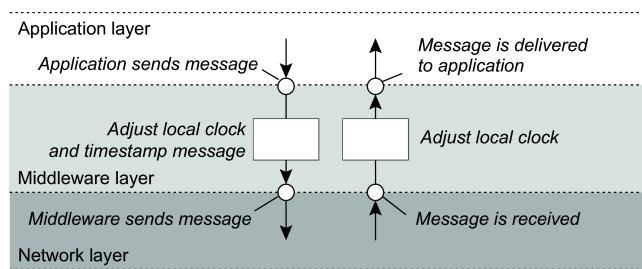
Se **due timestamp sono uguali** bisogna distinguere quale avviene prima e quale dopo => si inserisce l'ID del processo.

Senza clock logici i timestamp non sono confrontabili: se P_1 manda un messaggio a tempo 6 e P_2 riceve a tempo 16, non si sa quando è successo realmente facendo riferimento al contatore locale.

Aggiustamenti implementati nel middleware

Il middleware nei sistemi distribuiti aggiunge timestamp logico ed ID del processo, gestendo: **invio**, **ricezione** ed **aggiornamento** clock.

L'applicazione finale gestisce solo invio e ricezione dei messaggi.



Multicast totalmente ordinato

Il multicast totalmente ordinato è un'operazione di comunicazione di gruppo in cui **tutti i messaggi vengono consegnati a ciascun ricevitore esattamente nello stesso ordine**

In assenza di un ordine totale, i messaggi inviati contemporaneamente da diversi mittenti potrebbero arrivare ai destinatari in ordini differenti a causa dei ritardi di rete.

DB Replicato

Un esempio classico è quello di un *database bancario replicato* su due siti (es. New York e San Francisco):

- Supponiamo che un cliente versi 100€ (Operazione 1) e che contemporaneamente la banca applichi l'1% di interessi (Operazione 2).
- Se la replica di San Francisco esegue prima il deposito e poi l'interesse, il saldo finale sarà diverso rispetto alla replica di New York che, a causa di un ritardo, potrebbe eseguire prima l'interesse e poi il deposito.
- Il multicast totalmente ordinato risolve il problema assicurando che **entrambe le repliche eseguano le operazioni nella stessa sequenza**, mantenendo i dati identici ovunque.

Uno dei modi per implementare il multicast totalmente ordinato in modo distribuito è l'utilizzo degli **orologi logici di Lamport**. Assumendo una comunicazione FIFO:

1. Ogni processo mittente assegna al messaggio un **timestamp** basato sul proprio orologio logico locale e lo **invia** a tutti i membri del gruppo, incluso se stesso.
2. Quando un processo riceve un messaggio, lo inserisce in una **coda locale**, ordinata in base al timestamp.
3. Il ricevitore invia un messaggio di conferma (**acknowledgment**) a tutti gli altri processi del gruppo.
4. Un processo consegna il messaggio alla propria applicazione solo se sono soddisfatte due condizioni:
 - Il messaggio si trova in **testa alla coda** locale.
 - Il processo ha ricevuto, da ogni altro membro del gruppo, un messaggio (di qualsiasi tipo, anche un ACK) con un **timestamp superiore** a quello del messaggio in testa alla coda.

Vector clocks

Sebbene gli orologi di Lamport permettano di stabilire un ordine totale degli eventi, essi non catturano la **causalità**.

In un sistema con orologi di Lamport, se l'evento a accade prima di b ($a \rightarrow b$), allora il timestamp di a sarà minore di quello di b ($C(a) < C(b)$).

Tuttavia, non vale il viceversa: se osserviamo due timestamp tali che $C(a) < C(b)$, non possiamo concludere con certezza che a abbia causato b o sia avvenuto prima di esso; i due eventi potrebbero essere del tutto indipendenti (concorrenti).

Gli orologi vettoriali nascono proprio per risolvere questo problema, permettendo di determinare se due eventi sono causalmente correlati o se sono avvenuti in parallelo senza influenzarsi a vicenda.

📌 Causalità

La causalità potenziale tra due eventi significa che l'occorrenza del secondo evento potrebbe essere stata influenzata dalle informazioni prodotte dal primo.

Un orologio vettoriale per un sistema composto da n processi è rappresentato da un vettore di n numeri interi. Ogni processo P_i mantiene il proprio vettore VC_i con le seguenti proprietà:

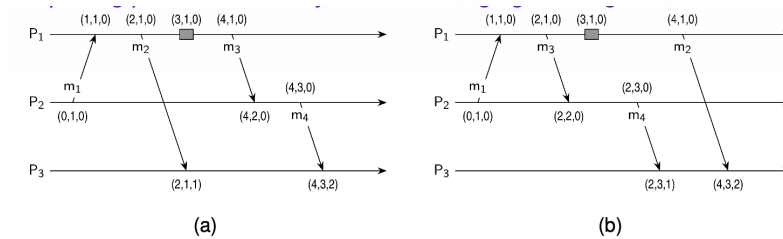
1. $VC_i[i]$: Indica il numero di eventi accaduti localmente nel processo P_i .
2. $VC_i[j]$ (con $j \neq i$): Rappresenta la conoscenza che il processo P_i ha del tempo logico (ovvero del numero di eventi accaduti) presso il processo P_j .

Ogni processo tiene traccia non solo del proprio "progresso", ma anche **dell'ultimo progresso noto** di tutti gli altri partecipanti al sistema.

Regole di aggiornamento

1. **Evento Locale:** prima di eseguire un evento interno (che sia l'invio di un messaggio, la ricezione o un'istruzione locale), il processo P_i incrementa la propria componente nel vettore: $VC_i[i] \leftarrow VC_i[i] + 1$.
2. **Invio di Messaggi:** quando P_i invia un messaggio m , esso allega l'intero vettore VC_i al messaggio come timestamp $ts(m)$, "trasporta" la storia causale del mittente al destinatario.
3. **Ricezione di Messaggi:** Quando il processo P_j riceve un messaggio m con timestamp $ts(m)$:
 - Aggiorna ogni componente del proprio vettore prendendo il massimo tra il valore corrente e quello ricevuto: $VC_j[k] \leftarrow \max(VC_j[k], ts(m)[k])$ per ogni k .
 - Successivamente, incrementa la propria componente locale $VC_j[j]$ prima di consegnare il messaggio all'applicazione.

Rilevamento della concorrenza



Analysis

Situation	$ts(m_2)$	$ts(m_4)$	$ts(m_2) < ts(m_4)$	$ts(m_2) > ts(m_4)$	Conclusion
(a)	(2, 1, 0)	(4, 3, 0)	Yes	No	m_2 may causally precede m_4
(b)	(4, 1, 0)	(2, 3, 0)	No	No	m_2 and m_4 may conflict

La vera potenza degli orologi vettoriali risiede nella capacità di confrontare due timestamp per determinare la relazione tra gli eventi.

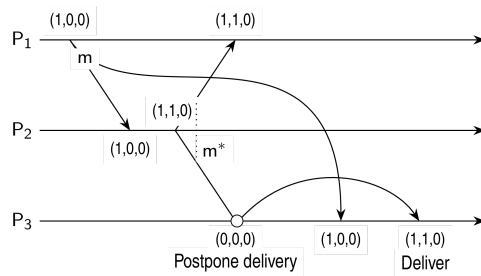
- $ts(a) \leq ts(b)$ se e solo se per ogni componente k , $ts(a)[k] \leq ts(b)[k]$.
- $ts(a) < ts(b)$ se $ts(a) \leq ts(b)$ e almeno una componente di $ts(a)$ è strettamente minore della corrispondente in $ts(b)$.
- **Causalità:** Se $ts(a) < ts(b)$, allora l'evento a precede causalmente b ($a \rightarrow b$).
- **Concorrenza (Conflitto):** Se non è vero che $ts(a) < ts(b)$ né che $ts(b) < ts(a)$, allora i due eventi sono **concorrenti**. Ciò significa che sono avvenuti indipendentemente e nessuno dei due ha potuto influenzare l'altro.

Multicasting ordinato causalmente

L'obiettivo è garantire che un messaggio venga consegnato a un'applicazione solo se tutti i messaggi che lo precedono causalmente sono già stati ricevuti e consegnati. Se due messaggi non sono correlati causalmente (ovvero sono concorrenti), l'ordine di consegna può variare tra i diversi nodi.

Per implementare questo meccanismo, i processi utilizzano gli orologi vettoriali con regole di aggiornamento specifiche:

1. **Invio:** Un processo P_i incrementa la propria componente $VC_i[i]$ solo quando invia un messaggio.
2. **Ricezione:** Quando un processo P_j riceve un messaggio m da P_i con timestamp $ts(m)$, la consegna all'applicazione viene ritardata finché non sono soddisfatte due condizioni critiche:
 - $ts(m)[i] = VC_j[i] + 1$: assicura che m sia il messaggio successivo che P_j si aspetta da P_i , evitando salti nella sequenza del mittente.
 - $ts(m)[k] \leq VC_j[k]$ per ogni $k \neq i$: garantisce che P_j abbia già consegnato tutti i messaggi provenienti da altri processi che P_i aveva già visto prima di inviare m .



Esempio

In una chat, se Alice (P1) invia un messaggio m e Bob (P2) lo riceve e risponde con m^* , il sistema deve garantire che ogni altro partecipante (es. Carlo, P3) veda m prima di m^* . Se m^* arriva a Carlo prima di m , Carlo vedrebbe una risposta a un messaggio non ancora apparso.

Mutua esclusione

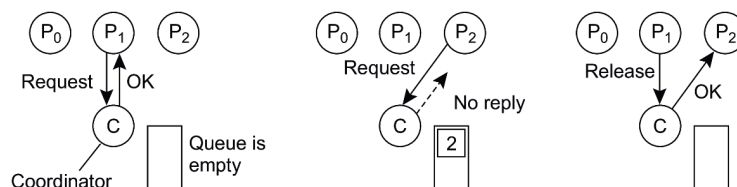
In un sistema distribuito, la mutua esclusione è necessaria per impedire che processi concorrenti corrompano risorse condivise accedendovi simultaneamente. Gli algoritmi si dividono principalmente in **basati su permessi** e **basati su token**.

Permission-based, centralizzato

È la simulazione di un sistema a processore singolo. Un processo viene eletto come **coordinatore**.

Per accedere a una risorsa, un processo invia una richiesta al coordinatore. Se la risorsa è libera, il coordinatore risponde con un OK; altrimenti, mette la richiesta in coda e non risponde (bloccando il richiedente) finché la risorsa non viene rilasciata.

- **Vantaggi:** Semplice (solo 3 messaggi: richiesta, concessione, rilascio) ed equo.
- **Svantaggi:** Punto singolo di fallimento (*Single Point of Failure*) e potenziale collo di bottiglia.



Ricart e Agrawala

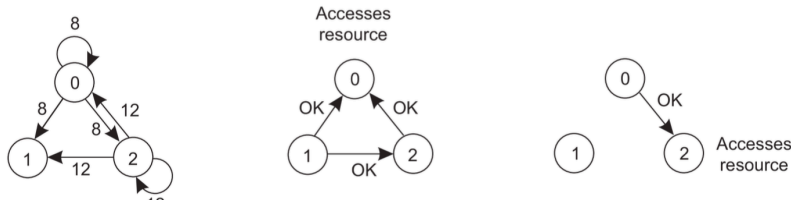
Ispirato dagli orologi di Lamport, non richiede un coordinatore centrale.

Un processo che vuole accedere alla risorsa invia un messaggio in multicast a tutti i membri del gruppo, includendo il proprio timestamp logico.

Un processo che riceve la richiesta risponde immediatamente con un OK solo se non è interessato alla risorsa o se ha una priorità inferiore (timestamp più alto). Altrimenti, rimanda la risposta.

Il richiedente entra nella regione critica solo dopo aver ricevuto l'OK da **tutti** gli altri processi.

- **Svantaggi:** Ha N punti di fallimento (se un nodo crasha e non risponde, blocca tutti gli altri) e richiede la gestione completa dell'appartenenza al gruppo.



Algoritmo Token Ring

I processi sono organizzati in un anello logico.

Un **token** circola costantemente nell'anello. Solo chi possiede il token può accedere alla risorsa. Una volta finito (o se non interessato), passa il token al vicino.

- **Vantaggi:** Evita la *starvation* (il token gira in ordine definito).
- **Svantaggi:** Problemi se il token viene perso o se un processo nell'anello crasha (richiede riconfigurazione dell'anello).

Algoritmo Decentralizzato (Voto di Maggioranza)

Utilizzato quando la risorsa è replicata N volte.

Ogni replica ha un coordinatore. Per accedere alla risorsa, un processo deve ottenere il voto della maggioranza ($m > N/2$) dei coordinatori.

È progettato per gestire crash temporanei dei coordinatori, ma presenta rischi di violazione se troppi coordinatori resettano contemporaneamente perdendo memoria dei permessi già concessi.

Riepilogo

Algoritmo	Messaggi per entrata/uscita	Ritardo prima dell'entrata (MTTU)
Centralizzato	3	2
Distribuito	$2(N - 1)$	$2(N - 1)$
Token Ring	da 1 a ∞	da 0 a $N - 1$
Decentralizzato	variabile ($> 3N$)	variabile

Esempio: ZooKeeper

ZooKeeper è un servizio di coordinamento logicamente centralizzato ma fisicamente distribuito. Il suo obiettivo primario è fornire un'astrazione che permetta a **processi concorrenti di collaborare** senza incorrere nelle tipiche problematiche di race condition o di inconsistenza dei dati.

- **Assenza di primitive bloccanti:** le operazioni di ZooKeeper non bloccano il chiamante; se un'operazione non può essere completata, il sistema restituisce immediatamente un errore o una notifica di fallimento.
- **Tolleranza ai guasti:** opera tipicamente attraverso un gruppo di server chiamato **ensemble**. Finché la maggioranza dei server è operativa, il servizio rimane disponibile.
- **Trasparenza:** Per i client, l'intero ensemble appare come un singolo server altamente affidabile.

Race condition

ZooKeeper organizza i dati in uno spazio dei nomi gerarchico, strutturato come un albero, in modo del tutto analogo al file system di Unix. Ogni nodo di questo albero è chiamato **znode**.

Esistono due categorie fondamentali di nodi che ne determinano il ciclo di vita:

1. **Nodi Persistenti:** creati e cancellati esplicitamente dalle applicazioni client.
2. **Nodi Effimeri:** legati alla sessione del client che li ha creati. Se la connessione tra il client e ZooKeeper si interrompe o scade, il nodo viene rimosso automaticamente dal sistema.

Per implementare un lock, un processo tenta di creare un nodo effimero chiamato `/lock`. Se l'operazione riesce, il processo detiene il lock. Se il processo fallisce, il nodo effimero scompare, "rilasciando" automaticamente il lock per gli altri contendenti.

Versionamento

Per gestire la coordinazione in modo efficiente, ZooKeeper utilizza due strumenti sofisticati:

- **Watchers (Notifiche):** ZooKeeper non è bloccante, i client non restano in attesa su un'operazione. Invece, possono sottoscrivere a un nodo per ricevere una notifica quando questo cambia o viene eliminato. Questo evita il *polling* continuo, migliorando drasticamente le prestazioni.
- **Versionamento:** per evitare che un client aggiorni un dato basandosi su informazioni obsolete, ZooKeeper associa ogni nodo a un numero di versione. Un client può richiedere di scrivere un valore solo se la versione corrente corrisponde a quella che lui ha letto in precedenza. Se un altro client ha modificato il nodo nel frattempo, la versione sarà aumentata e l'aggiornamento del primo client fallirà.

Algoritmi di elezione

In un sistema distribuito, molte operazioni richiedono che un processo agisca come coordinatore. L'obiettivo di questi algoritmi è garantire che, al termine di una procedura di elezione, tutti i processi attivi concordino su chi sia il nuovo leader.

Presupposti

Perché un'elezione abbia luogo, dobbiamo definire alcune regole di base:

- Ogni processo P possiede un identificativo unico $id(P)$.

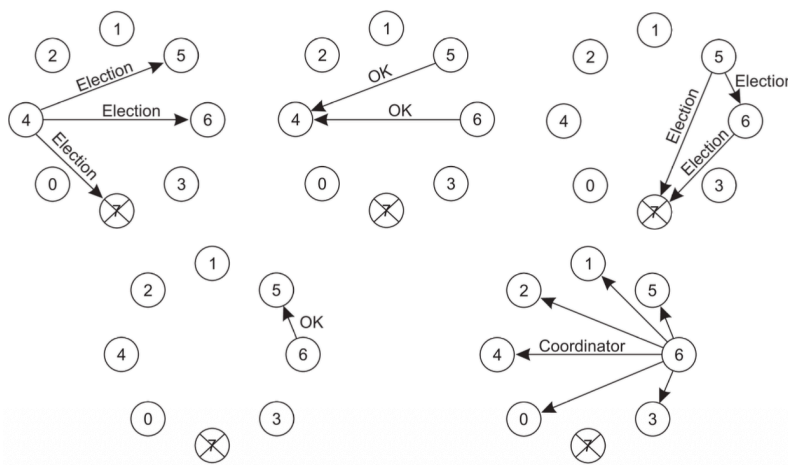
- In genere, l'obiettivo è eleggere il processo con l'**identificativo più alto** tra quelli attualmente in funzione.
- Ogni processo conosce gli identificativi di tutti gli altri membri del gruppo, ma non sa quali siano effettivamente attivi o guasti in un dato momento.

Bully algorithm

Viene chiamato "bullo" perché il processo con l'ID più alto prevale sugli altri.

Quando un processo P_k si accorge che il coordinatore non risponde più, avvia un'elezione:

1. P_k invia un messaggio di **ELECTION** a tutti i processi che hanno un identificativo superiore al suo ($P_{k+1}, P_{k+2}, \dots, P_{N-1}$).
2. Se nessuno risponde entro un certo tempo, P_k vince l'elezione e diventa il nuovo coordinatore.
3. Se uno dei processi superiori risponde con un messaggio **OK**, quest'ultimo prende in mano la gestione dell'elezione e il compito di P_k termina.
4. Il vincitore finale annuncia la sua vittoria inviando a tutti un messaggio di **COORDINATOR**.



Ring algorithm

Questo approccio sfrutta una struttura logica ad anello dove ogni processo conosce il proprio successore.

1. Quando un processo rileva il guasto del coordinatore, costruisce un messaggio di **ELECTION** contenente il proprio ID e lo invia al successore attivo più vicino.
2. Ogni processo che riceve il messaggio aggiunge il proprio identificativo alla lista contenuta nel messaggio e lo inoltra.
3. Quando il messaggio torna all'iniziatore (che riconosce il proprio ID nella lista), la lista contiene gli ID di tutti i processi attivi.
4. L'iniziatore trasforma il messaggio in **COORDINATOR**, indicando come leader il processo con l'ID più alto della lista, e lo fa circolare nuovamente per informare tutti.

Gossip-based coordination

Aggregazione

In un sistema distribuito di grandi dimensioni, la coordinazione centralizzata è spesso impossibile a causa dei colli di bottiglia e dei singoli punti di guasto.

Il **gossiping** mira invece a propagare informazioni e coordinare attività usando solo informazioni locali, senza alcun componente centrale.

Una delle applicazioni più potenti della coordinazione gossip è l'**aggregazione**:

1. Ogni nodo P_i inizia con un valore v_i .
2. Quando due nodi P_i e P_j entrano in contatto, aggiornano i rispettivi valori alla media locale:
$$v_i, v_j \leftarrow (v_i + v_j)/2.$$
3. Dopo pochi round di scambi casuali, tutti i nodi della rete avranno lo stesso valore, che corrisponderà alla media globale.

Se un solo nodo imposta il suo valore a 1 e tutti gli altri a 0, alla fine dell'aggregazione ogni nodo avrà il valore $1/N$. Invertendo il valore ($1/v_i$), ogni nodo potrà stimare autonomamente quanti altri nodi compongono il sistema.

Peer-sampling

Perché il gossiping funzioni, ogni nodo deve poter scegliere un altro nodo *a caso* dall'intera rete. In un sistema con milioni di nodi, conoscere l'elenco completo di tutti i partecipanti è impossibile.

La soluzione è il **Peer-Sampling Service**, costruito esso stesso tramite gossiping. Ogni nodo mantiene una **vista parziale** (un elenco di c vicini).

Periodicamente, i nodi scambiano alcuni elementi delle loro viste parziali.

Statisticamente, è dimostrato che selezionare un nodo dalla vista parziale aggiornata in questo modo è indistinguibile dallo scegliere un nodo uniformemente a caso dall'intera rete.

Gossip-based overlay

La coordinazione gossip permette anche di auto-organizzare i nodi in strutture specifiche (**overlay**), come anelli o griglie (tori 2D).

Il sistema usa un approccio a due livelli:

- **Livello inferiore:** nodi casuali (Peer-Sampling Service).
- **Livello superiore:** seleziona, tra i nodi casuali ricevuti, quelli che meglio soddisfano un criterio geometrico o di vicinanza (ad esempio, quelli più vicini in una griglia logica).

In questo modo, partendo da una configurazione totalmente casuale, la rete "cristallizza" in pochi round verso la topologia desiderata.

Secure gossiping

Il gossiping ha una vulnerabilità: la velocità di diffusione rende rapida anche la propagazione di informazioni false.

In un **attacco Hub**, un gruppo di attaccanti restituisce sistematicamente solo riferimenti ad altri attaccanti durante gli scambi di gossip. In meno di 300 round, in una rete di 100.000 nodi, gli attaccanti possono isolare i nodi onesti e prendere il controllo totale del traffico (**attacco Eclipse**).

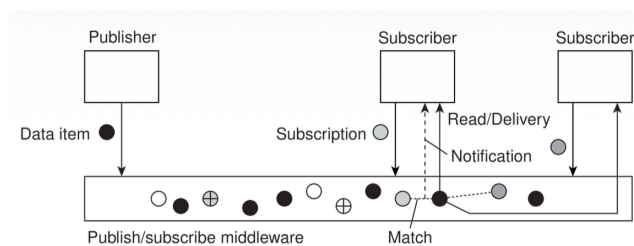
Soluzione: La raccolta di statistiche sulla distribuzione dei gradi di ingresso (*indegree*). Se un nodo nota che alcuni riferimenti appaiono troppo spesso rispetto a una distribuzione normale, può identificare e ignorare gli attaccanti.

Corrispondenza di eventi distribuiti

Il *distributed event matching* (o filtraggio delle notifiche) si basa su due pilastri fondamentali:

1. **Sottoscrizione (S):** un processo specifica gli eventi a cui è interessato.
2. **Notifica (N):** quando un processo pubblica una notifica sulla comparsa di un evento, il sistema deve verificare se S corrisponde a N .

Esiste una funzione logica, chiamata $\text{match}(S, N)$, che restituisce *vero* se la sottoscrizione corrisponde alla notifica, e *falso* in caso contrario. Il compito del sistema distribuito è facilitare questo accoppiamento e inviare la notifica (e i relativi dati) al sottoscrittore.



Implementazioni centralizzate e partizionate

In una soluzione centralizzata, un unico server gestisce tutte le sottoscrizioni e notifiche. Sebbene sia la soluzione canonica per sistemi come gli spazi di tuple *Linda*, non è scalabile.

Per aumentare la scalabilità, si può dividere il lavoro deterministicamente tra più server tramite due funzioni:

- $\text{sub2node}(S)$: mappa una sottoscrizione a un insieme di server.
- $\text{not2node}(N)$: mappa una notifica a un insieme di server. Il sistema garantisce che per ogni sottoscrizione e relativa notifica, esista almeno un server in comune (nodo di *rendezvous*) capace di gestire l'accoppiamento.

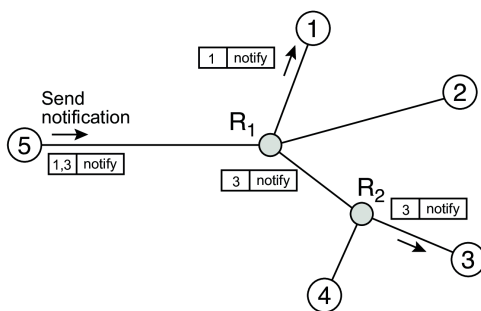
🔗 Osservazione

- **Topic-based:** Le sottoscrizioni e le notifiche sono etichettate con una parola chiave o un "topic" (es. "meteo.italia"). Il matching è semplice poiché si riduce a un controllo di uguaglianza tra stringhe.
- **Content-based:** È molto più complesso e potente. Qui la sottoscrizione può includere espressioni su attributi, come intervalli di valori (es. "temperatura >30") o prefissi di stringhe. Questo livello di espressività rende difficile distribuire la funzione di matching tra più server senza creare colli di bottiglia

Selective routing

In questa architettura, i server (broker) sono organizzati in una rete *overlay*. Invece di inondare la rete con notifiche, si installano dei **filtri di routing** nei router dell'overlay.

1. Ogni broker trasmette le proprie sottoscrizioni agli altri.
2. I router compongono questi filtri in tabelle di routing (es. "Invia notifiche con $a \in$ verso il router R2").
3. Le notifiche vengono inoltrate solo lungo i percorsi dove sono presenti sottoscrittori interessati.



Interface	Filter
To node 3	$a \in [0, 3]$
To node 4	$a \in [2, 5]$
Toward router R1	(non specificato)

Gossiping Sub-2-Sub

Per combinare scalabilità ed espressività nel *content-based matching*, si può usare il gossiping. Il sistema **Sub-2-Sub** modella le sottoscrizioni come sottoinsiemi in un **iperspazio N-dimensionale** di numeri a virgola mobile

- I nodi scambiano regolarmente le proprie sottoscrizioni tramite gossip.
- Se due nodi notano che i loro interessi si intersecano, mantengono riferimenti reciproci e si raggruppano in un overlay specifico per quell'intervallo di interessi.

- Una notifica pubblicata viene diffusa rapidamente solo all'interno dell'anello di nodi interessati a quel particolare segmento dell'iperspazio.

Secure publish-subscribe

In un sistema publish-subscribe tradizionale, i broker devono poter esaminare il contenuto dei messaggi (o i loro tag) per instradarli correttamente ai subscriber interessati. Tuttavia, questo introduce tre dilemmi critici:

1. Se un publisher cifra il messaggio per proteggerlo dai broker, questi ultimi non possono più leggerne il contenuto per decidere a chi inviarlo.
2. Il sistema deve garantire l'anonimato delle parti (**disaccoppiamento referenziale**), ma senza sapere chi ha inviato un messaggio è difficile verificare se sia stato manomesso da un malintenzionato.
3. Spesso i broker sono gestiti da terze parti (es. in ambienti cloud) che sono considerate "honest-but-curious": seguono il protocollo, ma potrebbero tentare di profilare gli interessi degli utenti o leggere dati sensibili.

La sicurezza deve bilanciare la necessità dei broker di avere informazioni per il routing con il diritto di publisher e subscriber alla privacy e alla protezione dei dati.

Public-Key Encryption with Keyword Search (PEKS)

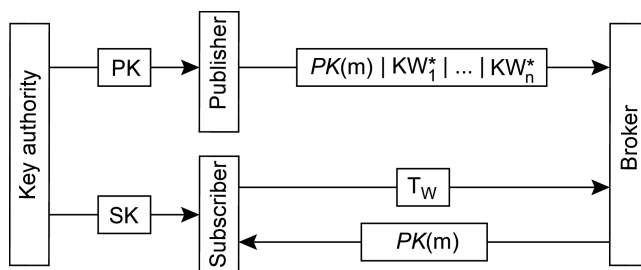
Una tecnica fondamentale per risolvere il conflitto tra cifratura e routing è la **Public Key Encryption with Keyword Search (PEKS)**

Il sistema si basa su una **Key Authority** fidata che genera le chiavi per le parti.

- Il publisher invia un messaggio m cifrato con una chiave pubblica PK . Al messaggio sono allegati dei tag cifrati singolarmente tramite la funzione PEKS:

$$m^* = [PK(m) \mid PEKS(PK, KW_1) \mid \dots \mid PEKS(PK, KW_n)]$$

- Il subscriber riceve dalla Key Authority una **trapdoor** T_W per una specifica parola chiave W a cui è interessato.
- Il broker riceve m^* e T_W . Usando la trapdoor, può verificare se W è tra i tag del messaggio senza però poter decifrare né il contenuto del messaggio, né le altre parole chiave. Se c'è un match, inoltra il messaggio cifrato al subscriber, che userà la propria chiave privata SK per leggerlo.



Sistemi di localizzazione

Posizionamento dei nodi

In un sistema distribuito su larga scala, conoscere la **posizione** (fisica o logica) dei nodi è fondamentale per ottimizzare le prestazioni, ad esempio per posizionare le repliche dei dati vicino agli utenti o per instradare i messaggi in base alla prossimità

Global Positioning System (GPS)

Il GPS si basa sulla **misurazione del tempo** che un segnale impiega per viaggiare da un satellite al ricevitore. Ogni satellite è dotato di orologi atomici estremamente precisi e trasmette continuamente la propria posizione e il timestamp locale.

1. Il tempo di propagazione del segnale non è istantaneo.
2. L'orologio del ricevitore non è sincronizzato con quello atomico dei satelliti.

Sia Δ_r la deviazione dell'orologio del ricevitore rispetto al tempo reale.

Quando un messaggio arriva dal satellite S_i con timestamp T_i , il ritardo misurato Δ_i è composto dal ritardo reale più la deviazione del ricevitore: $\Delta_i = (T_{now} - T_i) + \Delta_r$.

La distanza misurata $\tilde{d}_i$ è quindi $\tilde{d}_i = d_i + c \cdot \Delta_r$, dove d_i è la distanza reale e c la velocità della luce. Per trovare la posizione (x_r, y_r, z_r) e la deviazione temporale Δ_r , il sistema deve risolvere un insieme di equazioni quadratiche con quattro incognite.

Localizzazione indoor e WiFi

Il GPS non funziona efficacemente all'interno degli edifici. Una soluzione moderna sfrutta i punti di accesso WiFi (**AP**) già esistenti.

Per localizzarsi tramite WiFi, è necessario disporre di un database di coordinate degli AP. Questo database viene spesso popolato tramite il **war driving**: individui che percorrono aree urbane con dispositivi WiFi e ricevitori GPS, registrando la posizione in cui rilevano determinati SSID o indirizzi MAC.

La posizione di un AP viene stimata calcolando il **centroide** ($\vec{x}_{AP}$) delle N posizioni GPS in cui è stato rilevato:
$$\vec{x}_{AP} = \frac{\sum_{i=1}^N \vec{x}_i}{N}.$$

Sebbene popolare, questo metodo soffre di imprecisioni dovute alla portata non uniforme dei router e alla scarsità di campionamenti.

Posizionamento logico: Network Coordinates (NCS)

Nei sistemi distribuiti, spesso non ci interessa la posizione geografica assoluta, ma la **prossimità logica**. Un *Network Coordinates System* (NCS) assegna a ogni nodo una coordinata in uno spazio m -dimensionale in modo che la distanza geometrica rifletta una metrica di prestazione reale, solitamente la **latenza**.

- **Content Delivery Networks (CDN)**: Reindirizzare un client al server replica geometricamente più vicino per minimizzare i tempi di risposta.

- **Posizionamento Ottimale delle Repliche:** Calcolare le N posizioni migliori per replicare i contenuti basandosi sulla densità dei client.
- **Routing basato sulla posizione:** Inoltrare i messaggi al vicino che è geometricamente più vicino alla destinazione finale.

Implementazione delle coordinate di rete

Esistono due approcci principali per calcolare queste coordinate logiche.

1. **Approccio centralizzato (Landmark):** si utilizzano N nodi speciali chiamati **landmark**. Questi misurano le latenze reciproche e un nodo centrale calcola le loro coordinate minimizzando una funzione di errore aggregato. Un nuovo nodo P misura la sua distanza dai landmark e calcola le proprie coordinate di conseguenza.
2. **Approccio decentralizzato (Vivaldi):** l'algoritmo **Vivaldi** modella la rete come un sistema di molle. La differenza tra la latenza misurata e la distanza calcolata tra due nodi indica quanto la "molla" è tesa o compressa. Ogni nodo P_i aggiorna la propria posizione seguendo questi passaggi:
 1. Misura la latenza $\tilde{d}_{ij}$ verso un nodo P_j e riceve le sue coordinate $\vec{x}_j$.
 2. Calcola l'errore e e la direzione del vettore forza $\vec{u}$.
 3. Sposta la propria coordinata $\vec{x}$ i lungo il vettore forza per ridurre la tensione del sistema.

6. Naming

Nomi, Indirizzi e Identificatori

Per comprendere il naming, dobbiamo distinguere chiaramente tre concetti che spesso vengono confusi.

- **Nome:** È una stringa di bit o caratteri usata per riferirsi a un'entità (un processo, un file, un utente, una pagina web).
- **Indirizzo:** È il nome del **punto di accesso** (*access point*) di un'entità. Poiché un'entità può cambiare il suo punto di accesso nel tempo (pensa a un laptop che cambia rete Wi-fi), l'indirizzo non è un buon identificatore stabile.
- **Identificatore (Identifier):** È un tipo speciale di nome con tre proprietà cruciali:
 1. Si riferisce al massimo a un'entità.
 2. Ogni entità è riferita al massimo da un identificatore.
 3. Un identificatore non viene mai riutilizzato e punta sempre alla stessa entità.

Un sistema di naming efficace deve saper risolvere un nome (astratto) in un indirizzo (concreto) per permettere l'operatività.

Flat Naming

Nel **Flat Naming**, gli identificatori sono stringhe di bit casuali (spesso chiamate *pure names*) che non contengono alcuna informazione sulla posizione dell'entità. Il problema principale è come trovare l'indirizzo di un'entità se il suo nome non ci dà indizi su dove si trovi.

Ci sono varie soluzioni.

Broadcasting

Si invia un messaggio a tutte le macchine chiedendo "Chi ha l'identificatore X?". Solo la macchina interessata risponde con il proprio indirizzo. È semplice ma non scala oltre le reti locali (LAN).

☰ Address Resolution Protocol (ARP)

Questo principio viene utilizzato nell'ARP per trovare l'indirizzo MAC di una macchina quando viene fornito solo un indirizzo IP.

Forwarding pointers

Quando un'entità si sposta da A a B, lascia in A un **puntatore** verso B.

È facile da implementare: non appena un'entità è stata localizzata, un client può cercare l'indirizzo corrente seguendo la catena di puntatori di inoltro.

Tuttavia soffre di latenza elevata se le catene diventano lunghe e manca di tolleranza ai guasti (se un anello della catena si rompe, l'entità è persa).

Approcci Home-based

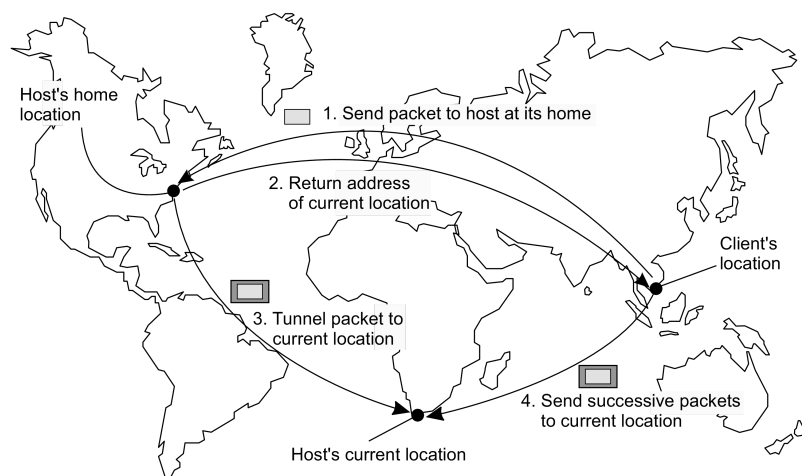
Un'entità ha una posizione fissa (**casa**) che tiene traccia della sua posizione attuale.

Il **Care-of Address** è l'indirizzo temporaneo che l'entità ottiene quando si trova in una rete straniera.

È il modello usato dal *Mobile IP*, ogni host mobile utilizza un indirizzo IP fisso, tutte le comunicazioni a quell'indirizzo IP sono inizialmente dirette all'home agent dell'host mobile.

I limiti sono:

1. **Scalabilità Geografica e Latenza:** Se un client si trova a Roma e l'host mobile si trova a Roma, ma la "casa" è a New York, il traffico deve comunque viaggiare fino a New York per essere reindirizzato.
2. **Single Point of Failure:** Se l'Home Agent smette di funzionare, l'entità mobile diventa irraggiungibile per chiunque, anche se l'entità stessa è perfettamente operativa nella sua rete attuale.
3. **Indirizzo Fisso Permanente:** L'indirizzo di casa deve essere mantenuto per tutta la durata della vita dell'entità.



Distributed hash tables

Sistemi come [Chord](#) organizzano i nodi in un anello logico. Ogni entità ha una chiave k gestita dal nodo con l'ID immediatamente superiore o uguale a k (il *successore*). Grazie a una *finger table* (scorciatoie), la risoluzione avviene in $O(\log N)$ passaggi, dove N è il numero di nodi.

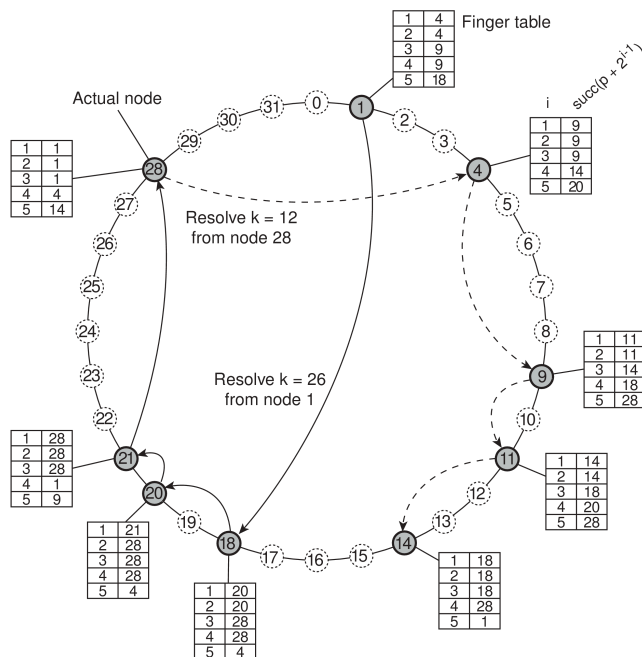
Ogni nodo p mantiene una **finger table** $FT_p[]$ con massimo m entries:

$$FT_p[i] = succ(p + 2^{i-1})$$

Per cercare una chiave k , il nodo inoltra la richiesta al nodo con indice j che soddisfa:

$$q = FT_p[j] \leq k < FT_p[j+1]$$

Se $p < k < FT_p[1]$, la richiesta è inoltrata a $FT_p[1]$



Sfruttare la vicinanza della rete

Sorge un problema di latenza fisica: due nodi che sono vicini logici nell'anello di Chord possono trovarsi ai lati opposti del pianeta. Una risoluzione che richiede tre salti logici potrebbe tradursi in tre trasferimenti di messaggi transatlantici, aumentando il tempo di risposta.

Per risolvere queste inefficienze e sfruttare la **prossimità di rete**, Chord e i sistemi derivati utilizzano tre strategie principali:

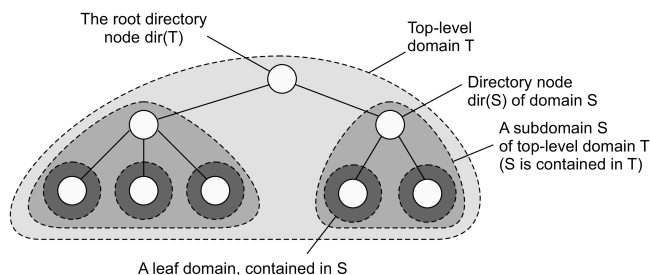
1. **Topology-aware node assignment**: assegnare gli identificatori ai nodi in modo che nodi fisicamente vicini abbiano ID vicini nello spazio delle chiavi. Estremamente difficile.
2. **Proximity Routing**: i nodi mantengono più alternative per ogni voce della finger table. Quando deve inoltrare una richiesta, il nodo sceglie tra i candidati quello che ha la latenza minore rispetto a se stesso, assicurandosi però di non "oltrepassare" la chiave cercata nell'anello.
3. **Proximity neighbor selection**: quando un nodo entra nel sistema o aggiorna la sua tabella, non si limita a inserire il primo nodo disponibile per uno slot. Se il protocollo permette di scegliere tra diversi nodi per riempire una posizione nella tabella, il nodo sceglierà sistematicamente quello fisicamente più vicino.

Hierarchical Location Services (HLS)

L'idea di base di un HLS è dividere la rete in una gerarchia di **domini** non sovrapposti. Ogni dominio è rappresentato da un nodo directory separato.

- **Domini Foglia (Leaf Domains)**: il livello più basso, solitamente corrispondente a una rete locale (LAN) o a una cella di telefonia mobile.
- **Gerarchia di Domini**: i domini foglia sono raggruppati in domini di livello superiore, che a loro volta formano domini ancora più vasti, fino ad arrivare a un unico **dominio radice (root)** che copre l'intera rete.

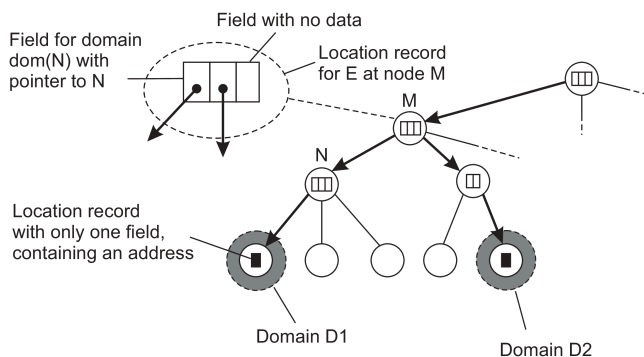
- **Nodi Direttorio (Directory Nodes):** Ogni dominio D ha un nodo direttorio associato, $dir(D)$, incaricato di tenere traccia delle entità presenti in quel particolare dominio.



HLS: Organizzazione dell'albero

Per garantire che la ricerca funzioni correttamente, l'albero deve rispettare tre regole logiche fondamentali, o invarianti:

1. L'indirizzo attuale di un'entità E è memorizzato nel nodo intermedio del dominio foglia in cui l'entità si trova.
2. Un nodo intermedio memorizza un puntatore a un suo figlio se e solo se l'entità è presente nel sottoalbero di quel figlio.
3. Il nodo radice contiene un record per ogni singola entità presente nel sistema, che punta al dominio di livello immediatamente inferiore dove l'entità può essere rintracciata.



Un'entità è rintracciabile seguendo una catena di puntatori che parte da un certo livello dell'albero (al limite dalla radice) e scende fino al dominio foglia che contiene l'indirizzo fisico.

HLS: Lookup

La risoluzione sfrutta la **località geografica**. Quando un client vuole localizzare l'entità E :

1. Invia una richiesta al nodo directory del proprio dominio foglia.
2. Se il nodo non conosce E , la richiesta sale al nodo padre (dominio più grande).
3. Il processo si ripete finché non si incontra un nodo M che possiede un record per E .
4. Da quel punto, si seguono i puntatori verso il basso attraverso i vari sotto-domini fino a raggiungere la foglia che contiene l'indirizzo cercato.

Questo meccanismo assicura che se l'entità è vicina al client, verrà trovata rapidamente senza disturbare i livelli alti della gerarchia.

HLS: Inserimento

Quando un'entità si sposta o viene creata una nuova replica, il sistema deve **aggiornare** i puntatori. L'inserimento di un indirizzo in un dominio foglia comporta l'invio di una richiesta verso l'alto fino a incontrare un nodo che conosce già l'entità; da lì si crea la nuova catena di puntatori verso il basso.

Scalabilità di HLS

Per scalare, il servizio distribuisce i nodi logici su una vasta collezione di host fisici:

- **Partizionamento per entità:** invece di avere un unico server per la radice, la responsabilità di gestire la radice per diverse entità è distribuita tra molti server diversi.
- **Hashing degli identificatori:** si può usare una funzione di hashing per decidere quale host fisico debba gestire il nodo radice (e i nodi intermedi) per una specifica entità.

In questo modo, ogni host gestisce solo una piccola frazione delle entità totali, distribuendo il carico in modo uniforme.

Sicurezza nel flat naming

Senza misure specifiche, un client deve fidarsi che il sistema di risoluzione restituisca l'indirizzo corretto associato a quel nome.

Per affrontare questo problema, esistono due strategie fondamentali: rendere l'associazione tra identificatore ed entità intrinsecamente sicura oppure rendere sicuro l'intero processo di risoluzione.

Self-certifying names

Per garantire la sicurezza senza dover necessariamente "fidarsi" del server che risolve il nome si usano i **self-certifying names** (nomi auto-certificanti). L'idea è che il nome stesso contenga le prove della sua validità.

Per risorse che non cambiano nel tempo, come un file di sola lettura, l'identificatore viene generato applicando una funzione di hash al contenuto stesso della risorsa:

$$\text{id(entità)} = \text{hash}(\text{dati dell'entità})$$

Una volta ricevuta l'entità dal sistema di risoluzione, il client calcola autonomamente l'hash del contenuto ricevuto, se corrisponde al nome cercato allora l'entità è autentica.

Se l'entità può cambiare (come un indirizzo IP o un file modificabile), l'uso del semplice hash non basta. Si utilizza quindi la crittografia asimmetrica:

$$\text{id(entità)} = \text{public key(entità)}$$

Quando il client richiede la risoluzione di questo identificatore, il sistema restituisce l'entità e informazioni aggiuntive che permettono al client di verificare l'associazione (*es.: digest firmato con la chiave privata*).

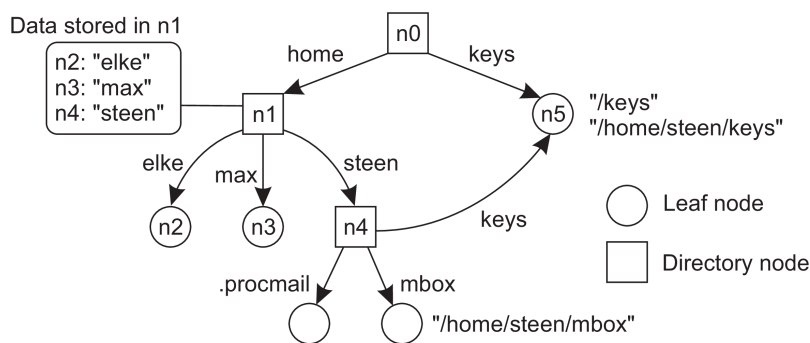
Structured Naming

Mentre i flat names sono identificatori casuali, i nomi strutturati sono composti da **stringhe di caratteri leggibili**, organizzati in modo da riflettere gerarchie o relazioni logiche, come avviene nei file system o nel DNS.

Name space

Per comprendere formalmente lo spazio dei nomi strutturato, lo rappresentiamo come un grafo diretto aciclico con etichette.

- **Nodi Foglia (Leaf Nodes)**: rappresentano l'entità finale (un file, un indirizzo IP, un oggetto). Non hanno archi uscenti e memorizzano le informazioni dell'entità o il suo indirizzo.
- **Nodi Direttorio (Directory Nodes)**: entità che hanno archi uscenti, ognuno con un'etichetta specifica. Al loro interno risiede una **directory table** che associa un'etichetta al corrispondente identificatore del nodo figlio.



Path names

Un nome di percorso è una sequenza di etichette che permette di navigare nel grafo.

- **Nome Assoluto**: parte dal **nodo radice** (root), che è l'unico nodo senza archi entranti.
- **Nome Relativo**: parte da un nodo specifico che funge da contesto iniziale.

Meccanismo di risoluzione dei nomi

La risoluzione è il processo che trasforma un nome strutturato (es. `/home/user/file.txt`) nel contenuto del nodo riferito.

🔗 Closure Mechanism

La risoluzione non può iniziare se non sappiamo da dove partire. Il meccanismo di chiusura seleziona il **nodo iniziale** del grafo.

Nel file system Unix, l'inode del root è in una posizione fissa del disco; in un browser, la "chiusura" è data dall'indirizzo IP del server DNS configurato nel sistema operativo.

Linking e mounting

Il linking permette di associare più nomi alla stessa entità, creando quelli che chiamiamo **alias**. Esistono due modalità principali per implementare questa funzione nel grafo di naming.

Hard Link

Due diversi percorsi assoluti puntano **direttamente allo stesso identificatore** di nodo nel grafo. Poiché entrambi i nomi si riferiscono direttamente all'identificatore del nodo, non esiste una gerarchia tra i due nomi. Se si rimuove uno dei nomi, l'altro continua a essere valido finché esiste almeno un percorso che porta a quel nodo.

Soft (Symbolic) Link

Un nodo foglia non contiene l'entità, ma **un altro nome di percorso**. Il risolutore, leggendo il soft link, ricomincia la risoluzione dal nuovo nome indicato.

Mounting

Il **mounting** permette di unire in modo trasparente spazi dei nomi diversi, spesso residenti su server fisici distinti, facendoli apparire come un'unica struttura logica all'utente.

- **Foreign Name Space**: lo spazio dei nomi esterno che si vuole integrare.
- **Mount Point**: il nodo nello spazio dei nomi corrente contenente l'identificatore del nodo dello spazio dei nomi esterno.
- **Mounting Point** (Punto di montaggio remoto): il nodo nello spazio dei nomi esterno (solitamente la radice) da cui prosegue la risoluzione.

Per effettuare il montaggio attraverso una rete, il sistema ha bisogno di tre informazioni fondamentali, ognuna delle quali deve essere risolta singolarmente:

1. **Nome del protocollo di accesso**: es. `nfs` per indicare come comunicare con il server.
2. **Nome del server**: risolto tramite sistemi come il DNS per ottenere l'indirizzo IP.
3. **Percorso del Mounting Point**: il nome del directory nel server remoto da cui iniziare la navigazione.

Implementazione name space

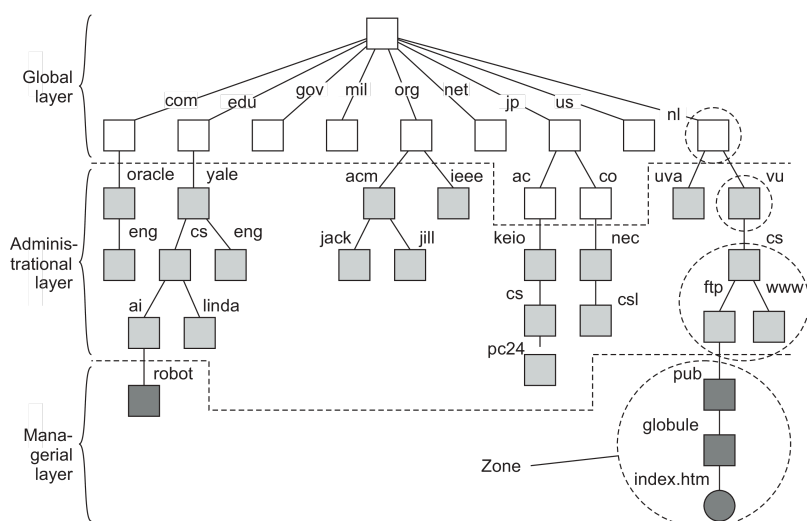
L'implementazione di un servizio di naming in un sistema distribuito su larga scala non può essere affidata a un singolo server, poiché diventerebbe rapidamente un collo di bottiglia e un singolo punto di

guasto.

Partizione logica dello spazio dei nomi

Per gestire nomi su scala mondiale, lo spazio dei nomi viene suddiviso gerarchicamente in tre strati logici distinti, ognuno con proprietà di stabilità e gestione differenti:

- **Global Layer:** formato dal nodo radice e dai suoi figli immediati (es. `.com`, `.it`, `.edu`). Questi nodi rappresentano organizzazioni o nazioni e sono caratterizzati da un'altissima stabilità: le tabelle dei nomi cambiano raramente.
- **Administrational Layer:** comprende i nodi gestiti da singole organizzazioni (es. `uniroma1.it` o `google.com`). Questi nomi sono meno stabili di quelli globali (vengono aggiunti nuovi dipartimenti o servizi), ma la loro gestione è centralizzata all'interno dell'organizzazione.
- **Managerial Layer:** contiene i nodi che cambiano con estrema frequenza, come i nomi degli host in una sottorete locale, i file nelle home directory degli utenti o i record degli utenti stessi. In questo strato, l'efficienza degli aggiornamenti è critica.



Requisiti di performance

Caratteristica	Strato Globale	Strato Amministrativo	Strato Manageriale
Scala geografica	Mondiale	Organizzazione	Dipartimento
Num. Nodi	Pochi	Molti	Vasti
Reattività (Lookup)	Secondi (accettabile)	Millisecondi	Immediata
Aggiornamenti	Pigri (Lazy)	Immediati	Immediati
Replicazione	Massiccia	Poca o nulla	Nulla
Caching lato client	Fondamentale	Utile	Talvolta efficace

Risoluzione iterativa

Il resolver del client interroga il server radice, che risolve la prima parte del nome e restituisce l'indirizzo del server successivo. Il resolver deve quindi contattare autonomamente il server indicato, ripetendo il processo fino al completamento.

Vantaggio: Allevia il carico sui server di alto livello (come i Root Server del DNS), che non devono gestire lo stato della ricerca per conto del client.

Risoluzione ricorsiva

Ogni server contattato si prende l'onere di interrogare il server successivo, aspettare la risposta e inoltrarla al richiedente.

- **Vantaggio 1 (Caching):** molto più efficace, poiché ogni server lungo il percorso impara e memorizza i risultati intermedi.
- **Vantaggio 2 (Costi di comunicazione):** se un client a San Francisco cerca un nome in Olanda, con la ricorsione il messaggio transatlantico viene inviato una sola volta, mentre con l'iterativa il client deve fare più viaggi "andata e ritorno" attraverso l'oceano.

Per assicurare la corretta gestione di un largo numero di richieste, supponiamo (almeno a livello globale e amministrativo) che il contenuto dei nodi non cambi quasi mai. Possiamo quindi applicare un'ampia replica mappando i nodi a più server e avviare la risoluzione del nome al server più vicino.

Un attributo importante di molti nodi è l'indirizzo dove l'entità rappresentata può essere contattata. La replica dei nodi rende i server dei nomi tradizionali su larga scala inadatti alla localizzazione di entità mobili.

Esempio: DNS

Il **Domain Name System (DNS)** è l'esempio più celebre di implementazione strutturata. Lo spazio dei nomi è diviso in **zone**, ovvero parti non sovrapposte gestite da server specifici.

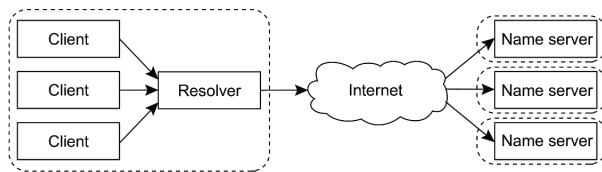
Lo spazio dei nomi DNS è organizzato come un **albero radicato** (*rooted tree*).

- **Etichette e Nomi di Dominio:** Ogni nodo ha un'etichetta (*label*) composta da caratteri alfanumerici. Il nome di dominio completo è la sequenza di etichette dalla foglia verso la radice, separate da punti. Ad esempio, `flits.cs.vu.nl.` termina con un punto che rappresenta esplicitamente la radice.
- **Domini:** Un sottotitolo dell'albero è chiamato dominio; un nome di percorso verso la radice di quel sottotitolo è il nome di dominio.

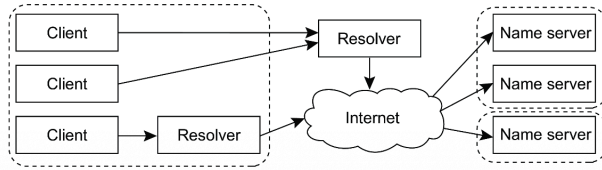
Record di Risorsa (RR): Ogni nodo nel database DNS contiene diversi tipi di dati:

- **A:** L'indirizzo IP di un host.
- **MX:** Il server di posta per quel dominio.
- **NS:** Il nome del name server per la zona.
- ...

I server Root del DNS sono solo 13 indirizzi logici, ma ognuno è implementato fisicamente come un cluster massicciamente replicato e distribuito in tutto il mondo per evitare che siano un *single point of failure*.



The traditional organization of the implementation of DNS



The modern organization of DNS

Secure DNS

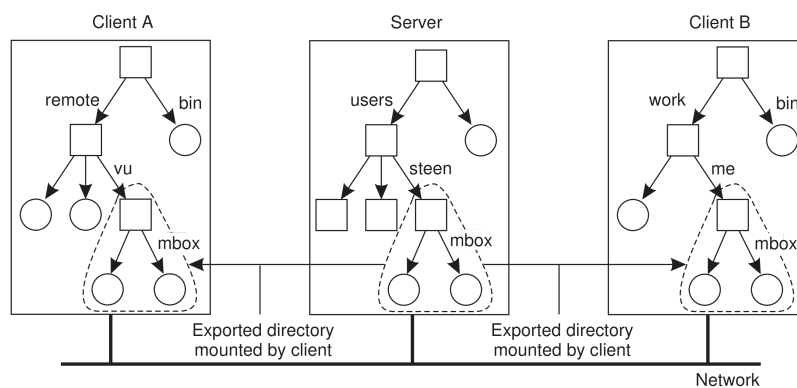
Il DNS tradizionale è vulnerabile perché un attaccante può intercettare una richiesta e inviare una risposta falsa prima del server legittimo (attacco di *cache poisoning*). Il DNSSEC è stato introdotto per garantire **integrità dei dati** ed **autenticazione dell'origine**.

Invece di firmare ogni singolo record, il DNSSEC raggruppa tutti i record dello stesso tipo in una zona (ad esempio, tutti gli indirizzi IPv4) in un unico set chiamato **RRset**. È questo set a venire firmato digitalmente.

Il sistema utilizza una gerarchia di chiavi basata su crittografia asimmetrica:

- **Zone-Signing Key (ZSK):** la chiave utilizzata dal proprietario della zona per firmare i set di record di risorse (RRset).
- **Key-Signing Key (KSK):** la ZSK stessa viene firmata da una seconda chiave, la KSK. Questo permette di cambiare la ZSK senza dover aggiornare le informazioni nel dominio padre.

Esempio: NFS



In NFS, ogni client costruisce il proprio spazio dei nomi in modo indipendente.

Se il Server esporta `/users/steen`, il **Client A** potrebbe montarlo sotto `/remote/vu`, mentre il **Client B** sotto `/work/me`. => Lo stesso file (es. `mbox`) avrà il percorso `/remote/vu/mbox` per il Client A e `/work/me/mbox` per il Client B. Questo rende difficile la condivisione di percorsi assoluti tra utenti su macchine diverse.

Mentre gli utenti usano nomi testuali, NFS opera internamente utilizzando i **File Handle**.

Il File Handle è un riferimento creato dal server quando il file viene generato, di cui il client non ne conosce il contenuto. Esso contiene informazioni necessarie al server per identificare univocamente il file all'interno dei propri dischi.

Attribute-based naming

Mentre il *flat naming* si concentra sull'identità univoca (l'ID) e lo *structured naming* sulla posizione gerarchica (il percorso), l'attribute-based naming permette di individuare un'entità partendo da una sua **descrizione**.

⚠ Problema di scalabilità

Il limite principale è che la ricerca è più costosa del semplice lookup. In un sistema centralizzato, la ricerca richiede scansioni esaustive. In un sistema distribuito su larga scala, inviare una query a migliaia di server è inefficiente.

In molti scenari, un utente non conosce il nome esatto o l'indirizzo di una risorsa, ma sa quali caratteristiche essa debba possedere.

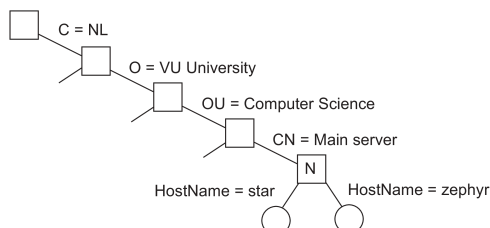
Nell'attribute-based naming, a ogni entità viene associata una collezione di attributi. Un attributo è tipicamente una coppia (**attributo, valore**).

I sistemi che supportano questo tipo di denominazione sono comunemente chiamati **directory services**.

Implementazioni gerarchiche: LDAP

L'implementazione più celebre e diffusa di un servizio di directory è **LDAP** (*Lightweight Directory Access Protocol*). LDAP combina la potenza della ricerca per attributi con l'efficienza di una struttura gerarchica.

- **Directory Entry:** l'unità base, una collezione di attributi (es. `Country=NL`, `Locality=Amsterdam`).
- **DIB (Directory Information Base):** l'insieme di tutte le voci della directory.
- **RDN (Relative Distinguished Name):** attributo specifico usato per dare un nome univoco a una voce all'interno di un ramo della gerarchia.
- **DIT (Directory Information Tree):** l'organizzazione gerarchica (albero) di tutte le voci.



Implementazioni decentralizzate

Distributed Index

Si associa ogni attributo a un server specifico. Per una query con più attributi, il client contatta più server e interseca i risultati.

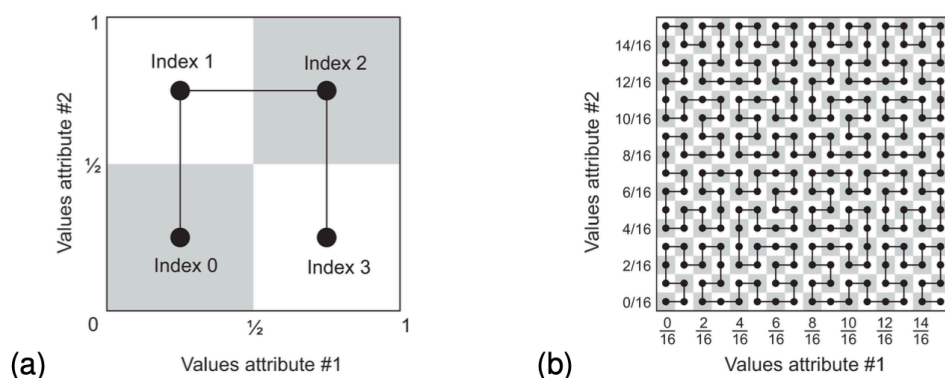
Svantaggio: se cerco un cognome comune come "Rossi", il server potrebbe restituirmi milioni di chiavi, sovraccaricando il client.

Space-filling Curves

È una tecnica avanzata per mappare uno spazio multi-dimensionale (N attributi) in una singola dimensione.

Ogni entità con valori $(v_1, v_2, \dots, v_n)$ viene vista come un punto in uno spazio a N dimensioni. Questo punto viene mappato su una curva che attraversa tutto lo spazio.

Una volta ottenuto un indice unidimensionale dalla curva, tale indice può essere gestito da una **DHT** (es. Chord). La ricerca di un intervallo di valori corrisponde ora a un segmento della curva, rendendo la ricerca distribuita molto più efficiente.



Named-data networking

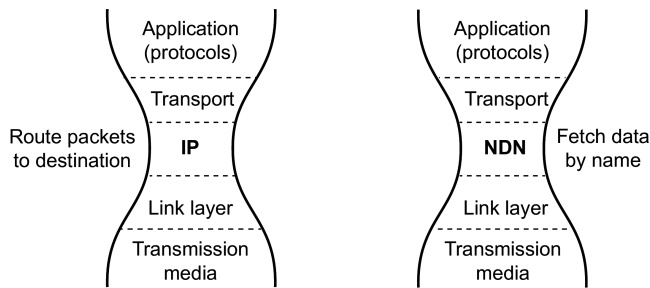
Il Named-Data Networking rappresenta una forma specifica e molto popolare di **Information-Centric Networking (ICN)**.

L'architettura attuale di Internet si basa sul protocollo IP, che è **host-centric** (incentrato sull'ospite). In questo modello, per ottenere una risorsa, un'applicazione deve prima conoscere l'indirizzo (IP) della macchina che la ospita.

Al contrario, il Named-Data Networking si basa sul principio che le applicazioni non sono realmente interessate a sapere *dove* un'entità sia memorizzata, ma piuttosto a ottenere una copia dei dati per accedervi localmente.

L'NDN sposta l'attenzione dall'indirizzo del nodo al **nome del dato** stesso.

Esempio di nome NDN: `/distributed-systems.net/books/Distributed Systems/4/01/Naming`.



Architettura di un router NDN

Per far funzionare questo sistema, i router NDN devono essere più complessi dei tradizionali router IP. Un router NDN è composto da tre elementi fondamentali:

1. **Content Store (CS):** una memoria cache. Il router restituisce immediatamente il dato se lo ha in memoria, senza inoltrare la richiesta oltre.
2. **Pending Interest Table (PIT):** una tabella che tiene traccia delle richieste ("Interest") attualmente in attesa. Se più utenti richiedono lo stesso dato contemporaneamente, il router registra le interfacce da cui provengono le richieste ma ne inoltra solo una verso la sorgente. Quando il dato torna indietro, il router usa la PIT per sapere a chi deve inviarlo.
3. **Forwarding Information Base (FIB):** simile a una tabella di instradamento tradizionale, ma invece di indirizzi IP, contiene **prefissi di nomi**. Indica al router verso quale interfaccia inoltrare una richiesta se il dato non è presente nel Content Store.

8. Fault Tolerance

Introduzione

La fault tolerance è la caratteristica che separa i sistemi distribuiti da quelli a macchina singola, proprio proprio per il concetto di **fallimento parziale**: una parte del sistema può guastarsi mentre il resto continua a operare correttamente.

Dependability

Un componente **fornisce servizi** ai client. Per fornire servizi, il componente può richiedere i servizi da altri componenti => un componente può dipendere da qualche altro componente.

Il concetto di **dependability** copre diversi requisiti fondamentali di un sistema distribuito:

- **Availability**: proprietà di un sistema di essere pronto all'uso immediato. Rappresenta la probabilità che il sistema operi correttamente in un dato istante temporale.
- **Reliability**: proprietà di un sistema di funzionare ininterrottamente senza guasti. A differenza della disponibilità, l'affidabilità è definita su un intervallo di tempo.
- **Safety**: capacità di un sistema di non generare situazioni catastrofiche in caso di guasto.
- **Maintainability**: con quanta facilità un sistema guasto può essere riparato.

Metriche tradizionali

1. **Mean Time To Failure (MTTF)**: tempo medio prima che un componente si guasti.
2. **Mean Time To Repair (MTTR)**: tempo medio necessario per riparare un componente dopo un guasto.
3. **Mean Time Between Failures (MTBF)**: la somma di MTTF + MTTR.

La **disponibilità a lungo termine (A)** può essere calcolata con la formula:

$$A = \frac{MTTF}{MTBF} = \frac{MTTF}{MTTF + MTTR}$$

Terminologia

Catena causale

Termine	Descrizione	Esempio
<u>Failure</u>	Si verifica quando il sistema non riesce a soddisfare le sue promesse o specifiche di servizio	Un programma che crasha
<u>Error</u>	Parte dello stato del sistema che può portare a un fallimento	Bug
<u>Fault</u>	La causa di un errore	Programmatore pigro

Gestione dei guasti

Termine	Descrizione
<u>Fault prevention</u>	Prevenire l'occorrenza di un guasto
<u>Fault tolerance</u>	Costruire un componente in grado di mascherare il guasto
<u>Fault removal</u>	Ridurre la presenza, numero o serietà di un guasto
<u>Fault forecasting</u>	Stimare la presenza attuale, incidenza futura e conseguenze dei guasti

Modelli di fallimento dei server

Tipo	Descrizione
<u>Crash failure</u>	Il server si arresta prematuramente ma ha funzionato correttamente fino al momento dell'arresto
<u>Omission failure</u>	Il server non risponde o non riceve le richieste o non riesce a inviare messaggi.
<u>Timing failure</u>	La risposta arriva, ma fuori da un intervallo di tempo specificato.
<u>Response failure</u>	La risposta è errata nel valore (Value failure) o nello stato (State-transition failure).
<u>Arbitrary (Byzantine) failure</u>	Il server produce risposte arbitrarie in tempi arbitrari; è il caso più grave e difficile da gestire

Dependability VS security

I fallimenti arbitrari a volte sono qualificati come **dannosi**. Facciamo la seguente distinzione:

- **Omission failure**: un componente fallisce non facendo un'azione che avrebbe dovuto fare.
- **Commission failure**: un componente fallisce facendo un'azione che non avrebbe dovuto fare.

-> I fallimenti deliberati, siano essi omissioni o fallimenti della commissione, sono in genere problemi di sicurezza.

Fermare i fallimenti

Sistemi sincroni VS asincroni

In un sistema **sincrono**, dove i tempi di consegna dei messaggi sono limitati, è possibile rilevare con certezza un crash tramite timeout. In un sistema **asincrono** (come Internet), è impossibile distinguere un server morto da uno estremamente lento.

Halting type	Descrizione
<u>Fail-stop</u>	Guasti di crash, ma rilevabili in modo affidabile
<u>Fail-noisy</u>	Guasti di crash, eventualmente rilevabili in modo affidabile

Halting type	Descrizione
<u>Fail-silent</u>	Omissione o guasti: i client non possono dire cos'è andato storto
<u>Fail-safe</u>	Fallimenti arbitrari, ma benigni
<u>Fail-arbitrary</u>	Arbitrari con fallimenti maligni

Ridondanza per mascherare i guasti

La tecnica chiave per ottenere la fault tolerance è la **ridondanza**. Ne esistono tre tipi:

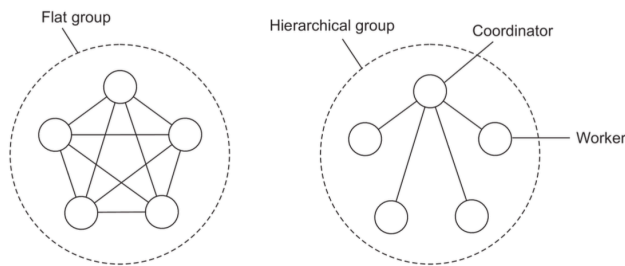
- **Ridondanza di informazione:** aggiunta di bit extra per recuperare dati corrotti (es. codici di Hamming).
- **Ridondanza temporale:** eseguire nuovamente un'azione se fallisce (es. ritrasmissione di pacchetti TCP o transazioni database).
- **Ridondanza fisica:** aggiungere hardware o processi extra. Un esempio classico è la **Triple Modular Redundancy (TMR)**, dove un componente è replicato tre volte e un "voter" decide il risultato in base alla maggioranza.

Resilienza dei processi

Il concetto chiave per ottenere la resilienza è la **ridondanza del software**: anziché fare affidamento su un singolo processo vulnerabile, utilizziamo un **gruppo di processi** identici che cooperano per apparire all'esterno come un'unica entità logica robusta.

Esistono due tipologie principali di organizzazione:

- **Gruppo Piatto (Flat Group):** tutti i processi sono uguali e non esiste un leader designato. Le decisioni vengono prese collettivamente, solitamente tramite votazione.
 - *Vantaggio:* è simmetrico e non presenta un *single point of failure* (se un processo cade, il gruppo si rimpicciolisce ma continua a operare).
 - *Svantaggio:* il processo decisionale è più complesso e costoso in termini di messaggi.
- **Gruppo Gerarchico:** esiste un coordinatore (leader) che riceve le richieste e le distribuisce ai "lavoratori" (worker).
 - *Vantaggio:* le decisioni sono rapide poiché il leader decide per tutti.
 - *Svantaggio:* se il coordinatore fallisce, l'intero gruppo si arresta finché non ne viene eletto uno nuovo tramite algoritmi di *leader election*.



Gruppi e mascheramento dei guasti

K-fault tolerance

Un sistema viene definito ***k*-fault tolerant** se può sopravvivere a guasti in ***k*** dei suoi componenti e continuare a soddisfare le proprie specifiche di servizio. La dimensione necessaria del gruppo dipende strettamente dal modello di guasto che vogliamo tollerare:

- Guasti di tipo Halting (Crash/Omission/Timing): se i processi falliscono silenziosamente (smettono semplicemente di rispondere), sono sufficienti $k + 1$ repliche per tollerare k guasti. Poiché i processi corretti non forniscono mai risultati errati, se k processi crashano, ne rimarrà almeno uno ($k + 1 - k = 1$) in grado di fornire la risposta corretta.
- Guasti Arbitrari (Bizantini): se i processi presentano fallimenti arbitrari (possono continuare a girare fornendo risposte errate, casuali o addirittura malevole), sono necessari almeno $2k + 1$ processi per tollerare k guasti.

Nel caso peggiore, i k processi guasti potrebbero colludere e fornire la stessa risposta errata. Per garantire che la maggioranza sia composta da processi onesti, dobbiamo avere almeno $k + 1$ processi corretti. Quindi: k (guasti) + $(k + 1)$ (corretti) = $2k + 1$.

Raggiungere il consenso

Affinché un gruppo di processi appaia come un unico server robusto, è necessario che tutti i membri non guasti concordino su quale operazione eseguire e in quale ordine. Questo è il problema del **consenso**.

Analizziamo i due protocolli più rilevanti per guasti di tipo crash.

Raft

Raft si basa su una gestione esplicita dei log e dei termini temporali.

Il tempo in Raft è diviso in **termini** numerati. Ogni termine inizia con un'elezione. Se un leader cade, inizia un nuovo termine con una nuova elezione. Questo meccanismo funge da orologio logico per rilevare informazioni obsolete.

Il funzionamento segue questi passaggi:

1. Il Leader riceve un comando dal client e lo appende al proprio log locale.
2. Il Leader invia il comando ai Follower.
3. Quando la **maggioranza** dei Follower conferma la ricezione, il Leader considera il comando "committed" (*confermato*), lo esegue e risponde al client.

4. Il Leader notifica i Follower dell'avvenuto commit nei messaggi successivi.

Per evitare che un nuovo leader perda dati, Raft impone che un server possa essere eletto solo se il suo log è più aggiornato (o uguale) rispetto alla maggioranza dei votanti. Se un leader ha divergenze con i follower, impone il proprio log sovrascrivendo quello dei follower.

Paxos

Paxos è un algoritmo robusto, progettato per sistemi parzialmente sincroni o asincroni dove i messaggi possono essere persi o ritardati ed i processi possono fallire interrompendosi, ma non si comportano in modo bizantino (non mentono volontariamente).

In un'esecuzione di Paxos, i processi logici si dividono in:

1. **Proposer**: riceve le richieste dai client e tenta di farle accettare al sistema. Solitamente un "leader" designato coordina il protocollo.
2. **Acceptor**: un'operazione è scelta solo se una maggioranza di Acceptor la accetta.
3. **Learner**: una volta che una proposta è stata scelta, i Learner la eseguono.

Utilizza i timestamp per distinguere le proposte e richiede che una maggioranza di Acceptor concordi su una proposta affinché diventi definitiva.

Paxos si articola in due fasi principali, ognuna suddivisa in due passaggi (a/b):

1. Fase 1 (Prepare & Promise):

- **1a (Prepare)**: il Proposer sceglie un timestamp unico t e lo invia agli Acceptor, chiedendo loro di non accettare più proposte con timestamp inferiori.
- **1b (Promise)**: se t è il timestamp più alto mai ricevuto, l'Acceptor promette di ignorare i futuri messaggi con timestamp minori. Se aveva già accettato una proposta in precedenza, la restituisce al Proposer.

2. Fase 2 (Accept & Learn):

- **2a (Accept)**: se il Proposer riceve promesse dalla maggioranza, invia un messaggio di `accept( $t$ ,  $o$ )`. Se nella Fase 1b gli è stata restituita un'operazione già esistente, deve "adottare" quella invece della propria.
- **2b (Learn)**: gli Acceptor accettano l'operazione (se non hanno nel frattempo promesso fedeltà a un timestamp maggiore) e informano i Learner.

In un sistema asincrono, è teoricamente impossibile distinguere con certezza assoluta un processo che è andato in crash da uno che è semplicemente estremamente lento o i cui messaggi sono stati ritardati dalla rete.

La tecnica standard utilizzata è l'uso di processi **heartbeat** e **timeout** per verificare la se il processo è vivo.

Paxos ha bisogno di almeno tre server, un server S non può eseguire un'operazione o fino a quando non ha ricevuto almeno un (altro) messaggio `LEARN( $o$ )`, in modo da sapere che la maggior parte dei server eseguirà o .

Cosa succede se la failure detection sbaglia? Immaginiamo che il Server 2 (S2) sospetti falsamente che il Leader (S1) sia crashato, mentre S1 è solo rallentato. S2 cercherà di diventare il nuovo leader, portando alla presenza di due "leader" contemporaneamente.

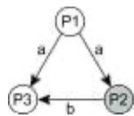
Per evitare il caos, Paxos introduce due meccanismi di difesa:

1. **Inclusione dell'ID del Leader:** ogni messaggio di `ACCEPT` deve includere l'identificativo del leader che lo ha generato. Se un nodo riceve un messaggio da un "vecchio" leader che credeva morto, può scartarlo perché sa che è stata già avviata un'elezione per un nuovo leader.
2. **Timestamp/Numeri di Proposta crescenti:** il nuovo leader utilizzerà un timestamp (o numero di proposta) più alto. Gli Acceptor, avendo promesso di ignorare timestamp inferiori durante la Fase 1b (Promise), bloccheranno automaticamente ogni tentativo del vecchio leader di far approvare vecchie operazioni.

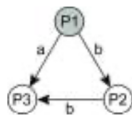
Consenso in caso di guasti arbitrari

Un **guasto arbitrario** si verifica quando un componente del sistema non si limita a interrompersi (come nei *crash failure*), ma continua a operare violando le proprie specifiche.

Consideriamo un gruppo di processi nel quale la comunicazione è **inconsistente**.



Forwarding errato



Messaggi differenti

Per poter affermare che un gruppo di processi ha raggiunto un consenso bizantino, devono essere soddisfatti due requisiti formali fondamentali, assumendo un processo primario P e $n-1$ backup B:

1. **BA1 (Accordo):** Ogni processo di backup non guasto deve memorizzare lo stesso valore.
2. **BA2 (Validità):** Se il processo primario è onesto, allora ogni backup non guasto deve memorizzare esattamente il valore inviato dal primario.

Se il primario è guasto, il requisito **BA1** impone comunque che i backup onesti si accordino su un unico valore, anche se diverso da quello inviato dal primario "bugiardo".

❗ Perché servono $3k+1$ processi?

Mentre per tollerare k crash failures sono sufficienti $k + 1$ processi, per gestire k guasti bizantini sono necessari almeno $3k + 1$ processi.

Perché un sistema con soli 3 nodi non può tollerare un singolo traditore? Consideriamo due scenari:

- **Scenario A (Primario guasto):** Il primario invia "Vero" a B1 e "Falso" a B2. Quando B1 e B2 si scambiano le informazioni per verificare, entrambi ricevono il set {Vero, Falso} e non possono sapere chi stia mentendo.
- **Scenario B (Backup guasto):** Il primario onesto invia "Vero" a tutti. Il backup B1 (guasto) dice a B2 di aver ricevuto "Falso". Anche in questo caso B2 riceve {Vero, Falso} e non può decidere.

Con 4 processi, se il primario è guasto e invia {Vero, Falso, Vero} ai tre backup, questi ultimi, scambiandosi i messaggi, otterranno tutti il set {Vero, Vero, Falso}. La maggioranza (2 su 3) è composta da valori onesti e coerenti, permettendo di raggiungere il consenso sul valore "Vero".

Practical Byzantine Fault Tolerance (PBFT)

Il protocollo **PBFT** è stata la prima soluzione pratica capace di gestire guasti bizantini con prestazioni accettabili. Si basa su un modello primario-backup e si articola in fasi per garantire che i nodi onesti concordino su un ordine totale delle operazioni.

Il modello di esecuzione è **parzialmente sincrono**, il server può presentare guasti arbitrari, i messaggi possono essere persi, ritardati o ricevuti out-of-order ed hanno un mittente identificabile.

Ha quattro fasi:

1. **Pre-prepare:** il primario assegna un numero di sequenza alla richiesta del client e la invia ai backup.
2. **Prepare:** ogni backup, se accetta la sequenza, invia un messaggio di "prepare" a tutti gli altri. Quando un nodo raccoglie $2k$ messaggi coerenti (più il proprio), ha la prova che la maggioranza concorda sull'ordine.
3. **Commit:** i nodi annunciano che sono pronti a eseguire l'operazione. Quando un nodo riceve $2k + 1$ conferme, esegue il comando e risponde al client.
4. **Reply:** Il client accetta il risultato solo se riceve almeno $k + 1$ risposte identiche da server diversi.

L'uso di quorum di $2k + 1$ su un totale di $3k + 1$ garantisce che due quorum diversi abbiano sempre un'intersezione di almeno $k + 1$ nodi, assicurando che in quell'intersezione ci sia almeno un nodo onesto che garantisce la coerenza.

Implementare la fault tolerance

Organizzare i processi replicati in un gruppo aiuta ad aumentare la tolleranza ai guasti, ma con una potenziale perdita di prestazioni. Molte volte la realizzazione non è sempre possibile.

Consenso distribuito: quando si può raggiungere

Process behavior	Message ordering				Commun. delay
	Unordered		Ordered		
	Unicast	Multicast	Unicast	Multicast	
Synchronous	✓	✓	✓	✓	Bounded
			✓	✓	Unbounded
Asynchronous				✓	Bounded
				✓	UnBounded
	Unicast	Multicast	Unicast	Multicast	
Message transmission					

Requisiti formali per il consenso sono:

- I processi producono lo stesso valore di output.
- Ogni valore di output deve essere valido.
- Ogni processo deve, alla fine, fornire output.

Consistenza, disponibilità, e partizionamento

🔗 Teorema di CAP

Il Teorema di CAP, stabilisce che un sistema distribuito che fornisce dati condivisi può garantire contemporaneamente solo due delle seguenti tre proprietà:

- **Consistency - C:** Ogni operazione di lettura su un dato replicato deve restituire il valore dell'operazione di scrittura più recente, facendo apparire il sistema come se esistesse un'unica copia sempre aggiornata.
- **Availability - A:** Il sistema deve essere in grado di rispondere a ogni richiesta ricevuta, garantendo che gli aggiornamenti vengano eventualmente eseguiti, anche se alcuni nodi sono lenti o irraggiungibili.
- **Partition Tolerance - P):** Il sistema continua a operare correttamente nonostante il verificarsi di partizioni della rete, ovvero interruzioni della comunicazione che dividono il gruppo di processi in sottogruppi isolati che non possono scambiarsi messaggi.

Quindi, in una rete soggetta a guasti nelle comunicazioni, è impossibile realizzare una lettura/scrittura atomica con memoria condivisa che garantisce una risposta a ogni richiesta.

Immaginiamo due processi, P e Q, che non possono più comunicare a causa di un guasto alla rete. Se un client invia un aggiornamento a P:

1. Se decidiamo di far procedere l'operazione su P per garantire la **disponibilità**, Q non verrà aggiornato, portando a una perdita di **coerenza**.
2. Se decidiamo di bloccare l'operazione finché P e Q non tornano a comunicare per garantire la **coerenza**, il sistema non risponde al client, sacrificando la **disponibilità**.
3. Se volessimo sia coerenza che disponibilità, dovremmo assumere che la partizione non avvenga mai, rinunciando alla **tolleranza alle partizioni**, il che è impossibile in un sistema geograficamente distribuito.

Pertanto, in presenza di un fallimento della comunicazione, è impossibile realizzare una memoria condivisa che garantisca una risposta a ogni richiesta e sia contemporaneamente atomica (coerente).

Rilevamento dei malfunzionamenti

Come rileviamo che un processo si è arrestato (crash)?

Ogni processo è dotato di un modulo di rilevamento dei malfunzionamenti.

Un processo P sonda un altro processo Q per una reazione, se Q reagisce è considerato vivo da P, altrimenti Q è sospettato di essersi arrestato.

Per i sistemi sincroni i tempi di esecuzione dei processi e i ritardi di consegna dei messaggi sono limitati e noti a priori. In questo scenario ideale, un sospetto di guasto equivale alla **certezza** del guasto: se un componente non risponde entro il tempo limite previsto, è garantito che sia andato in crash.

Implementazione

Se P non riceve un **heartbeat** da Q entro il tempo t : P sospetta di Q.

Se Q più tardi invia un messaggio (ricevuto da P), P smette di sospettare di Q ed aumenta il valore di timeout t .

Se Q è effettivamente in crash, P continua a sospettare.

Comunicazione affidabile client-server

In un mondo ideale, un client invia una richiesta e il server risponde istantaneamente e correttamente.

Nella realtà, ci scontriamo con i cosiddetti **guasti parziali**, dove componenti del sistema falliscono mentre altri continuano a operare.

RPC affidabile

Il modello di comunicazione più diffuso è la **Remote Procedure Call (RPC)**, che mira a far apparire una chiamata remota come se fosse locale. Tuttavia, quando sorgono guasti, la trasparenza diventa difficile da mantenere. Possiamo classificare i problemi in cinque categorie principali:

1. Il client non riesce a localizzare il server.
2. Il messaggio di richiesta dal client al server viene perso.
3. Il server va in crash dopo aver ricevuto una richiesta.
4. Il messaggio di risposta dal server al client viene perso.
5. Il client va in crash dopo aver inviato una richiesta.

Il client non riesce a localizzare il server

Può accadere se il server è spento o se la sua interfaccia è cambiata (versioning).

Soluzione: Il sistema solleva un'eccezione o un segnale (come `SIGNOSERVER` in C).

Questo distrugge la trasparenza. In una chiamata locale, non scriveresti mai un gestore per "procedura non trovata" a runtime se il codice è compilato correttamente.

Il messaggio di richiesta dal client al server viene perso

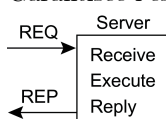
È il guasto più semplice da gestire: il sistema operativo o lo stub del client avviano un **timer** dopo l'invio. Se scade, la richiesta viene reinviata.

Crash del server

Il server può crashare *dopo* aver ricevuto la richiesta ma *prima* di eseguirla, oppure *dopo* averla eseguita ma *prima* di rispondere. Il client non può distinguere i due casi: vede solo che il timer è scaduto. Esistono tre filosofie di gestione:

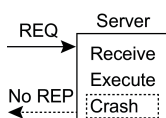
1. **At-least-once (Almeno una volta):** Il client continua a provare finché non ottiene risposta.

Garantisce l'esecuzione, ma rischia duplicati.

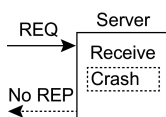


2. **At-most-once (Al massimo una volta):** Il client rinuncia immediatamente e riporta un errore.

Nessun rischio di duplicati, ma l'operazione potrebbe non essere avvenuta.



3. **Nessuna garanzia:** Il client non riceve aiuto. L'operazione può essere stata eseguita da zero a molte volte.



L'impossibilità del "Exactly-once": È teoricamente dimostrato che non è possibile garantire che un server esegua un'operazione *esattamente* una volta in modo totalmente trasparente dopo un crash. Ad esempio, in un sistema che stampa documenti, se il server crasha, il client non saprà mai se il documento è finito nel buffer di stampa un istante prima o dopo il guasto.

Reissue strategy	Strategy M → P			Strategy P → M		
	MPC	MC(P)	C(MP)	PMC	PC(M)	C(PM)
Always	DUP	OK	OK	DUP	DUP	OK
Never	OK	ZERO	ZERO	OK	OK	ZERO
Only when ACKed	DUP	OK	ZERO	DUP	OK	ZERO
Only when not ACKed	OK	ZERO	OK	OK	DUP	OK

Client

Server

OK

DUP

ZERO

=

Document processed once

=

Document processed twice

=

Document not processed at all

Server

Perdita dei messaggi di reply

Il server ha eseguito l'operazione, ma la risposta non arriva al client.

Se l'operazione è **idempotente** (può essere ripetuta più volte senza cambiare il risultato finale, come leggere un blocco di dati), il client può semplicemente ritrasmettere.

Se l'operazione non lo è (es. "trasferisci 100€"), la ritrasmissione causerebbe un danno.

Soluzione: Uso di **numeri di sequenza**. Il server tiene traccia dell'ultimo numero ricevuto da ogni client per identificare i duplicati e non rieseguire la logica, limitandosi a rispedire l'ultima risposta salvata.

Crash del client

Se il client crasha dopo aver inviato una richiesta, si genera un **orphan computation** (calcolo orfano): il server lavora e occupa risorse per un risultato che nessuno riceverà.

Metodi di gestione:

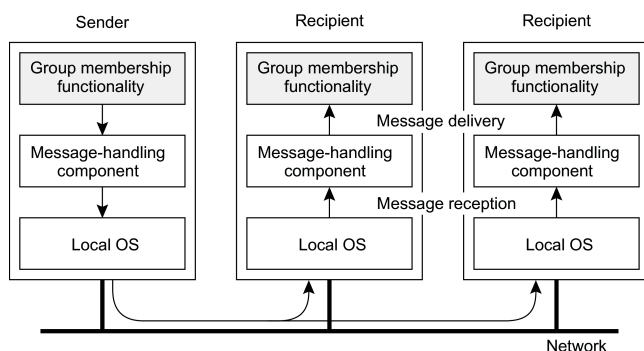
- **Extermination:** Uccidere esplicitamente l'orfano dopo il riavvio del client (costoso e difficile da tracciare).
- **Reincarnation:** Il client, al riavvio, trasmette un messaggio di "nuova epoca". Tutti i server uccidono i calcoli legati a epoche precedenti.
- **Expiration:** Ogni RPC riceve un quanto di tempo T . Se non finisce o non viene rinnovato, viene ucciso dal server.

Comunicazione affidabile di un gruppo

Vediamo innanzitutto la differenza tra **ricezione** e **consegna** (*delivery*) di un messaggio.

- **Ricezione:** quando il componente di gestione dei messaggi (livello middleware o kernel) preleva i dati dalla rete e li inserisce in un buffer locale.
- **Consegna:** quando il middleware passa effettivamente il messaggio all'applicazione sovrastante affinché venga elaborato.

Il sistema potrebbe *ricevere* un messaggio ma decidere di non *consegnarlo* subito (o affatto) per rispettare vincoli di ordine o di atomicità (ad esempio, se il mittente è crashato prima di terminare l'invio a tutto il gruppo).



La comunicazione di gruppo è **affidabile** quando si può garantire che un messaggio sia ricevuto e successivamente consegnato a tutti i membri del gruppo non difettosi.

Inoltre si può ridurre a un **multicast affidabile**: il messaggio è consegnato e ricevuto da tutti.

Commit distribuito

In un sistema distribuito, spesso dobbiamo garantire che un'operazione venga eseguita da **tutti** i membri di un gruppo di processi o da **nessuno**. Questa proprietà è nota come **atomicità**.

Il problema del commit distribuito nasce dalla necessità di far convergere processi indipendenti, che comunicano su canali potenzialmente inaffidabili, verso una decisione unanime.

Protocollo Two-Phase Commit (2PC)

Il protocollo **Two-Phase Commit** è la soluzione standard più diffusa. Divide l'operazione in due fasi distinte: una di votazione e una di decisione.

Ruoli del protocollo:

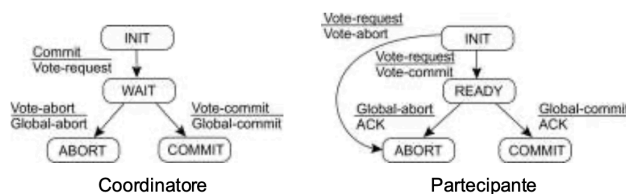
- **Coordinatore:** il processo che avvia l'operazione (spesso il client).
- **Partecipanti:** i processi che devono eseguire l'azione locale.

1. Fase 1 (Votazione):

- Il coordinatore invia un messaggio di `VOTE_REQUEST` a tutti i partecipanti.
- Ogni partecipante esegue il lavoro internamente (ma non lo rende definitivo) e risponde con `VOTE_COMMIT` (se è pronto) o `VOTE_ABORT` (se ha fallito).

2. Fase 2 (Decisione):

- Il coordinatore raccoglie i voti. Se **tutti** hanno votato COMMIT, invia un messaggio di `GLOBAL_COMMIT`.
- Se anche solo un partecipante ha votato ABORT (o se scade un timeout), il coordinatore invia `GLOBAL_ABORT`.
- I partecipanti ricevono l'ordine finale e rendono definitive le modifiche (commit) o annullano tutto (rollback).



Per permettere il recupero dopo un crash, sia il coordinatore che i partecipanti devono registrare i cambiamenti di stato in un **log su memoria persistente** prima di inviare i messaggi.

- Se un partecipante crasha dopo aver votato COMMIT (stato **READY**), al riavvio deve consultare il log per sapere cosa ha deciso il coordinatore.

Il 2PC è un protocollo **bloccante**. Se il coordinatore fallisce mentre tutti i partecipanti sono nello stato **READY** (hanno votato ma non hanno ancora ricevuto la decisione finale), i partecipanti non possono decidere autonomamente. Devono attendere il ripristino del coordinatore perché non sanno se qualcuno ha già ricevuto un ordine di commit o di abort.

Recovery

L'obiettivo fondamentale della recovery è sostituire uno stato errato (una parte dello stato del sistema che può portare a un fallimento) con uno **stato privo di errori**:

- **Forward Error Recovery**: quando il sistema entra in uno stato errato, si tenta di portarlo in un nuovo stato corretto dal quale possa continuare l'esecuzione.
- **Backward Error Recovery**: riportare il sistema ad uno stato precedente privo dell'errore.

Recovery consistente

Per effettuare il backward recovery, dobbiamo salvare lo stato del sistema in un cosiddetto **checkpoint**. In un sistema distribuito, non basta che ogni processo salvi il proprio stato; l'insieme dei checkpoint deve formare uno **stato globale coerente** (o *distributed snapshot*)

La recovery line è la collezione più recente di checkpoint locali che costituisce uno stato globale coerente. In questo stato, se un processo P ha registrato la ricezione di un messaggio, deve esistere un processo Q che ne ha registrato l'invio.

Checkpointing coordinato

Tutti i processi sincronizzano le proprie azioni per scrivere congiuntamente il proprio stato su una memoria stabile.

Si usa un protocollo di blocking in due fasi:

- Un coordinatore invia in multicast un messaggio di **checkpoint request**.
- Quando un partecipante riceve un messaggio del genere, effettua il checkpoint, smette di inviare messaggi e conferma di aver effettuato un checkpoint.
- Quando tutti i checkpoint sono stati confermati dal coordinatore, quest'ultimo trasmette un messaggio di **checkpoint done** per permettere agli altri processi di continuare.

È possibile considerare solo quei processi che dipendono dal recupero del coordinatore e ignorare il resto.

Rollback a cascata

Se il checkpoint viene effettuato nei momenti "sbagliati", la linea di recupero potrebbe trovarsi al momento dell'avvio del sistema. Abbiamo un cosiddetto **rollback a cascata**.

Checkpointing indipendente

Ogni processo salva il proprio stato in modo autonomo e non coordinato, tuttavia se i checkpoint sono presi nei momenti "sbagliati", il ripristino di un processo potrebbe invalidare lo stato di un altro, portando rollback a cascata.

Message logging

Poiché il checkpointing è un'operazione costosa (scrittura su disco), molti sistemi lo combinano con il **message logging**. L'idea è salvare i messaggi scambiati tra un checkpoint e l'altro per poter "ripassare" (replay) l'esecuzione dopo un crash

Il logging funziona se assumiamo che l'esecuzione di un processo sia una serie di intervalli che iniziano con un **evento non deterministico** (come la ricezione di un messaggio) seguiti da passi **completamente deterministici**. Se registriamo tutti gli eventi non deterministici, possiamo ricostruire esattamente lo stato del processo partendo da un checkpoint vecchio.

Un **processo orfano** è un processo superstite il cui stato è incoerente con quello di un processo crashato dopo il suo ripristino. Ciò accade se un messaggio inviato prima del crash non viene registrato e quindi non viene "ripassato" durante la recovery.

- **DEP(m)** : processi ai quali m è stato consegnato. Se m^* è causalmente dipendente da m e m^* è stato consegnato a Q , allora $Q \in DEP(m)$.
- **COPY(m)** : processi che posseggono una copia di m , ma non lo hanno ancora salvato.
- **FAIL** : collezione dei processi in crash.

$$Q \text{ è orfano} \iff \exists m : Q \in DEP(m) \text{ e } COPY(m) \subseteq FAIL$$

Esistono due approcci principali per evitare processi orfani:

- **Pessimistic Logging (Pessimistico)**: Si garantisce che ogni messaggio non stabile (non ancora su disco) sia consegnato ad **al massimo un** processo. Un messaggio deve essere reso stabile prima che il processo possa inviarne uno successivo.
- **Optimistic Logging (Ottimistico)**: Si effettua il logging in modo asincrono, sperando che non ci siano crash. Se avviene un crash e si creano orfani, questi vengono individuati e costretti al rollback.

9. Sicurezza

Introduzione alla sicurezza

Dependability

Un sistema "dependable" fornisce:

1. **Disponibilità**
2. **Affidabilità**
3. **Sicurezza**
4. **Manutenibilità**
5. **Confidenzialità**: informazioni divulgate solo alle parti autorizzate.
6. **Integrità**: le modifiche agli asset possono essere apportate solo in modo autorizzato e con completezza.

Possiamo definire un sistema distribuito come sicuro se protegge i propri servizi e dati contro tre classi fondamentali di **minacce**:

1. Divulgazione non autorizzata di informazioni (*confidenzialità*).
2. Modifica non autorizzata delle informazioni (*integrità*).
3. Interruzione del servizio (DOS) (*disponibilità*).

Meccanismi di sicurezza

- **Crittografia**: trasformazione dei dati in un formato non leggibile dall'attaccante.
- **Autenticazione**: verificare un'identità dichiarata.
- **Autorizzazione**: controllare se un'entità ha diritti corretti per l'accesso alle risorse.
- **Monitoring e Auditing**: tracciare gli accessi alle risorse.

Principi di sicurezza

Progettare la sicurezza "dopo il fatto" è quasi sempre un fallimento. Esistono principi guida che rimangono validi tutt'oggi:

- **Fail-safe defaults**: L'accesso deve basarsi su permessi espliciti piuttosto che su esclusioni. Se un meccanismo fallisce, il sistema deve risultare protetto (es. negare l'accesso di default).
- **Open design**: La sicurezza non deve basarsi sull'oscurità (*security by obscurity*). Il design deve essere pubblico e testabile; solo le chiavi crittografiche devono rimanere segrete.
- **Separation of privilege**: Le funzioni critiche dovrebbero richiedere più di una credenziale (es. due amministratori diversi per autorizzare una transazione importante).

- **Least privilege:** Ogni processo o utente deve operare con il set minimo di privilegi necessari per completare il proprio compito.
- **Least common mechanism:** Evitare di condividere meccanismi comuni tra troppi utenti, per limitare i canali di compromissione.

Trust Computing Base

L'insieme di tutti i meccanismi di sicurezza in un sistema informatico (distribuito) che sono necessari e sufficienti per far rispettare una politica di sicurezza.

Privacy

La **privacy** può essere invasa, mentre la **riservatezza** può essere violata. Garantire la riservatezza non è sufficiente per garantire la privacy.

Il diritto alla privacy riguarda "il diritto di un corretto flusso di informazioni personali". Controlla chi può vedere cosa, quando e come.

Una persona dovrebbe essere in grado di fermare e revocare il flusso delle proprie informazioni personali.

Il **GDPR (General Data Protection Regulation)** è un insieme di norme e regolamenti atti a proteggere i dati personali.

Crittografia

La crittografia è la tecnica di trasformare un messaggio (testo in chiaro, *plaintext*) in una forma intelligibile (testo cifrato, *ciphertext*) tramite una chiave.

Sistemi crittografici

- **Crittografia Simmetrica:** Mittente e destinatario condividono la stessa chiave segreta per cifrare e decifrare. È veloce ma presenta il problema della distribuzione sicura delle chiavi.
- **Crittografia Asimmetrica (Chiave Pubblica):** Ogni entità ha una coppia di chiavi: una **chiave pubblica** (PK), nota a tutti, e una **chiave privata** (SK), segreta.
 - Per la *riservatezza*: si cifra con la chiave pubblica del destinatario.
 - Per l'*autenticazione*: si cifra con la propria chiave privata.

Funzioni di hash

Una **funzione di hash** trasforma un messaggio di lunghezza arbitraria in una stringa a lunghezza fissa. Le proprietà chiave sono:

1. **One-way:** Impossibile risalire al messaggio originale dall'hash.
2. **Resistenza alle collisioni:** Computazionalmente impossibile trovare due messaggi diversi che producano lo stesso hash.

La **firma digitale** si ottiene calcolando l'hash di un messaggio e cifrandolo con la propria chiave privata. Il destinatario ricalcola l'hash e lo confronta con quello decifrato tramite la chiave pubblica del mittente per verificarne l'autenticità e l'integrità.

Gestione delle chiavi

Il cuore della gestione delle chiavi è rispondere alla domanda: *come possono Alice e Bob scambiarsi una chiave in modo sicuro se il canale di comunicazione è inizialmente insicuro?*

Uno dei protocolli per stabilire una chiave segreta su un canale insicuro è lo **scambio di chiavi Diffie-Hellman**.

Diffie-Hellman key exchange

1. Alice e Bob concordano pubblicamente su due numeri grandi: un numero primo p e un generatore g .
2. Alice sceglie un numero segreto x e invia a Bob $g^x \mod p$.
3. Bob sceglie un numero segreto y e invia ad Alice $g^y \mod p$.
4. Alice calcola $(g^y)^x \mod p = g^{xy} \mod p$.
5. Bob calcola $(g^x)^y \mod p = g^{xy} \mod p$.

Entrambi ora possiedono la chiave condivisa $g^{xy} \mod p$, ma un attaccante che osserva il traffico non può derivare x o y a causa della difficoltà del calcolo del logaritmo discreto.

Diffie-Hellman da solo non previene l'attacco "man-in-the-middle". Sebbene Alice e Bob stabiliscano una chiave segreta, non sanno *con chi* la stanno stabilendo a meno che i messaggi non siano autenticati.

Autenticazione

L'autenticazione è il processo di verifica di un'identità dichiarata. Può basarsi su:

1. **Ciò che sai:** Password o PIN.
2. **Ciò che hai:** Token, smartphone o smart card.
3. **Ciò che sei:** Biometria (impronte, volto).
4. **Ciò che fai:** Pattern di digitazione o voce.

KDC

Per risolvere il problema della scalabilità (N utenti richiederebbero $N(N-1)/2$ chiavi), si utilizza un **Key Distribution Center (KDC)**. Ogni utente condivide una sola chiave segreta con il KDC.

Se Alice vuole parlare con Bob, chiede al KDC una chiave di sessione. Il KDC genera $K_{A,B}$ e la invia ad Alice in due forme:

1. Cifrata con la chiave di Alice ($K_{A,KDC}$).
2. All'interno di un **Ticket** cifrato con la chiave di Bob ($K_{B,KDC}$), che Alice dovrà poi girare a Bob.

Protocollo Needham-Schroeder

È il fondamento di Kerberos. Introduce l'uso di **nonce** (numeri casuali usati una sola volta) per prevenire attacchi di *replay*.

Senza di esso, un attaccante (Chuck) potrebbe intercettare un vecchio ticket e inviarlo di nuovo a Bob, facendogli credere di essere Alice. Inserendo un nonce nella richiesta originale, Alice può verificare che la risposta del KDC sia fresca e non un replay.

Fiducia in sistemi distribuiti

Fiducia

La **fiducia** è la garanzia che un'entità agirà secondo specifiche concordate.

L'autenticazione verifica l'identità, ma non garantisce che quell'identità si comporterà in modo onesto. Un sistema può autenticare perfettamente un utente, ma se quell'utente non è fidato, l'autenticazione ha scarso valore pratico.

- **Guasti Bizantini:** In un gruppo di processi che deve raggiungere il consenso (come in PBFT), non abbiamo bisogno di fidarci dei singoli membri. L'ipotesi di base è che alcuni processi possano comportarsi in modo arbitrario o malevolo; la fiducia viene spostata dal singolo individuo alla **specifica del gruppo**. Se il gruppo rispetta la condizione $n \geq 3k + 1$ (dove n sono i membri totali e k i falliti), il sistema nel suo complesso è fidato perché garantisce il corretto funzionamento nonostante i singoli traditori.

Se un gruppo di processi non soddisfa più le sue specifiche, si può dubitare della sua affidabilità, che è diversa rispetto alla mancanza di fiducia nel sistema.

Sybil attack

Uno dei problemi più complessi nei sistemi aperti (come i P2P) è l'**attacco Sybil**. In questo scenario, un singolo utente malintenzionato crea molteplici identità fittizie per ottenere un'influenza sproporzionata nel sistema (ad esempio, per alterare un voto o influenzare un algoritmo di reputazione).

Soluzioni per le blockchain

Le blockchain sono spesso descritte come sistemi "trustless", ma sarebbe più corretto dire che **spostano la fiducia** dai partecipanti umani e dalle istituzioni centrali agli **algoritmi crittografici** e al **consenso distribuito**.

- **Integrità del Dato:** La fiducia nella blockchain risiede nella sua struttura a catena, dove ogni blocco contiene l'hash del precedente. Qualsiasi modifica a un blocco passato renderebbe invalidi tutti i blocchi successivi, rendendo la manomissione immediatamente rilevabile.
- **Proof-of-Work (PoW):** I validatori gareggiano tra loro in un puzzle computazionale. Richiede risorse di calcolo.

- **Proof-of-Stake (PoS):** Scegliere un validatore in funzione del numero di token che possiede.
Richiede il rischio di perdere dei token.
- **Limiti della Fiducia:** Nonostante la tecnologia, rimane necessario fidarsi dell'**implementazione del software** e degli intermediari (es. exchange) che permettono l'accesso alle applicazioni blockchain.

Eclipse attack

Un **attacco Eclipse** mira ad isolare un nodo onesto facendogli conoscere solo vicini malevoli.

Per prevenire questi attacchi, si deve utilizzare una **certification authority** centralizzata.